Ganpat University

Department of Computer Science

M.SC(CA & IT) SEM – 1 Practical Documentation

Subject : MERN Stack Development

Academic Year : 2026–2027

Submitted by :

Name : **Prince Prajapati**

Enrolment Number : **26084231038**

Index of Practicals

Practical No.	Practical Title	Page No.
01	Build a React application using Functional Components, JSX and Component Reusability concepts.	3
02	Develop a React application implementing Props, State Management and Event Handling.	15
03	Create a Student Management System using React Forms, Controlled Components and Validation.	29
04	Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.	46
05	Create a backend server using Express.js implementing routing and CRUD operation.	63
06	Build RESTful APIs for Student Records Management using Express.js and Postman testing.	75
07	Develop an Employee Management API integrating Express.js with MongoDB Atlas.	86
08	Create a backend server using Express.js implementing routing, middleware and request handling.	99
09	Create a file upload module using Multer and store file metadata in MongoDB.	114
10	Develop a complete MERN application integrating React Frontend, Express Backend and MongoDB Database.	127
11	Create a Blog Management System implementing CRUD Operations, Authentication and Role-Based Access Control.	150
12	Develop a Real-Time Chat Application using Socket.IO, React and Node.js.	158
13	Mini Project: Design and develop a full-stack industry-oriented MERN application (Document Management System).	163

Practical – 01 : Build a React application using Functional Components, JSX and Component Reusability concepts.

1. Commands :

```
npm create vite@latest MERN_01
cd MERN_01
npm install
npm run dev
```

2. Project structure :

```
MERN_01/
|-- package.json
|-- index.html
|-- vite.config.js
|-- src/main.jsx
|-- src/index.css
|-- src/App.jsx
|-- src/components/Header.jsx
|-- src/components/Button.jsx
|-- src/components/Badge.jsx
|-- src/components/Card.jsx
|-- src/components/CourseCard.jsx
|-- src/components/Footer.jsx
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-01",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.2"
  }
}
```

- **index.html :**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Practical 01 - React Functional Components</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
```

```
  </body>
</html>
```

• **vite.config.js :**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: './',
  plugins: [react()],
});
```

• **src/main.jsx :**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

• **src/index.css :**

```
:root {
  --primary: #6366f1;
  --primary-hover: #4f46e5;
  --primary-light: #e0e7ff;
  --secondary: #0ea5e9;
  --accent: #f59e0b;
  --success: #10b981;
  --danger: #ef4444;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
  --radius-md: 12px;
  --radius-lg: 16px;
  --font-family: 'Inter', system-ui, -apple-system, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
  -webkit-font-smoothing: antialiased;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
```

```css
}

.main-content {
  flex: 1;
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
  width: 100%;
}

/* Header Component */
.header {
  background: linear-gradient(135deg, #1e1b4b 0%, #312e81 50%, #4338ca 100%);
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: var(--shadow-md);
  position: relative;
  overflow: hidden;
}

.header::after {
  content: '';
  position: absolute;
  top: -50%;
  right: -20%;
  width: 300px;
  height: 300px;
  background: radial-gradient(circle, rgba(255, 255, 255, 0.15) 0%, transparent 70%);
  border-radius: 50%;
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(10px);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 600;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
  letter-spacing: -0.025em;
}

.header-subtitle {
  font-size: 1.1rem;
  color: #c7d2fe;
  max-width: 650px;
  margin: 0 auto;
}

/* Section styling */
.section {
  margin-bottom: 3rem;
}

.section-title {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--text-main);
  margin-bottom: 0.5rem;
  display: flex;
```

```
  align-items: center;
  gap: 0.5rem;
}

.section-desc {
  color: var(--text-muted);
  font-size: 0.95rem;
  margin-bottom: 1.5rem;
}

/* Reusable Grid */
.grid-3 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 1.5rem;
}

.grid-2 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(450px, 1fr));
  gap: 1.5rem;
}

/* Reusable Card */
.custom-card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-lg);
  padding: 1.75rem;
  box-shadow: var(--shadow-sm);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  display: flex;
  flex-direction: column;
}

.custom-card:hover {
  transform: translateY(-4px);
  box-shadow: var(--shadow-lg);
}

.card-icon {
  width: 48px;
  height: 48px;
  border-radius: var(--radius-md);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.5rem;
  margin-bottom: 1rem;
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 0.5rem;
  color: var(--text-main);
}

.card-description {
  color: var(--text-muted);
  font-size: 0.925rem;
  line-height: 1.5;
  margin-bottom: 1.25rem;
  flex: 1;
}

.card-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
```

```
  margin-top: auto;
  padding-top: 1rem;
  border-top: 1px solid var(--border-color);
}

/* Reusable Button */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  font-family: inherit;
  font-size: 0.9rem;
  font-weight: 600;
  padding: 0.6rem 1.25rem;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  text-decoration: none;
}

.btn-primary {
  background: var(--primary);
  color: white;
}

.btn-primary:hover {
  background: var(--primary-hover);
  box-shadow: 0 4px 12px rgba(99, 102, 241, 0.35);
}

.btn-secondary {
  background: #f1f5f9;
  color: #334155;
  border: 1px solid var(--border-color);
}

.btn-secondary:hover {
  background: #e2e8f0;
}

.btn-success {
  background: var(--success);
  color: white;
}

.btn-success:hover {
  background: #059669;
}

.btn-sm {
  padding: 0.4rem 0.85rem;
  font-size: 0.8rem;
}

/* Reusable Badge */
.badge {
  display: inline-flex;
  align-items: center;
  padding: 0.25rem 0.75rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge-primary {
  background: var(--primary-light);
```

```
  color: var(--primary);
}

.badge-success {
  background: #d1fae5;
  color: #065f46;
}

.badge-warning {
  background: #fef3c7;
  color: #92400e;
}

.badge-info {
  background: #e0f2fe;
  color: #0369a1;
}

/* Info Box */
.concept-box {
  background: #ffffff;
  border-left: 4px solid var(--primary);
  border-radius: 0 var(--radius-md) var(--radius-md) 0;
  padding: 1.25rem 1.5rem;
  margin-bottom: 2rem;
  box-shadow: var(--shadow-sm);
}

.concept-box h3 {
  font-size: 1.1rem;
  color: var(--primary);
  margin-bottom: 0.5rem;
}

.concept-box p {
  color: var(--text-muted);
  font-size: 0.95rem;
}

/* Footer */
.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
}

.footer strong {
  color: #f8fafc;
}
```

- **src/App.jsx:**

```
import React from 'react';
import Header from './components/Header';
import Footer from './components/Footer';
import Card from './components/Card';
import CourseCard from './components/CourseCard';
import Button from './components/Button';
import Badge from './components/Badge';

function App() {
  // Sample data to demonstrate component reusability with JSX and props
  const coreConcepts = [
    {
      id: 1,
      icon: '⚛',
```

```
      title: 'Functional Components',
      description: 'JavaScript functions that accept props as an argument and return JSX. They are
simpler, lightweight, and modern standard in React.',
      category: 'React Core',
    },
    {
      id: 2,
      icon: '✨',
      title: 'JSX Syntax',
      description: 'JavaScript XML allows writing HTML-like syntax inside JavaScript, transpiled to
React.createElement() calls for high readability.',
      category: 'Syntax',
    },
    {
      id: 3,
      icon: '🧩',
      title: 'Component Reusability',
      description: 'Building isolated, reusable UI blocks (Buttons, Cards, Badges) that receive dynamic
props to adapt across different application views.',
      category: 'Architecture',
    }
  ];

  const courses = [
    {
      id: 'CS101',
      courseCode: 'REACT-101',
      title: 'Modern React with JSX',
      instructor: 'Dr. John Doe',
      duration: '6 Weeks',
      level: 'Beginner',
      rating: 4.8
    },
    {
      id: 'CS102',
      courseCode: 'NODE-201',
      title: 'Backend Mastery with Express',
      instructor: 'Prof. Jane Smith',
      duration: '8 Weeks',
      level: 'Intermediate',
      rating: 4.9
    },
    {
      id: 'CS103',
      courseCode: 'MERN-301',
      title: 'Full Stack MERN Architecture',
      instructor: 'Alex Johnson',
      duration: '12 Weeks',
      level: 'Advanced',
      rating: 4.7
    }
  ];

  const handleEnroll = (courseTitle) => {
    alert(`⚛️ Successfully enrolled in: ${courseTitle}`);
  };

  const handleLearnMore = (conceptTitle) => {
    alert(`⚛️ Exploring details for: ${conceptTitle}`);
  };

  return (
    <div className="app-container">
      {/* 1. Header Functional Component */}
      <Header
        practicalNo="Practical No. 01"
        title="Functional Components, JSX & Reusability"
        subtitle="Demonstrating React Functional Component Architecture, JSX templating, and reusable UI
components"
      />
```

```
      <main className="main-content">
        {/* Concept Introduction Box */}
        <div className="concept-box">
          <h3>⚛️ Practical Objective</h3>
          <p>
            Build a clean React application utilizing Functional Components, JSX expressions, and modular
            Component Reusability. Reusable components like <code>Button</code>, <code>Badge</code>,
<code>Card</code>,
            and <code>CourseCard</code> are reused throughout the interface with varying props.
          </p>
        </div>

        {/* Section 1: Reusable Core Concept Cards */}
        <section className="section">
          <h2 className="section-title">
            <span>⚛️</span> Core React Concepts (Reusable Card Component)
          </h2>
          <p className="section-desc">
            The same <code>&lt;Card /&gt;</code> component is reused 3 times with different props data.
          </p>

          <div className="grid-3">
            {coreConcepts.map((item) => (
              <Card
                key={item.id}
                icon={item.icon}
                title={item.title}
                description={item.description}
                category={item.category}
                actionLabel="Explore Concept"
                onAction={() => handleLearnMore(item.title)}
              />
            ))}
          </div>
        </section>

        {/* Section 2: Reusable Course Cards */}
        <section className="section">
          <h2 className="section-title">
            <span>📚</span> Available Courses (Reusable CourseCard Component)
          </h2>
          <p className="section-desc">
            Demonstrating nested reusable components (<code>&lt;Badge /&gt;</code> and <code>&lt;Button
/&gt;</code> inside <code>&lt;CourseCard /&gt;</code>).
          </p>

          <div className="grid-3">
            {courses.map((course) => (
              <CourseCard
                key={course.id}
                courseCode={course.courseCode}
                title={course.title}
                instructor={course.instructor}
                duration={course.duration}
                level={course.level}
                rating={course.rating}
                onEnroll={handleEnroll}
              />
            ))}
          </div>
        </section>

        {/* Section 3: Standalone Reusable UI Elements */}
        <section className="section">
          <h2 className="section-title">
            <span>🎛️</span> Reusable UI Buttons & Badges Showcase
          </h2>
          <p className="section-desc">
            Buttons and Badges rendering with different variant properties.
```

```
          </p>

          <div className="custom-card" style={{ gap: '1.25rem' }}>
            <div>
              <h4 style={{ marginBottom: '0.5rem', color: 'var(--text-muted)' }}>Button Variants:</h4>
              <div style={{ display: 'flex', gap: '0.75rem', flexWrap: 'wrap' }}>
                <Button variant="primary" onClick={() => alert('Primary Clicked')}>Primary
Button</Button>
                <Button variant="secondary" onClick={() => alert('Secondary Clicked')}>Secondary
Button</Button>
                <Button variant="success" onClick={() => alert('Success Clicked')}>Success
Button</Button>
                <Button variant="primary" size="sm" onClick={() => alert('Small Clicked')}>Small
Button</Button>
              </div>
            </div>

            <div>
              <h4 style={{ marginBottom: '0.5rem', color: 'var(--text-muted)' }}>Badge Variants:</h4>
              <div style={{ display: 'flex', gap: '0.75rem', flexWrap: 'wrap' }}>
                <Badge text="Primary Tag" variant="primary" />
                <Badge text="Active Status" variant="success" />
                <Badge text="Pending Review" variant="warning" />
                <Badge text="Informational" variant="info" />
              </div>
            </div>
          </div>
        </section>
      </main>

      {/* Footer Functional Component */}
      <Footer />
    </div>
  );
}

export default App;
```

• **src/components/Header.jsx :**

```
import React from 'react';

/**
 * Functional Component: Header
 * Demonstrates Functional component syntax, JSX and props usage.
 */
function Header({ title, subtitle, practicalNo }) {
  return (
    <header className="header">
      {practicalNo && <span className="header-badge">{practicalNo}</span>}
      <h1 className="header-title">{title}</h1>
      <p className="header-subtitle">{subtitle}</p>
    </header>
  );
}

export default Header;
```

• **src/components/Button.jsx :**

```
import React from 'react';

/**
 * Reusable Functional Component: Button
 * Reusable across different variants, sizes and click events.
 */
function Button({ children, variant = 'primary', size = 'md', onClick, icon }) {
  const sizeClass = size === 'sm' ? 'btn-sm' : '';
```

```
  const variantClass = `btn-${variant}`;
  return (
    <button className={`btn ${variantClass} ${sizeClass}`} onClick={onClick}>
      {icon && <span className="btn-icon">{icon}</span>}
      {children}
    </button>
  );
}

export default Button;
```

- **src/components/Badge.jsx :**

```
import React from 'react';

/**
 * Reusable Functional Component: Badge
 * Demonstrates conditional class names and reusable tag rendering.
 */
function Badge({ text, variant = 'primary' }) {
  return (
    <span className={`badge badge-${variant}`}>
      {text}
    </span>
  );
}

export default Badge;
```

- **src/components/Card.jsx :**

```
import React from 'react';
import Badge from './Badge';
import Button from './Button';

/**
 * Reusable Functional Component: Feature Card
 * Demonstrates component nesting and JSX reusability
 */
function Card({ icon, title, description, category, actionLabel = "Learn More", onAction }) {
  return (
    <div className="custom-card">
      <div className="card-icon" style={{ background: '#eef2ff' }}>
        {icon}
      </div>
      <div style={{ marginBottom: '0.5rem' }}>
        <Badge text={category} variant="primary" />
      </div>
      <h3 className="card-title">{title}</h3>
      <p className="card-description">{description}</p>
      <div className="card-footer">
        <Button variant="primary" size="sm" onClick={onAction}>
          {actionLabel}
        </Button>
      </div>
    </div>
  );
}

export default Card;
```

- **src/components/CourseCard.jsx :**

```
import React from 'react';
import Badge from './Badge';
import Button from './Button';
```

```
/**
 * Reusable Course Card component
 */
function CourseCard({ courseCode, title, instructor, duration, level, rating, onEnroll }) {
  const badgeVariant = level === 'Beginner' ? 'success' : level === 'Intermediate' ? 'info' : 'warning';
  return (
    <div className="custom-card">
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'1rem' }}>
        <span style={{ fontSize: '0.85rem', fontWeight: '700', color: 'var(--primary)' }}>
          {courseCode}
        </span>
        <Badge text={level} variant={badgeVariant} />
      </div>

      <h3 className="card-title">{title}</h3>

      <div style={{ margin: '0.75rem 0', fontSize: '0.9rem', color: 'var(--text-muted)' }}>
        <p>👨‍🏫 <strong>Instructor:</strong> {instructor}</p>
        <p>⏱️ <strong>Duration:</strong> {duration}</p>
        <p>⭐ <strong>Rating:</strong> {rating} / 5.0</p>
      </div>
      <div className="card-footer">
        <Button variant="success" size="sm" onClick={() => onEnroll(title)}>
          Enroll Now
        </Button>
        <Button variant="secondary" size="sm" onClick={() => alert(`Syllabus for ${title}`)}>
          View Syllabus
        </Button>
      </div>
    </div>
  );
}

export default CourseCard;
```

• **src/components/Footer.jsx :**

```
import React from 'react';

/**
 * Functional Component: Footer
 */
function Footer() {
  const currentYear = new Date().getFullYear();
  return (
    <footer className="footer">
      <p>
        <strong>Web Technology Practicals Journal</strong> | Built with React.js & Functional Components
      </p>
      <p style={{ marginTop: '0.4rem', fontSize: '0.8rem', color: '#64748b' }}>
        Practical 1: Functional Components, JSX & Reusability &bull; &copy; {currentYear}
      </p>
    </footer>
  );
}

export default Footer;
```

4. Output :

The application demonstrates functional component architecture, JSX syntax expressions, and nested component reusability with modular Cards, Badges, and Buttons.

PRACTICAL NO. 01

Functional Components, JSX & Reusability

Demonstrating React Functional Component Architecture, JSX templating, and reusable UI components

Practical Objective

Build a clean React application utilizing Functional Components, JSX expressions, and modular Component Reusability. Reusable components like `Button`, `Badge`, `Card`, and `CourseCard` are reused throughout the interface with varying props.

Core React Concepts (Reusable Card Component)

The same `<Card />` component is reused 3 times with different props data.

REACT CORE

Functional Components

JavaScript functions that accept props as an argument and return JSX. They are simpler, lightweight, and modern standard in React.

SYNTAX

JSX Syntax

JavaScript XML allows writing HTML-like syntax inside JavaScript, transpiled to React.createElement() calls for high readability.

ARCHITECTURE

Component Reusability

Building isolated, reusable UI blocks (Buttons, Cards, Badges) that receive dynamic props to adapt across different application views.

Practical – 02 : Develop a React application implementing Props, State Management and Event Handling.

1. Commands :

```
npm create vite@latest MERN_02
cd MERN_02
npm install
npm run dev
```

2. Project structure :

```
MERN_02/
|-- package.json
|-- index.html
|-- vite.config.js
|-- src/main.jsx
|-- src/index.css
|-- src/App.jsx
|-- src/components/Header.jsx
|-- src/components/CounterSection.jsx
|-- src/components/TaskManager.jsx
|-- src/components/TaskItem.jsx
|-- src/components/UserProfileCard.jsx
|-- src/components/InteractiveBox.jsx
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-02",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.2"
  }
}
```

- **index.html :**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Practical 02 - React Props, State & Event Handling</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
```

```
  </body>
</html>
```

• **vite.config.js :**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: './',
  plugins: [react()],
});
```

• **src/main.jsx :**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

• **src/index.css :**

```
:root {
  --primary: #0284c7;
  --primary-dark: #0369a1;
  --primary-light: #e0f2fe;
  --accent: #8b5cf6;
  --success: #10b981;
  --danger: #ef4444;
  --warning: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --shadow-sm: 0 1px 3px 0 rgb(0 0 0 / 0.1);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
  --radius: 12px;
  --font-family: 'Plus Jakarta Sans', system-ui, -apple-system, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
```

```css
.main-content {
  flex: 1;
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
  width: 100%;
}

.header {
  background: linear-gradient(135deg, #0c4a6e 0%, #0369a1 50%, #0284c7 100%);
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: var(--shadow-md);
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(8px);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 700;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
}

.header-subtitle {
  font-size: 1.05rem;
  color: #bae6fd;
  max-width: 650px;
  margin: 0 auto;
}

.grid-2 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(400px, 1fr));
  gap: 1.75rem;
  margin-bottom: 2rem;
}

.card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: var(--shadow-sm);
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  color: var(--text-main);
  margin-bottom: 0.5rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.card-subtitle {
  font-size: 0.9rem;
  color: var(--text-muted);
  margin-bottom: 1.5rem;
```

```
}

/* Counter styles */
.counter-box {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 1.25rem;
  padding: 1.5rem;
  background: #f0f9ff;
  border: 1px dashed var(--primary);
  border-radius: var(--radius);
}

.counter-display {
  font-size: 3.5rem;
  font-weight: 800;
  color: var(--primary-dark);
  line-height: 1;
}

.counter-controls {
  display: flex;
  gap: 0.75rem;
  flex-wrap: wrap;
  justify-content: center;
}

/* Button UI */
.btn {
  padding: 0.6rem 1.25rem;
  font-size: 0.9rem;
  font-weight: 600;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  transition: all 0.2s ease;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-success { background: var(--success); color: white; }
.btn-success:hover { background: #059669; }
.btn-danger { background: var(--danger); color: white; }
.btn-danger:hover { background: #dc2626; }
.btn-warning { background: var(--warning); color: white; }
.btn-warning:hover { background: #d97706; }
.btn-outline { background: transparent; border: 1px solid var(--border-color); color: var(--text-main); }
.btn-outline:hover { background: #f1f5f9; }
.btn-sm { padding: 0.35rem 0.75rem; font-size: 0.8rem; }

/* Input controls */
.form-input {
  width: 100%;
  padding: 0.65rem 0.9rem;
  font-size: 0.95rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  outline: none;
  font-family: inherit;
  transition: border-color 0.2s;
}

.form-input:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(2, 132, 199, 0.15);
```

```css
}

/* Task list styles */
.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.85rem 1rem;
  background: #f8fafc;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  margin-bottom: 0.6rem;
  transition: all 0.2s ease;
}

.task-item.completed {
  background: #f1f5f9;
  opacity: 0.7;
}

.task-item.completed .task-text {
  text-decoration: line-through;
  color: var(--text-muted);
}

.task-left {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.task-checkbox {
  width: 18px;
  height: 18px;
  cursor: pointer;
  accent-color: var(--primary);
}

/* User Card Styles */
.user-preview {
  display: flex;
  gap: 1.25rem;
  align-items: center;
  padding: 1.25rem;
  border-radius: var(--radius);
  background: linear-gradient(135deg, #f0f9ff 0%, #e0f2fe 100%);
  border: 1px solid #bae6fd;
}

.user-avatar {
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--primary);
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.75rem;
  font-weight: 700;
  box-shadow: var(--shadow-sm);
}

.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
```

```
  border-top: 1px solid #1e293b;
}
```

- **src/App.jsx :**

```
import React from 'react';
import Header from './components/Header';
import CounterSection from './components/CounterSection';
import TaskManager from './components/TaskManager';
import UserProfileCard from './components/UserProfileCard';
import InteractiveBox from './components/InteractiveBox';

function App() {
  return (
    <div className="app-container">
      {/* Header */}
      <Header
        practicalNo="Practical No. 02"
        title="Props, State Management & Event Handling"
        subtitle="Practical implementation of Uni-directional Data Flow, useState Hook, and React
Synthetic Event Handlers"
      />

      <main className="main-content">
        {/* Info Box */}
        <div style={{
          background: '#ffffff',
          borderLeft: '4px solid var(--primary)',
          padding: '1.25rem 1.5rem',
          borderRadius: '0 12px 12px 0',
          marginBottom: '2rem',
          boxShadow: '0 1px 3px rgba(0,0,0,0.05)'
        }}>
          <h3 style={{ fontSize: '1.1rem', color: 'var(--primary-dark)', marginBottom: '0.4rem' }}>
            ⚛ Practical Objective
          </h3>
          <p style={{ color: 'var(--text-muted)', fontSize: '0.95rem' }}>
            Develop a comprehensive React application demonstrating Props passing between parent-child
components,
            local State Management with the <code>useState</code> hook, and handling diverse React
Synthetic Events
            (<code>onClick</code>, <code>onChange</code>, <code>onSubmit</code>, <code>onKeyDown</code>,
<code>onMouseMove</code>).
          </p>
        </div>

        {/* Row 1: Interactive Counter & Task Manager */}
        <div className="grid-2">
          <CounterSection />
          <TaskManager />
        </div>

        {/* Row 2: Live Profile Customizer & Event Playground */}
        <div className="grid-2">
          <UserProfileCard />
          <InteractiveBox />
        </div>
      </main>

      {/* Footer */}
      <footer className="footer">
        <p><strong>Web Technology Practicals Journal</strong> &bull; Practical 02</p>
        <p style={{ marginTop: '0.35rem', fontSize: '0.8rem', color: '#64748b' }}>
          React Props, State Management & Event Handling System
        </p>
      </footer>
    </div>
  );
}
```

```
export default App;
```

- **src/components/Header.jsx :**

```
import React from 'react';

function Header({ title, subtitle, practicalNo }) {
  return (
    <header className="header">
      {practicalNo && <span className="header-badge">{practicalNo}</span>}
      <h1 className="header-title">{title}</h1>
      <p className="header-subtitle">{subtitle}</p>
    </header>
  );
}

export default Header;
```

- **src/components/CounterSection.jsx :**

```
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. useState hook for managing numeric count and step size
 * 2. onClick event handling with increment, decrement, reset
 * 3. onChange event handling for dynamic step adjustments
 */
function CounterSection() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  const handleIncrement = () => setCount(prev => prev + Number(step));
  const handleDecrement = () => setCount(prev => (prev - Number(step)) >= 0 ? prev - Number(step) : 0));
  const handleReset = () => setCount(0);
  const handleDouble = () => setCount(prev => prev * 2);

  return (
    <div className="card">
      <h3 className="card-title">
        <span>⚛️</span> Interactive Counter (State & Click Events)
      </h3>
      <p className="card-subtitle">
        Demonstrates React <code>useState</code>, numeric state updates, and <code>onClick</code>
handlers.
      </p>

      <div className="counter-box">
        <div className="counter-display">{count}</div>

        <div style={{ width: '100%', maxWidth: '240px' }}>
          <label style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>
            Step Value:
          </label>
          <input
            type="number"
            min="1"
            max="100"
            value={step}
            onChange={(e) => setStep(Math.max(1, parseInt(e.target.value) || 1))}
            className="form-input"
            style={{ marginTop: '0.25rem', textAlign: 'center' }}
          />
        </div>

        <div className="counter-controls">
          <button className="btn btn-primary" onClick={handleIncrement}>
```

```
            +{step} Increment
          </button>
          <button className="btn btn-danger" onClick={handleDecrement}>
            -{step} Decrement
          </button>
          <button className="btn btn-warning" onClick={handleDouble}>
            ✕ 2 Double
          </button>
          <button className="btn btn-outline" onClick={handleReset}>
            ↺ Reset
          </button>
        </div>
      </div>
    </div>
  );
}

export default CounterSection;
```

• **src/components/TaskManager.jsx :**

```
import React, { useState } from 'react';
import TaskItem from './TaskItem';

/**
 * TaskManager component
 * Demonstrates:
 * 1. Managing array state in React
 * 2. Form submission event (onSubmit & e.preventDefault())
 * 3. Passing functions as props down to child components
 * 4. Filtering state (All, Active, Completed)
 */
function TaskManager() {
  const [tasks, setTasks] = useState([
    { id: 1, text: 'Learn React Props & Component Hierarchy', completed: true },
    { id: 2, text: 'Master useState hook and functional updates', completed: false },
    { id: 3, text: 'Handle Synthetic Events in React', completed: false }
  ]);
  const [inputTask, setInputTask] = useState('');
  const [filter, setFilter] = useState('all');

  const handleAddTask = (e) => {
    e.preventDefault();
    if (!inputTask.trim()) return;
    const newTask = {
      id: Date.now(),
      text: inputTask.trim(),
      completed: false
    };
    setTasks([newTask, ...tasks]);
    setInputTask('');
  };

  const handleToggleTask = (id) => {
    setTasks(tasks.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    ));
  };

  const handleDeleteTask = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
  };

  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed;
    if (filter === 'completed') return task.completed;
    return true;
  });
```

```
  return (
    <div className="card">
      <h3 className="card-title">
        <span>⚛️</span> Task Manager (Props, List State & Callbacks)
      </h3>
      <p className="card-subtitle">
        Demonstrates parent-child state synchronization and callback prop patterns.
      </p>

      {/* Input Form with onSubmit */}
      <form onSubmit={handleAddTask} style={{ display: 'flex', gap: '0.5rem', marginBottom: '1.25rem' }}>
        <input
          type="text"
          placeholder="Type a new task and press Add or Enter..."
          value={inputTask}
          onChange={(e) => setInputTask(e.target.value)}
          className="form-input"
        />
        <button type="submit" className="btn btn-primary" style={{ whiteSpace: 'nowrap' }}>
          ➕ Add Task
        </button>
      </form>

      {/* Filter Tabs */}
      <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem' }}>
        <button
          className={`btn btn-sm ${filter === 'all' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('all')}
        >
          All ({tasks.length})
        </button>
        <button
          className={`btn btn-sm ${filter === 'active' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('active')}
        >
          Active ({tasks.filter(t => !t.completed).length})
        </button>
        <button
          className={`btn btn-sm ${filter === 'completed' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('completed')}
        >
          Completed ({tasks.filter(t => t.completed).length})
        </button>
      </div>

      {/* Task List */}
      <div style={{ maxHeight: '250px', overflowY: 'auto' }}>
        {filteredTasks.length === 0 ? (
          <p style={{ textAlign: 'center', color: 'var(--text-muted)', padding: '1.5rem 0' }}>
            No tasks found in this view.
          </p>
        ) : (
          filteredTasks.map(task => (
            <TaskItem
              key={task.id}
              task={task}
              onToggle={handleToggleTask}
              onDelete={handleDeleteTask}
            />
          ))
        )}
      </div>
    </div>
  );
}

export default TaskManager;
```

- **src/components/TaskItem.jsx:**

```jsx
import React from 'react';

function TaskItem({ task, onToggle, onDelete }) {
  return (
    <div className={`task-item ${task.completed ? 'completed' : ''}`}>
      <div className="task-left">
        <input
          type="checkbox"
          checked={task.completed}
          onChange={() => onToggle(task.id)}
          className="task-checkbox"
        />
        <span className="task-text">{task.text}</span>
      </div>
      <button
        className="btn btn-danger btn-sm"
        onClick={() => onDelete(task.id)}
        title="Delete task"
      >
        ×
      </button>
    </div>
  );
}

export default TaskItem;
```

- **src/components/UserProfileCard.jsx :**

```jsx
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. Object state management in React
 * 2. Controlled form input handling with event target name & value
 * 3. Passing updated state values as Props to display sub-components
 */
function UserProfileCard() {
  const [profile, setProfile] = useState({
    name: 'Prince Prajapati',
    role: 'Full Stack Developer',
    experience: '3 Years',
    location: 'Ahmedabad, India',
    isAvailable: true
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setProfile(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  return (
    <div className="card">
      <h3 className="card-title">
        <span>⚛️</span> Live User Profile (Object State & Change Events)
      </h3>
      <p className="card-subtitle">
        Update the inputs below to see immediate two-way state reflection in the profile card.
      </p>

      {/* Form Controls */}
      <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem', marginBottom:
'1.25rem' }}>
        <div>
          <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)' }}>Name:
</label>
```

```
            <input
              type="text"
              name="name"
              value={profile.name}
              onChange={handleChange}
              className="form-input"
            />
          </div>
          <div>
            <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)' }}>Role:
</label>
            <input
              type="text"
              name="role"
              value={profile.role}
              onChange={handleChange}
              className="form-input"
            />
          </div>
          <div>
            <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)' }}>Experience:
</label>
            <input
              type="text"
              name="experience"
              value={profile.experience}
              onChange={handleChange}
              className="form-input"
            />
          </div>
          <div>
            <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)' }}>Location:
</label>
            <input
              type="text"
              name="location"
              value={profile.location}
              onChange={handleChange}
              className="form-input"
            />
          </div>
        </div>

        <div style={{ marginBottom: '1.25rem' }}>
          <label style={{ display: 'inline-flex', alignItems: 'center', gap: '0.5rem', cursor: 'pointer',
fontSize: '0.9rem', fontWeight: 600 }}>
            <input
              type="checkbox"
              name="isAvailable"
              checked={profile.isAvailable}
              onChange={handleChange}
              className="task-checkbox"
            />
            Available for Freelance / Hire
          </label>
        </div>

        {/* Live Preview Card */}
        <div className="user-preview">
          <div className="user-avatar">
            {profile.name ? profile.name.charAt(0).toUpperCase() : '?'}
          </div>
          <div>
            <h4 style={{ fontSize: '1.15rem', fontWeight: 700 }}>{profile.name || 'Anonymous'}</h4>
            <p style={{ color: 'var(--primary-dark)', fontWeight: 600, fontSize: '0.9rem' }}>{profile.role
|| 'No Role'}</p>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
              📍 {profile.location} &bull; 💼 {profile.experience}
            </p>
            <span style={{
```

```
            display: 'inline-block',
            marginTop: '0.35rem',
            padding: '0.2rem 0.6rem',
            borderRadius: '9999px',
            fontSize: '0.75rem',
            fontWeight: 700,
            background: profile.isAvailable ? '#dcfce7' : '#fee2e2',
            color: profile.isAvailable ? '#15803d' : '#b91c1c'
          }}>
            {profile.isAvailable ? '● Available' : '○ Unavailable'}
          </span>
        </div>
      </div>
    </div>
  );
}

export default UserProfileCard;
```

• **src/components/InteractiveBox.jsx :**

```
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. Mouse Events: onMouseEnter, onMouseLeave, onMouseMove
 * 2. Keyboard Events: onKeyDown
 * 3. Dynamic styling based on component state
 */
function InteractiveBox() {
  const [bgChoice, setBgChoice] = useState('indigo');
  const [isHovered, setIsHovered] = useState(false);
  const [lastKeyPressed, setLastKeyPressed] = useState('None');
  const [mouseCoord, setMouseCoord] = useState({ x: 0, y: 0 });

  const colorPalettes = {
    indigo: 'linear-gradient(135deg, #6366f1 0%, #4338ca 100%)',
    emerald: 'linear-gradient(135deg, #10b981 0%, #047857 100%)',
    rose: 'linear-gradient(135deg, #f43f5e 0%, #be123c 100%)',
    amber: 'linear-gradient(135deg, #f59e0b 0%, #b45309 100%)'
  };

  const handleMouseMove = (e) => {
    const rect = e.currentTarget.getBoundingClientRect();
    setMouseCoord({
      x: Math.round(e.clientX - rect.left),
      y: Math.round(e.clientY - rect.top)
    });
  };

  return (
    <div className="card">
      <h3 className="card-title">
        <span> ⚡ </span> Mouse & Keyboard Events Playground
      </h3>
      <p className="card-subtitle">
        Hover, move cursor inside the canvas, and press keys to inspect synthetic event payload.
      </p>

      {/* Color theme selectors */}
      <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem', alignItems: 'center' }}>
        <span style={{ fontSize: '0.85rem', fontWeight: 600 }}>Palette:</span>
        {Object.keys(colorPalettes).map((color) => (
          <button
            key={color}
            onClick={() => setBgChoice(color)}
            style={{
              padding: '0.35rem 0.75rem',
              borderRadius: '6px',
```

```
              border: 'none',
              cursor: 'pointer',
              fontWeight: 600,
              fontSize: '0.8rem',
              textTransform: 'capitalize',
              background: bgChoice === color ? '#0f172a' : '#e2e8f0',
              color: bgChoice === color ? '#ffffff' : '#334155'
            }}
          >
            {color}
          </button>
        ))}
      </div>

      {/* Interactive canvas box */}
      <div
        onMouseEnter={() => setIsHovered(true)}
        onMouseLeave={() => setIsHovered(false)}
        onMouseMove={handleMouseMove}
        style={{
          background: colorPalettes[bgChoice],
          color: 'white',
          borderRadius: '12px',
          padding: '2rem 1.5rem',
          textAlign: 'center',
          transition: 'transform 0.2s ease, box-shadow 0.2s ease',
          transform: isHovered ? 'scale(1.02)' : 'scale(1)',
          boxShadow: isHovered ? '0 10px 25px rgba(0,0,0,0.2)' : 'none',
          cursor: 'crosshair',
          marginBottom: '1rem'
        }}
      >
        <h4 style={{ fontSize: '1.25rem', marginBottom: '0.5rem' }}>
          {isHovered ? '✨ Mouse is Inside Box!' : '🎯 Hover Cursor Here'}
        </h4>
        <p style={{ fontSize: '0.9rem', opacity: 0.9 }}>
          Cursor Coordinates: <strong>X: {mouseCoord.x}px | Y: {mouseCoord.y}px</strong>
        </p>
      </div>

      {/* Keydown testing input */}
      <div>
        <label style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>
          Test Keyboard Event (Type any key inside input):
        </label>
        <input
          type="text"
          placeholder="Press any key here..."
          onKeyDown={(e) => setLastKeyPressed(`${e.key} (Code: ${e.code})`)}
          className="form-input"
          style={{ marginTop: '0.25rem' }}
        />
        <p style={{ marginTop: '0.4rem', fontSize: '0.85rem', color: 'var(--text-muted)' }}>
          Last Key Detected: <strong style={{ color: 'var(--primary-dark)' }}>{lastKeyPressed}</strong>
        </p>
      </div>
    </div>
  );
}

export default InteractiveBox;
```

4. Output :

Demonstrates uni-directional data flow, state management with useState, array state manipulation, form submissions, and mouse/keyboard synthetic event handlers.

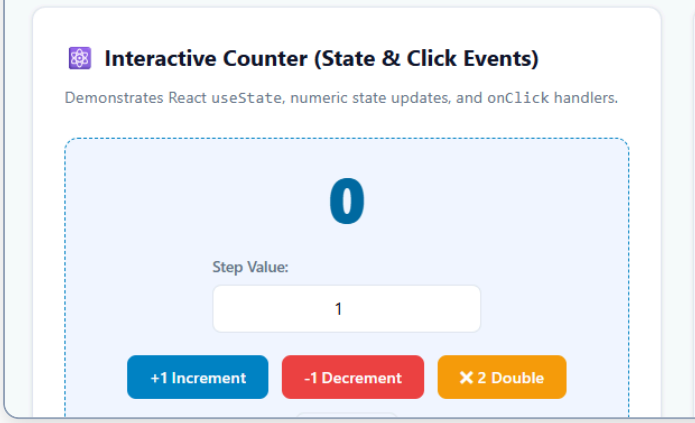
PRACTICAL NO. 02

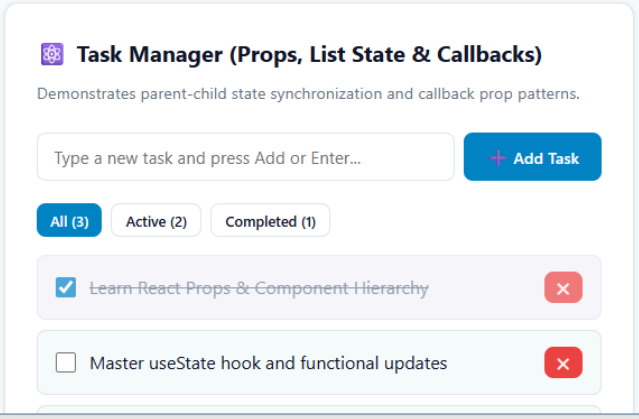
Props, State Management & Event Handling

Practical implementation of Uni-directional Data Flow, useState Hook, and React Synthetic Event Handlers

Practical Objective

Develop a comprehensive React application demonstrating Props passing between parent-child components, local State Management with the `useState` hook, and handling diverse React Synthetic Events (`onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onMouseMove`).





Practical – 03 : Create a Student Management System using React Forms, Controlled Components and Validation.

1. Commands :

```
npm create vite@latest MERN_03
cd MERN_03
npm install
npm run dev
```

2. Project structure :

```
MERN_03/
|-- package.json
|-- index.html
|-- vite.config.js
|-- src/main.jsx
|-- src/index.css
|-- src/App.jsx
|-- src/utils/validation.js
|-- src/components/Header.jsx
|-- src/components/StatsOverview.jsx
|-- src/components/StudentForm.jsx
|-- src/components/StudentTable.jsx
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-03",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.2"
  }
}
```

- **index.html :**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Practical 03 - Student Management System Forms & Validation</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

• **vite.config.js :**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: './',
  plugins: [react()],
});
```

• **src/main.jsx :**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

• **src/index.css :**

```
:root {
  --primary: #059669;
  --primary-dark: #047857;
  --primary-light: #d1fae5;
  --accent: #3b82f6;
  --danger: #ef4444;
  --danger-light: #fee2e2;
  --warning: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --radius: 12px;
  --font-family: 'Inter', system-ui, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.main-content {
  flex: 1;
  max-width: 1300px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
  width: 100%;
}
```

```css
.header {
  background: linear-gradient(135deg, #064e3b 0%, #065f46 50%, #059669 100%);
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1);
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 700;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
}

.header-subtitle {
  font-size: 1.05rem;
  color: #a7f3d0;
  max-width: 700px;
  margin: 0 auto;
}

/* Stats Cards */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
  margin-bottom: 2rem;
}

.stat-card {
  background: white;
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.25rem;
  display: flex;
  align-items: center;
  gap: 1rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}

.stat-icon {
  width: 48px;
  height: 48px;
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.5rem;
}

.stat-number {
  font-size: 1.6rem;
  font-weight: 800;
  color: var(--text-main);
  line-height: 1.1;
}

.stat-label {
```

```
  font-size: 0.85rem;
  color: var(--text-muted);
}

/* Main Two Columns */
.layout-grid {
  display: grid;
  grid-template-columns: 420px 1fr;
  gap: 2rem;
  align-items: start;
}

@media (max-width: 950px) {
  .layout-grid {
    grid-template-columns: 1fr;
  }
}

.card {
  background: white;
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.04);
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 1.25rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

/* Form Styles */
.form-group {
  margin-bottom: 1rem;
}

.form-label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-main);
  margin-bottom: 0.3rem;
}

.form-input, .form-select, .form-textarea {
  width: 100%;
  padding: 0.65rem 0.85rem;
  font-size: 0.9rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  outline: none;
  font-family: inherit;
  transition: all 0.2s;
}

.form-input:focus, .form-select:focus, .form-textarea:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(5, 150, 105, 0.15);
}

.form-input.has-error, .form-select.has-error {
  border-color: var(--danger);
  background-color: #fffafb;
}

.error-text {
```

```
  color: var(--danger);
  font-size: 0.78rem;
  margin-top: 0.25rem;
  display: flex;
  align-items: center;
  gap: 0.25rem;
}

.radio-group, .checkbox-group {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  margin-top: 0.35rem;
}

.radio-label, .checkbox-label {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.88rem;
  cursor: pointer;
}

.form-actions {
  display: flex;
  gap: 0.75rem;
  margin-top: 1.5rem;
}

/* Buttons */
.btn {
  padding: 0.6rem 1.2rem;
  font-size: 0.88rem;
  font-weight: 600;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  transition: all 0.2s;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-secondary { background: #e2e8f0; color: #334155; }
.btn-secondary:hover { background: #cbd5e1; }
.btn-danger { background: var(--danger); color: white; }
.btn-danger:hover { background: #dc2626; }
.btn-outline-danger { background: transparent; border: 1px solid var(--danger); color: var(--danger); }
.btn-outline-danger:hover { background: var(--danger-light); }
.btn-outline-primary { background: transparent; border: 1px solid var(--primary); color: var(--primary);
}
.btn-outline-primary:hover { background: var(--primary-light); }
.btn-sm { padding: 0.35rem 0.65rem; font-size: 0.8rem; }

/* Table Styles */
.table-container {
  overflow-x: auto;
}

.student-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
  font-size: 0.88rem;
}

.student-table th {
```

```
  background: #f1f5f9;
  padding: 0.75rem 1rem;
  font-weight: 700;
  color: var(--text-main);
  border-bottom: 2px solid var(--border-color);
}

.student-table td {
  padding: 0.85rem 1rem;
  border-bottom: 1px solid var(--border-color);
  vertical-align: middle;
}

.student-table tr:hover {
  background: #f8fafc;
}

.badge {
  display: inline-block;
  padding: 0.2rem 0.6rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
}

.badge-dept { background: #dbeafe; color: #1e40af; }
.badge-sem { background: #fef3c7; color: #92400e; }
.badge-skill { background: #f1f5f9; color: #475569; margin-right: 0.25rem; font-size: 0.7rem; }

.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
}
```

- **src/App.jsx :**

```
import React, { useState } from 'react';
import Header from './components/Header';
import StatsOverview from './components/StatsOverview';
import StudentForm from './components/StudentForm';
import StudentTable from './components/StudentTable';

const INITIAL_STUDENTS = [
  {
    id: 'STU-1001',
    fullName: 'Prince Prajapati',
    rollNo: '26084231038',
    email: 'prince@student.edu',
    phone: '9876543210',
    department: 'Computer Engineering',
    semester: 'Semester 6',
    gender: 'Male',
    skills: ['React', 'Node.js', 'MongoDB', 'JavaScript'],
    address: 'Ahmedabad, Gujarat'
  },
  {
    id: 'STU-1002',
    fullName: 'Priya Sharma',
    rollNo: '21IT012',
    email: 'priya.sharma@student.edu',
    phone: '9812345678',
    department: 'Information Technology',
    semester: 'Semester 6',
    gender: 'Female',
    skills: ['Python', 'SQL', 'React'],
```

```
    address: 'Mumbai, Maharashtra'
  },
  {
    id: 'STU-1003',
    fullName: 'Aman Verma',
    rollNo: '22AI088',
    email: 'aman.verma@student.edu',
    phone: '9765432109',
    department: 'AI & Data Science',
    semester: 'Semester 4',
    gender: 'Male',
    skills: ['Python', 'Java', 'SQL'],
    address: 'Pune, Maharashtra'
  }
];

function App() {
  const [students, setStudents] = useState(INITIAL_STUDENTS);
  const [editingStudent, setEditingStudent] = useState(null);
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedDeptFilter, setSelectedDeptFilter] = useState('All');
  const [alertMessage, setAlertMessage] = useState(null);

  const showAlert = (text, type = 'success') => {
    setAlertMessage({ text, type });
    setTimeout(() => setAlertMessage(null), 3500);
  };

  // Add or Update student handler
  const handleSaveStudent = (studentData) => {
    if (editingStudent) {
      setStudents(prev => prev.map(s => s.id === editingStudent.id ? { ...studentData, id: s.id } : s));
      showAlert(`✅ Student record "${studentData.fullName}" updated successfully!`);
      setEditingStudent(null);
    } else {
      const newStudent = {
        ...studentData,
        id: `STU-${Date.now().toString().slice(-4)}`
      };
      setStudents(prev => [newStudent, ...prev]);
      showAlert(`🎉 Student "${studentData.fullName}" added successfully!`);
    }
  };

  // Delete student handler
  const handleDeleteStudent = (id, name) => {
    if (window.confirm(`Are you sure you want to delete student: ${name}?`)) {
      setStudents(prev => prev.filter(s => s.id !== id));
      if (editingStudent && editingStudent.id === id) {
        setEditingStudent(null);
      }
      showAlert(`🗑️ Student record "${name}" deleted.`, 'danger');
    }
  };

  // Edit initiate
  const handleEditStudent = (student) => {
    setEditingStudent(student);
    window.scrollTo({ top: 150, behavior: 'smooth' });
  };

  // Filter students based on search and department
  const filteredStudents = students.filter(student => {
    const matchesSearch =
      student.fullName.toLowerCase().includes(searchQuery.toLowerCase()) ||
      student.rollNo.toLowerCase().includes(searchQuery.toLowerCase()) ||
      student.email.toLowerCase().includes(searchQuery.toLowerCase());
    const matchesDept = selectedDeptFilter === 'All' || student.department === selectedDeptFilter;
    return matchesSearch && matchesDept;
  });
```

```
  return (
    <div className="app-container">
      {/* Header */}
      <Header
        practicalNo="Practical No. 03"
        title="Student Management System"
        subtitle="Comprehensive React Forms, Controlled Components, Live Field Validations & Full CRUD
Operations"
      />

      <main className="main-content">
        {/* Toast Alert Notification */}
        {alertMessage && (
          <div style={{
            background: alertMessage.type === 'danger' ? '#fee2e2' : '#dcfce7',
            color: alertMessage.type === 'danger' ? '#991b1b' : '#166534',
            border: `1px solid ${alertMessage.type === 'danger' ? '#f87171' : '#86efac'}`,
            padding: '0.85rem 1.25rem',
            borderRadius: '8px',
            marginBottom: '1.5rem',
            fontWeight: 600,
            fontSize: '0.9rem',
            boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
          }}>
            {alertMessage.text}
          </div>
        )}

        {/* Top Stats Overview */}
        <StatsOverview students={students} />

        {/* 2-Column Responsive Layout: Form on Left, Table on Right */}
        <div className="layout-grid">
          {/* Left Column: Form with Controlled Components */}
          <StudentForm
            onSaveStudent={handleSaveStudent}
            editingStudent={editingStudent}
            onCancelEdit={() => setEditingStudent(null)}
            existingStudents={students}
          />

          {/* Right Column: Search, Filter & Student Records Table */}
          <div className="card">
            <div className="card-title">
              <span>📋 Registered Student Directory</span>
              <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
                Showing {filteredStudents.length} of {students.length}
              </span>
            </div>

            {/* Filter and Search Bar */}
            <div style={{ display: 'grid', gridTemplateColumns: '1fr auto', gap: '0.75rem', marginBottom:
'1.25rem' }}>
              <input
                type="text"
                placeholder="🔍 Search by name, roll no, email..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="form-input"
              />
              <select
                value={selectedDeptFilter}
                onChange={(e) => setSelectedDeptFilter(e.target.value)}
                className="form-select"
                style={{ width: 'auto', minWidth: '180px' }}
              >
                <option value="All">All Departments</option>
                <option value="Computer Engineering">Computer Engineering</option>
                <option value="Information Technology">Information Technology</option>
```

```
                <option value="AI & Data Science">AI & Data Science</option>
                <option value="Electronics & Comm">Electronics & Comm</option>
                <option value="Mechanical Engineering">Mechanical Engineering</option>
              </select>
            </div>

            {/* Table */}
            <StudentTable
              students={filteredStudents}
              onEdit={handleEditStudent}
              onDelete={handleDeleteStudent}
            />
          </div>
        </div>
      </main>

      {/* Footer */}
      <footer className="footer">
        <p><strong>Web Technology Practicals Journal</strong> &bull; Practical 03</p>
        <p style={{ marginTop: '0.35rem', fontSize: '0.8rem', color: '#64748b' }}>
          Student Management System using React Forms, Controlled Components and Validation
        </p>
      </footer>
    </div>
  );
}

export default App;
```

- **src/utils/validation.js :**

```
/**
 * Utility for Form Validation rules
 */
export const validateStudentForm = (formData, existingStudents = [], currentEditingId = null) => {
  const errors = {};

  // 1. Full Name Validation
  if (!formData.fullName || !formData.fullName.trim()) {
    errors.fullName = 'Full Name is required.';
  } else if (formData.fullName.trim().length < 3) {
    errors.fullName = 'Full Name must be at least 3 characters.';
  } else if (!/^[a-zA-Z\s]+$/.test(formData.fullName.trim())) {
    errors.fullName = 'Full Name can only contain letters and spaces.';
  }

  // 2. Roll Number Validation
  if (!formData.rollNo || !formData.rollNo.trim()) {
    errors.rollNo = 'Enrollment / Roll No is required.';
  } else if (!/^[A-Za-z0-9_-]{3,15}$/.test(formData.rollNo.trim())) {
    errors.rollNo = 'Roll No must be 3-15 alphanumeric characters.';
  } else {
    // Check uniqueness
    const duplicate = existingStudents.find(
      s => s.rollNo.toLowerCase() === formData.rollNo.trim().toLowerCase() && s.id !== currentEditingId
    );
    if (duplicate) {
      errors.rollNo = 'This Roll No is already registered.';
    }
  }

  // 3. Email Validation
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!formData.email || !formData.email.trim()) {
    errors.email = 'Email address is required.';
  } else if (!emailRegex.test(formData.email.trim())) {
    errors.email = 'Please enter a valid email address (e.g. name@domain.com).';
  }
```

```
  // 4. Phone Number Validation
  const phoneRegex = /^[6-9]\d{9}$/;
  if (!formData.phone || !formData.phone.trim()) {
    errors.phone = '10-digit Mobile Number is required.';
  } else if (!phoneRegex.test(formData.phone.trim())) {
    errors.phone = 'Enter a valid 10-digit Indian mobile number starting with 6-9.';
  }

  // 5. Department / Course Selection
  if (!formData.department) {
    errors.department = 'Please select a department / branch.';
  }

  // 6. Semester Selection
  if (!formData.semester) {
    errors.semester = 'Please select the current semester.';
  }

  // 7. Gender Validation
  if (!formData.gender) {
    errors.gender = 'Please select gender.';
  }

  // 8. Skills / Interests Validation (Minimum 1 skill selected)
  if (!formData.skills || formData.skills.length === 0) {
    errors.skills = 'Please select at least one skill or interest.';
  }

  return {
    isValid: Object.keys(errors).length === 0,
    errors
  };
};
```

- **src/components/Header.jsx :**

```
import React from 'react';

function Header({ title, subtitle, practicalNo }) {
  return (
    <header className="header">
      {practicalNo && <span className="header-badge">{practicalNo}</span>}
      <h1 className="header-title">{title}</h1>
      <p className="header-subtitle">{subtitle}</p>
    </header>
  );
}

export default Header;
```

- **src/components/StatsOverview.jsx :**

```
import React from 'react';

function StatsOverview({ students }) {
  const totalStudents = students.length;
  const cseCount = students.filter(s => s.department === 'Computer Engineering').length;
  const itCount = students.filter(s => s.department === 'Information Technology').length;
  const aiCount = students.filter(s => s.department === 'AI & Data Science').length;

  return (
    <div className="stats-grid">
      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#ecfdf5', color: '#059669' }}>👥</div>
        <div>
          <div className="stat-number">{totalStudents}</div>
          <div className="stat-label">Total Enrolled Students</div>
        </div>
```

```
      </div>
      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#eff6ff', color: '#2563eb' }}>💻</div>
        <div>
          <div className="stat-number">{cseCount}</div>
          <div className="stat-label">Computer Engg</div>
        </div>
      </div>
      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#f5f3ff', color: '#7c3aed' }}>🌐</div>
        <div>
          <div className="stat-number">{itCount}</div>
          <div className="stat-label">Information Tech</div>
        </div>
      </div>
      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#fffbeb', color: '#d97706' }}>🤖</div>
        <div>
          <div className="stat-number">{aiCount}</div>
          <div className="stat-label">AI & Data Science</div>
        </div>
      </div>
    </div>
  );
}

export default StatsOverview;
```

- **src/components/StudentForm.jsx :**

```
import React, { useState, useEffect } from 'react';
import { validateStudentForm } from '../utils/validation';

const initialFormState = {
  fullName: '',
  rollNo: '',
  email: '',
  phone: '',
  department: '',
  semester: '',
  gender: '',
  skills: [],
  address: ''
};

const AVAILABLE_SKILLS = ['JavaScript', 'React', 'Node.js', 'Python', 'MongoDB', 'SQL', 'Java'];

function StudentForm({ onSaveStudent, editingStudent, onCancelEdit, existingStudents }) {
  const [formData, setFormData] = useState(initialFormState);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});

  useEffect(() => {
    if (editingStudent) {
      setFormData(editingStudent);
      setErrors({});
      setTouched({});
    } else {
      setFormData(initialFormState);
      setErrors({});
      setTouched({});
    }
  }, [editingStudent]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));

    // Clear error for this field as user types
```

```
    if (errors[name]) {
      setErrors(prev => ({ ...prev, [name]: '' }));
    }
  };

  const handleBlur = (e) => {
    const { name } = e.target;
    setTouched(prev => ({ ...prev, [name]: true }));
    const { errors: newErrors } = validateStudentForm(formData, existingStudents, editingStudent ?
editingStudent.id : null);
    if (newErrors[name]) {
      setErrors(prev => ({ ...prev, [name]: newErrors[name] }));
    }
  };

  const handleSkillToggle = (skill) => {
    setFormData(prev => {
      const skills = prev.skills.includes(skill)
        ? prev.skills.filter(s => s !== skill)
        : [...prev.skills, skill];
      return { ...prev, skills };
    });
    if (errors.skills) {
      setErrors(prev => ({ ...prev, skills: '' }));
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const { isValid, errors: validationErrors } = validateStudentForm(
      formData,
      existingStudents,
      editingStudent ? editingStudent.id : null
    );

    if (!isValid) {
      setErrors(validationErrors);
      // Mark all fields as touched to display errors
      const allTouched = Object.keys(formData).reduce((acc, key) => ({ ...acc, [key]: true }), {});
      setTouched(allTouched);
      return;
    }

    onSaveStudent(formData);
    setFormData(initialFormState);
    setErrors({});
    setTouched({});
  };

  return (
    <div className="card">
      <div className="card-title">
        <span>{editingStudent ? '🖍️ Edit Student Record' : '➕ Add New Student'}</span>
        {editingStudent && (
          <span style={{ fontSize: '0.8rem', color: 'var(--primary-dark)', fontWeight: 600 }}>
            Editing ID: {editingStudent.id}
          </span>
        )}
      </div>

      <form onSubmit={handleSubmit} noValidate>
        {/* Full Name */}
        <div className="form-group">
          <label className="form-label">Full Name *</label>
          <input
            type="text"
            name="fullName"
            placeholder="e.g. Prince Prajapati"
            value={formData.fullName}
            onChange={handleChange}
```

```
            onBlur={handleBlur}
            className={`form-input ${touched.fullName && errors.fullName ? 'has-error' : ''}`}
          />
          {touched.fullName && errors.fullName && <p className="error-text">⚠ {errors.fullName}</p>}
        </div>

        {/* Roll Number & Email in 2 columns */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Enrollment / Roll No *</label>
            <input
              type="text"
              name="rollNo"
              placeholder="e.g. 26084231038"
              value={formData.rollNo}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.rollNo && errors.rollNo ? 'has-error' : ''}`}
            />
            {touched.rollNo && errors.rollNo && <p className="error-text">⚠ {errors.rollNo}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Email Address *</label>
            <input
              type="email"
              name="email"
              placeholder="e.g. prince@student.edu"
              value={formData.email}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.email && errors.email ? 'has-error' : ''}`}
            />
            {touched.email && errors.email && <p className="error-text">⚠ {errors.email}</p>}
          </div>
        </div>

        {/* Phone & Department */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Phone Number *</label>
            <input
              type="tel"
              name="phone"
              maxLength="10"
              placeholder="10-digit Mobile"
              value={formData.phone}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.phone && errors.phone ? 'has-error' : ''}`}
            />
            {touched.phone && errors.phone && <p className="error-text">⚠ {errors.phone}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Department / Branch *</label>
            <select
              name="department"
              value={formData.department}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-select ${touched.department && errors.department ? 'has-error' : ''}`}
            >
              <option value="">-- Select Branch --</option>
              <option value="Computer Engineering">Computer Engineering</option>
              <option value="Information Technology">Information Technology</option>
              <option value="AI & Data Science">AI & Data Science</option>
              <option value="Electronics & Comm">Electronics & Comm</option>
              <option value="Mechanical Engineering">Mechanical Engineering</option>
            </select>
```

```
            {touched.department && errors.department && <p className="error-text">⚠ {errors.department}
</p>}
          </div>
        </div>

        {/* Semester & Gender */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Current Semester *</label>
            <select
              name="semester"
              value={formData.semester}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-select ${touched.semester && errors.semester ? 'has-error' : ''}`}
            >
              <option value="">-- Select Sem --</option>
              {[1, 2, 3, 4, 5, 6, 7, 8].map(s => (
                <option key={s} value={`Semester ${s}`}>Semester {s}</option>
              ))}
            </select>
            {touched.semester && errors.semester && <p className="error-text">⚠ {errors.semester}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Gender *</label>
            <div className="radio-group" style={{ marginTop: '0.6rem' }}>
              {['Male', 'Female', 'Other'].map(g => (
                <label key={g} className="radio-label">
                  <input
                    type="radio"
                    name="gender"
                    value={g}
                    checked={formData.gender === g}
                    onChange={handleChange}
                  />
                  {g}
                </label>
              ))}
            </div>
            {touched.gender && errors.gender && <p className="error-text">⚠ {errors.gender}</p>}
          </div>
        </div>

        {/* Skills / Interests (Checkboxes) */}
        <div className="form-group">
          <label className="form-label">Technical Skills & Interests *</label>
          <div className="checkbox-group">
            {AVAILABLE_SKILLS.map(skill => (
              <label key={skill} className="checkbox-label">
                <input
                  type="checkbox"
                  checked={formData.skills.includes(skill)}
                  onChange={() => handleSkillToggle(skill)}
                />
                {skill}
              </label>
            ))}
          </div>
          {touched.skills && errors.skills && <p className="error-text">⚠ {errors.skills}</p>}
        </div>

        {/* Address / Remarks */}
        <div className="form-group">
          <label className="form-label">Residential Address / City (Optional)</label>
          <textarea
            name="address"
            rows="2"
            placeholder="Enter student city or address details..."
            value={formData.address}
```

```
            onChange={handleChange}
            className="form-textarea"
          ></textarea>
        </div>

        {/* Form Actions */}
        <div className="form-actions">
          <button type="submit" className="btn btn-primary" style={{ flex: 1 }}>
            {editingStudent ? '💾 Update Student' : '➕ Register Student'}
          </button>
          {editingStudent && (
            <button type="button" className="btn btn-secondary" onClick={onCancelEdit}>
              Cancel
            </button>
          )}
        </div>
      </form>
    </div>
  );
}

export default StudentForm;
```

• **src/components/StudentTable.jsx :**

```
import React from 'react';

function StudentTable({ students, onEdit, onDelete }) {
  if (students.length === 0) {
    return (
      <div style={{ textAlign: 'center', padding: '3rem 1rem', color: 'var(--text-muted)' }}>
        <p style={{ fontSize: '2.5rem', marginBottom: '0.5rem' }}>📋</p>
        <h4>No Student Records Found</h4>
        <p style={{ fontSize: '0.88rem' }}>Register a student from the form on the left or clear your
filter.</p>
      </div>
    );
  }

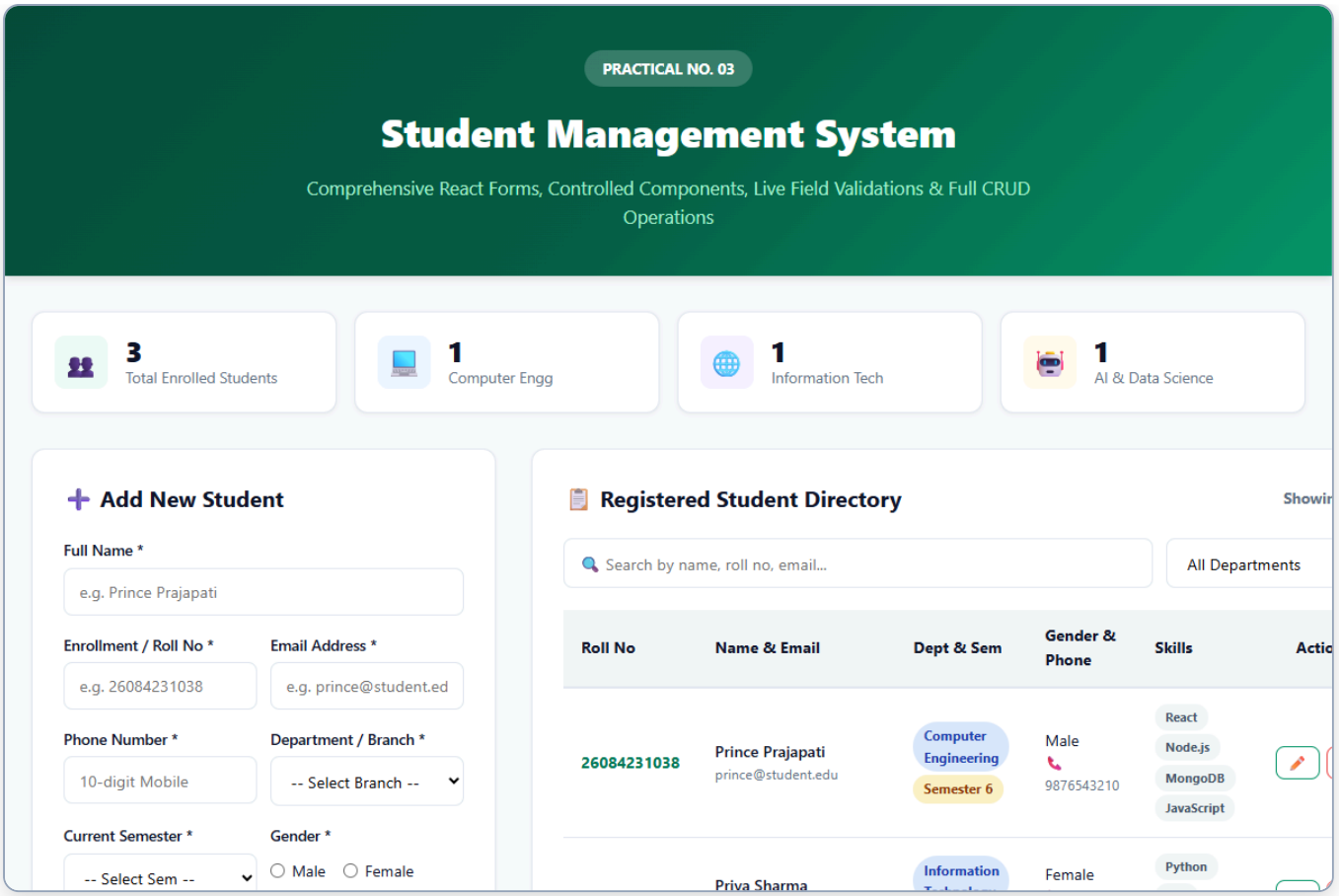
  return (
    <div className="table-container">
      <table className="student-table">
        <thead>
          <tr>
            <th>Roll No</th>
            <th>Name & Email</th>
            <th>Dept & Sem</th>
            <th>Gender & Phone</th>
            <th>Skills</th>
            <th style={{ textAlign: 'center' }}>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student.id}>
              <td>
                <strong style={{ color: 'var(--primary-dark)' }}>{student.rollNo}</strong>
              </td>
              <td>
                <div style={{ fontWeight: 600 }}>{student.fullName}</div>
                <div style={{ fontSize: '0.78rem', color: 'var(--text-muted)' }}>{student.email}</div>
              </td>
              <td>
                <div><span className="badge badge-dept">{student.department}</span></div>
                <div style={{ marginTop: '0.2rem' }}><span className="badge badge-sem">{student.semester}
</span></div>
              </td>
              <td>
                <div>{student.gender}</div>
```

```
                <div style={{ fontSize: '0.78rem', color: 'var(--text-muted)' }}>📞 {student.phone}</div>
              </td>
              <td>
                <div style={{ display: 'flex', flexWrap: 'wrap', gap: '0.2rem', maxWidth: '200px' }}>
                  {student.skills && student.skills.map(s => (
                    <span key={s} className="badge badge-skill">{s}</span>
                  ))}
                </div>
              </td>
              <td>
                <div style={{ display: 'flex', gap: '0.4rem', justifyContent: 'center' }}>
                  <button
                    className="btn btn-outline-primary btn-sm"
                    onClick={() => onEdit(student)}
                    title="Edit Record"
                  >
                    ✏️
                  </button>
                  <button
                    className="btn btn-outline-danger btn-sm"
                    onClick={() => onDelete(student.id, student.fullName)}
                    title="Delete Record"
                  >
                    🗑️
                  </button>
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default StudentTable;
```

4. Output :

Controlled React forms implementing live Regex validation for names, roll numbers, emails, and Indian mobile numbers, complete with an interactive student directory.


PRACTICAL NO. 03
Student Management System
Comprehensive React Forms, Controlled Components, Live Field Validations & Full CRUD Operations
3
Total Enrolled Students
1
Computer Engg
1
Information Tech
1
AI & Data Science
Add New Student
Full Name *
e.g. Prince Prajapati
Enrollment / Roll No *
e.g. 26084231038
Email Address *
e.g. prince@student.ed
Phone Number *
10-digit Mobile
Department / Branch *
-- Select Branch --
Current Semester *
-- Select Sem --
Gender *
Male
Female
Registered Student Directory
Search by name, roll no, email...
All Departments
Roll No
Name & Email
Dept & Sem
Gender & Phone
Skills
26084231038
Prince Prajapati
prince@student.edu
Computer Engineering
Semester 6
Male
9876543210
React
Node.js
MongoDB
JavaScript
Priya Sharma
Female
Python

Practical – 04 : Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.

1. Commands :

```
npm create vite@latest MERN_04
cd MERN_04
npm install react-router-dom
npm run dev
```

2. Project structure :

```
MERN_04/
|-- package.json
|-- index.html
|-- vite.config.js
|-- src/main.jsx
|-- src/index.css
|-- src/App.jsx
|-- src/data/coursesData.js
|-- src/components/Navbar.jsx
|-- src/components/Footer.jsx
|-- src/pages/Home.jsx
|-- src/pages/Courses.jsx
|-- src/pages/CourseDetails.jsx
|-- src/pages/About.jsx
|-- src/pages/Contact.jsx
|-- src/pages/NotFound.jsx
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-04",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.30.6"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.2"
  }
}
```

- **index.html :**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Practical 04 - Single Page Application with React Router</title>
  </head>
  <body>
```

```
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

- **vite.config.js :**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: './',
  plugins: [react()],
});
```

- **src/main.jsx :**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { HashRouter } from 'react-router-dom';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <HashRouter>
      <App />
    </HashRouter>
  </React.StrictMode>,
);
```

- **src/index.css :**

```
:root {
  --primary: #7c3aed;
  --primary-dark: #6d28d9;
  --primary-light: #ede9fe;
  --secondary: #0ea5e9;
  --accent: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --radius: 12px;
  --font-family: 'Plus Jakarta Sans', system-ui, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-wrapper {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
```

```css
/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 100;
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--border-color);
  padding: 0.9rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.nav-brand {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  font-size: 1.25rem;
  font-weight: 800;
  color: var(--primary);
  text-decoration: none;
}

.nav-links {
  display: flex;
  gap: 0.5rem;
  list-style: none;
  align-items: center;
}

.nav-item a {
  text-decoration: none;
  color: var(--text-muted);
  font-weight: 600;
  font-size: 0.925rem;
  padding: 0.5rem 1rem;
  border-radius: 8px;
  transition: all 0.2s ease;
}

.nav-item a:hover {
  color: var(--primary);
  background: var(--primary-light);
}

.nav-item a.active {
  color: var(--primary-dark);
  background: var(--primary-light);
  font-weight: 700;
}

.main-content {
  flex: 1;
  max-width: 1200px;
  width: 100%;
  margin: 0 auto;
  padding: 2.5rem 1.5rem;
}

/* Hero Section */
.hero-section {
  text-align: center;
  padding: 3.5rem 1rem;
  background: linear-gradient(135deg, #f5f3ff 0%, #ede9fe 100%);
  border-radius: 18px;
  border: 1px solid #ddd6fe;
  margin-bottom: 3rem;
}
```

```css
.hero-title {
  font-size: 2.75rem;
  font-weight: 800;
  color: #4c1d95;
  margin-bottom: 1rem;
  line-height: 1.2;
}

.hero-subtitle {
  font-size: 1.15rem;
  color: #6d28d9;
  max-width: 650px;
  margin: 0 auto 1.75rem;
}

.hero-buttons {
  display: flex;
  gap: 1rem;
  justify-content: center;
  flex-wrap: wrap;
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.95rem;
  font-weight: 600;
  padding: 0.7rem 1.5rem;
  border-radius: 9px;
  text-decoration: none;
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-outline { background: white; color: var(--text-main); border: 1px solid var(--border-color); }
.btn-outline:hover { background: #f1f5f9; }
.btn-sm { padding: 0.4rem 0.85rem; font-size: 0.85rem; }

/* Grid */
.grid-2 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(380px, 1fr));
  gap: 1.75rem;
}

.grid-3 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 1.75rem;
}

/* Cards */
.card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
  transition: transform 0.2s, box-shadow 0.2s;
  display: flex;
  flex-direction: column;
}

.card:hover {
```

```
  transform: translateY(-3px);
  box-shadow: 0 10px 20px rgba(0,0,0,0.08);
}

.badge {
  display: inline-block;
  padding: 0.25rem 0.75rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
}

.badge-purple { background: var(--primary-light); color: var(--primary); }
.badge-blue { background: #e0f2fe; color: #0369a1; }
.badge-amber { background: #fef3c7; color: #b45309; }

/* Breadcrumbs */
.breadcrumb {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.9rem;
  color: var(--text-muted);
  margin-bottom: 1.5rem;
}

.breadcrumb a {
  color: var(--primary);
  text-decoration: none;
  font-weight: 600;
}

.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
  margin-top: auto;
}
```

- **src/App.jsx:**

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import Home from './pages/Home';
import About from './pages/About';
import Courses from './pages/Courses';
import CourseDetails from './pages/CourseDetails';
import Contact from './pages/Contact';
import NotFound from './pages/NotFound';

function App() {
  return (
    <div className="app-wrapper">
      {/* Persistent Navigation Bar */}
      <Navbar />

      {/* Main Content Area Rendering Current Route */}
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/courses" element={<Courses />} />
          <Route path="/courses/:courseId" element={<CourseDetails />} />
```

```
          <Route path="/contact" element={<Contact />} />
          {/* Wildcard 404 Route */}
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>

      {/* Persistent Footer */}
      <Footer />
    </div>
  );
}

export default App;
```

- **src/data/coursesData.js :**

```
export const COURSES = [
  {
    id: 'react-mastery',
    title: 'Modern React.js & Redux Toolkit',
    shortDesc: 'Master Functional Components, React Hooks, Context API, and modern state architecture.',
    fullDesc: 'This comprehensive course takes you from fundamental React concepts to advanced enterprise
patterns. You will build single page applications using React Router, manage complex global states with
Redux Toolkit, and master custom hooks for optimal performance.',
    duration: '8 Weeks',
    level: 'Intermediate',
    category: 'Frontend',
    instructor: 'Dr. Sarah Jenkins',
    rating: 4.9,
    enrolledStudents: 1240,
    modules: [
      'React Fundamentals & JSX Syntax',
      'Hooks in Depth (useState, useEffect, useMemo, useCallback)',
      'Single Page Routing with React Router v6',
      'Global State Management & Performance Optimization',
      'Testing with Vitest and React Testing Library'
    ],
    prerequisites: ['Basic JavaScript (ES6+)', 'HTML5 & CSS3']
  },
  {
    id: 'nodejs-express-api',
    title: 'Backend Engineering with Node.js & Express',
    shortDesc: 'Build secure, scalable RESTful APIs with Express, JWT authentication, and middleware.',
    fullDesc: 'Learn how to architect robust backend web servers using Node.js and Express.js framework.
Discover middleware design patterns, error handling, rate limiting, REST best practices, and integration
with databases.',
    duration: '6 Weeks',
    level: 'Intermediate',
    category: 'Backend',
    instructor: 'Prof. Michael Chang',
    rating: 4.8,
    enrolledStudents: 980,
    modules: [
      'Node.js Event Loop & Asynchronous Architecture',
      'Express.js Routing, Middleware & Controllers',
      'REST API Design & HTTP Status Codes',
      'Authentication & Authorization with JWT',
      'API Security, Rate Limiting & Deployment'
    ],
    prerequisites: ['Core JavaScript', 'Understanding of HTTP Protocol']
  },
  {
    id: 'fullstack-mern',
    title: 'Full Stack MERN Developer Bootcamp',
    shortDesc: 'End-to-end web applications combining MongoDB, Express, React, and Node.js.',
    fullDesc: 'Transform into a full-stack engineer by connecting MongoDB databases, Express backends,
and responsive React user interfaces into production-ready web platforms.',
    duration: '12 Weeks',
    level: 'Advanced',
```

```
    category: 'Full Stack',
    instructor: 'Alex Rivera',
    rating: 4.95,
    enrolledStudents: 2150,
    modules: [
      'Full Stack Architecture & Monorepos',
      'Mongoose Schemas, Models & MongoDB Atlas',
      'CRUD API Integration with React Query / Axios',
      'Stateful Authentication & Protected Routes',
      'Full CI/CD Deployment to Cloud Services'
    ],
    prerequisites: ['React Basics', 'Node.js Basics']
  },
  {
    id: 'mongodb-database-design',
    title: 'MongoDB Atlas & NoSQL Architecture',
    shortDesc: 'Data modeling, indexing, aggregation pipelines, and cloud database optimization.',
    fullDesc: 'Deep dive into document-oriented database design with MongoDB. Learn schema optimization,
indexes for lightning fast queries, complex aggregation pipelines, and cloud clustering with MongoDB
Atlas.',
    duration: '4 Weeks',
    level: 'Beginner',
    category: 'Database',
    instructor: 'Pooja Patel',
    rating: 4.7,
    enrolledStudents: 650,
    modules: [
      'Document DB vs Relational Databases',
      'MongoDB Atlas Setup & Security Configuration',
      'CRUD Operations & Query Operators',
      'Aggregation Pipeline Mastery',
      'Indexing & Query Performance Tuning'
    ],
    prerequisites: ['Basic understanding of JSON & databases']
  }
];
```

- **src/components/Navbar.jsx :**

```
import React from 'react';
import { NavLink, Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar">
      <Link to="/" className="nav-brand">
        <span> ⚡ </span> EduTech SPA
      </Link>
      <ul className="nav-links">
        <li className="nav-item">
          <NavLink to="/" end>Home</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/about">About Us</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/courses">Courses</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/contact">Contact</NavLink>
        </li>
      </ul>
    </nav>
  );
}

export default Navbar;
```

• **src/components/Footer.jsx :**

```
import React from 'react';

function Footer() {
  return (
    <footer className="footer">
      <p>Single Page Application (SPA) with React Router &bull; Practical 04</p>
    </footer>
  );
}

export default Footer;
```

• **src/pages/Home.jsx :**

```
import React from 'react';
import { Link } from 'react-router-dom';
import { COURSES } from '../data/coursesData';

function Home() {
  const featuredCourses = COURSES.slice(0, 3);

  return (
    <div>
      {/* Hero Section */}
      <section className="hero-section">
        <span className="badge badge-purple" style={{ marginBottom: '1rem' }}>
          Practical 04 Demonstration
        </span>
        <h1 className="hero-title">Single Page Application with React Router</h1>
        <p className="hero-subtitle">
          Experience instant client-side transitions, deep dynamic linking, active navigation styling,
          and breadcrumb path tracking without full-page reloads.
        </p>
        <div className="hero-buttons">
          <Link to="/courses" className="btn btn-primary">
            Explore All Courses →
          </Link>
          <Link to="/about" className="btn btn-outline">
            About This Practical
          </Link>
        </div>
      </section>

      {/* Highlights Grid */}
      <section style={{ marginBottom: '3rem' }}>
        <h2 style={{ fontSize: '1.6rem', fontWeight: 800, marginBottom: '1.5rem', textAlign: 'center' }}>
          ✨ React Router Architecture Highlights
        </h2>
        <div className="grid-3">
          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>⚡</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Client-Side
Routing</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Transitions between views happen dynamically in browser memory without sending full
document requests to the server.
            </p>
          </div>
          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>🔗</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Dynamic URL
Parameters</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Routes like <code>/courses/:courseId</code> extract URL slug variables via the
<code>useParams()</code> hook.
            </p>
          </div>
```

```
          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>🛡️</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Catch-All 404
Pages</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Unmatched route paths are automatically caught by the wildcard <code>path="*"</code>
fallback handler.
            </p>
          </div>
        </div>
      </section>

      {/* Featured Courses */}
      <section>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '1.5rem' }}>
          <h2 style={{ fontSize: '1.5rem', fontWeight: 800 }}>Featured Specializations</h2>
          <Link to="/courses" style={{ color: 'var(--primary)', textDecoration: 'none', fontWeight: 600,
fontSize: '0.95rem' }}>
            View All Courses &rarr;
          </Link>
        </div>
        <div className="grid-3">
          {featuredCourses.map(course => (
            <div key={course.id} className="card">
              <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '0.75rem' }}>
                <span className="badge badge-purple">{course.category}</span>
                <span style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>⭐
{course.rating}</span>
              </div>
              <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>{course.title}
</h3>
              <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem', marginBottom: '1.25rem', flex:
1 }}>
                {course.shortDesc}
              </p>
              <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
borderTop: '1px solid var(--border-color)', paddingTop: '1rem' }}>
                <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>⏰ {course.duration}
</span>
                <Link to={`/courses/${course.id}`} className="btn btn-primary btn-sm">
                  View Details &rarr;
                </Link>
              </div>
            </div>
          ))}
        </div>
      </section>
    </div>
  );
}

export default Home;
```

- **src/pages/Courses.jsx :**

```
import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { COURSES } from '../data/coursesData';

function Courses() {
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [search, setSearch] = useState('');

  const categories = ['All', 'Frontend', 'Backend', 'Full Stack', 'Database'];

  const filtered = COURSES.filter(course => {
    const matchCategory = selectedCategory === 'All' || course.category === selectedCategory;
    const matchSearch = course.title.toLowerCase().includes(search.toLowerCase()) ||
```

```
      course.shortDesc.toLowerCase().includes(search.toLowerCase());
    return matchCategory && matchSearch;
  });

  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>Courses Catalog</span>
      </div>

      <div style={{ marginBottom: '2rem' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '0.5rem' }}>
          Explore Technical Specializations
        </h1>
        <p style={{ color: 'var(--text-muted)', fontSize: '1rem' }}>
          Click on any course card to test <strong>Dynamic URL Route Parameters</strong> with
<code>/courses/:courseId</code>.
        </p>
      </div>

      {/* Filter and Search Bar */}
      <div style={{ display: 'flex', gap: '1rem', flexWrap: 'wrap', marginBottom: '2rem', alignItems:
'center' }}>
        <input
          type="text"
          placeholder="🔍 Search courses by keyword..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          style={{
            flex: 1,
            minWidth: '250px',
            padding: '0.65rem 1rem',
            borderRadius: '8px',
            border: '1px solid var(--border-color)',
            outline: 'none',
            fontSize: '0.95rem'
          }}
        />

        <div style={{ display: 'flex', gap: '0.4rem', flexWrap: 'wrap' }}>
          {categories.map(cat => (
            <button
              key={cat}
              onClick={() => setSelectedCategory(cat)}
              className={`btn btn-sm ${selectedCategory === cat ? 'btn-primary' : 'btn-outline'}`}
            >
              {cat}
            </button>
          ))}
        </div>
      </div>

      {/* Courses Grid */}
      {filtered.length === 0 ? (
        <div style={{ textAlign: 'center', padding: '3rem', color: 'var(--text-muted)' }}>
          <h3>No courses matched your query.</h3>
        </div>
      ) : (
        <div className="grid-2">
          {filtered.map(course => (
            <div key={course.id} className="card">
              <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '0.75rem' }}>
                <span className="badge badge-purple">{course.category}</span>
                <span style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>
                  🎯 {course.level}
                </span>
              </div>
              <h2 style={{ fontSize: '1.35rem', fontWeight: 700, marginBottom: '0.5rem' }}>
                {course.title}
```

```
              </h2>
              <p style={{ color: 'var(--text-muted)', fontSize: '0.925rem', marginBottom: '1.25rem',
flex: 1 }}>
                {course.shortDesc}
              </p>
              <div style={{ fontSize: '0.85rem', color: 'var(--text-muted)', marginBottom: '1rem' }}>
                <p>👩‍🏫 <strong>Instructor:</strong> {course.instructor}</p>
                <p>⏱️ <strong>Duration:</strong> {course.duration} &bull; ⭐ {course.rating} / 5.0</p>
              </div>
              <div style={{ borderTop: '1px solid var(--border-color)', paddingTop: '1rem', display:
'flex', justifyContent: 'space-between', alignItems: 'center' }}>
                <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
                  👥 {course.enrolledStudents} learners
                </span>
                <Link to={`/courses/${course.id}`} className="btn btn-primary btn-sm">
                  View Syllabus & Details →
                </Link>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

export default Courses;
```

• **src/pages/CourseDetails.jsx**:

```
import React from 'react';
import { useParams, Link, useNavigate } from 'react-router-dom';
import { COURSES } from '../data/coursesData';

function CourseDetails() {
  const { courseId } = useParams();
  const navigate = useNavigate();

  const course = COURSES.find(c => c.id === courseId);

  if (!course) {
    return (
      <div style={{ textAlign: 'center', padding: '4rem 1rem' }}>
        <h2 style={{ fontSize: '2rem', marginBottom: '1rem', color: '#ef4444' }}>Course Not Found</h2>
        <p style={{ color: 'var(--text-muted)', marginBottom: '1.5rem' }}>
          No course found with URL slug: <code>{courseId}</code>
        </p>
        <Link to="/courses" className="btn btn-primary">
          ← Back to Courses
        </Link>
      </div>
    );
  }

  return (
    <div>
      {/* Breadcrumb Navigation */}
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <Link to="/courses">Courses</Link> <span>/</span>
        <span>{course.title}</span>
      </div>

      <div style={{ maxWidth: '900px', margin: '0 auto' }}>
        {/* Course Header Banner */}
        <div className="card" style={{ background: 'linear-gradient(135deg, #1e1b4b 0%, #312e81 100%)',
color: 'white', marginBottom: '2rem' }}>
          <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem' }}>
            <span className="badge badge-purple" style={{ background: 'rgba(255,255,255,0.2)', color:
'white' }}>
```

```
              {course.category}
            </span>
            <span className="badge badge-amber">
              {course.level}
            </span>
          </div>
          <h1 style={{ fontSize: '2.4rem', fontWeight: 800, marginBottom: '0.75rem', lineHeight: 1.2 }}>
            {course.title}
          </h1>

          <p style={{ fontSize: '1.1rem', color: '#c7d2fe', marginBottom: '1.5rem', lineHeight: 1.6 }}>
            {course.fullDesc}
          </p>

          <div style={{ display: 'flex', gap: '2rem', flexWrap: 'wrap', fontSize: '0.95rem', borderTop:
'1px solid rgba(255,255,255,0.15)', paddingTop: '1.25rem' }}>
            <div>👨‍🏫 <strong>Instructor:</strong> {course.instructor}</div>
            <div>⏰ <strong>Duration:</strong> {course.duration}</div>
            <div>⭐ <strong>Rating:</strong> {course.rating} / 5.0</div>
            <div>👥 <strong>Enrolled:</strong> {course.enrolledStudents} students</div>
          </div>
        </div>

        {/* Dynamic Route Info Box */}
        <div style={{
          background: '#f0fdf4',
          border: '1px solid #bbf7d0',
          borderRadius: '12px',
          padding: '1.25rem',
          marginBottom: '2rem',
          fontSize: '0.9rem',
          color: '#166534'
        }}>
          🔍 <strong>Dynamic Parameter Inspection:</strong> This page received URL parameter
<code>useParams().courseId = "{courseId}"</code>.
        </div>

        {/* Course Modules */}
        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.3rem', fontWeight: 700, marginBottom: '1rem' }}>📖 Course Curriculum
& Modules</h3>
          <ul style={{ listStyle: 'none', display: 'flex', flexDirection: 'column', gap: '0.75rem' }}>
            {course.modules.map((mod, index) => (
              <li key={index} style={{
                display: 'flex',
                alignItems: 'center',
                gap: '0.75rem',
                padding: '0.75rem 1rem',
                background: '#f8fafc',
                borderRadius: '8px',
                border: '1px solid var(--border-color)',
                fontSize: '0.95rem'
              }}>
                <span style={{
                  background: 'var(--primary-light)',
                  color: 'var(--primary)',
                  fontWeight: 700,
                  width: '28px',
                  height: '28px',
                  borderRadius: '50%',
                  display: 'flex',
                  alignItems: 'center',
                  justifyContent: 'center',
                  fontSize: '0.85rem'
                }}>
                  {index + 1}
                </span>
                <span>{mod}</span>
              </li>
            ))}
```

```
          </ul>
        </div>

        {/* Prerequisites & Action */}
        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.3rem', fontWeight: 700, marginBottom: '0.75rem' }}>Prerequisites</h3>
          <div style={{ display: 'flex', gap: '0.5rem', flexWrap: 'wrap', marginBottom: '1.5rem' }}>
            {course.prerequisites.map(p => (
              <span key={p} className="badge badge-blue">{p}</span>
            ))}
          </div>
          <div style={{ display: 'flex', gap: '1rem', borderTop: '1px solid var(--border-color)',
paddingTop: '1.25rem' }}>
            <button
              className="btn btn-primary"
              onClick={() => alert(`🎉 You have registered for: ${course.title}`)}
            >
              🚀 Enroll in this Specialization
            </button>
            <button
              className="btn btn-outline"
              onClick={() => navigate('/courses')}
            >
              ← Back to Catalog (useNavigate)
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

export default CourseDetails;
```

• **src/pages/About.jsx :**

```
import React from 'react';
import { Link } from 'react-router-dom';

function About() {
  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>About Us</span>
      </div>
      <div style={{ maxWidth: '850px', margin: '0 auto' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '1rem', color: '#1e1b4b' }}>
          About Our Single Page Application
        </h1>
        <p style={{ fontSize: '1.1rem', color: 'var(--text-muted)', lineHeight: 1.7, marginBottom: '2rem'
}}>
          This project illustrates the core mechanics of modern client-side routing in single-page
applications.
          By utilizing the HTML5 History API (via React Router), our application provides an instant,
desktop-app like
          user experience with instantaneous page switches.
        </p>

        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.25rem', fontWeight: 700, marginBottom: '1rem' }}>
            ⚛️ Core Concepts Covered in Practical 04:
          </h3>
          <ul style={{ paddingLeft: '1.5rem', lineHeight: 2, color: 'var(--text-main)', fontSize:
'0.95rem' }}>
            <li><strong>BrowserRouter:</strong> Wraps the root application and syncs the UI with the
browser address bar.</li>
            <li><strong>Routes & Route:</strong> Declaratively maps URL path patterns to specific page
components.</li>
            <li><strong>Link & NavLink:</strong> Custom navigation components that intercept browser
```

```
clicks and avoid page refreshes.</li>
            <li><strong>useParams Hook:</strong> Reads dynamic URL parameters (e.g.,
<code>:courseId</code>) directly into components.</li>
            <li><strong>useNavigate Hook:</strong> Programmatically triggers navigation from handlers and
button callbacks.</li>
            <li><strong>Wildcard Route (`*`):</strong> Seamlessly handles 404 errors when visitors input
unknown URLs.</li>
          </ul>
        </div>

        <div className="grid-2">
          <div className="card">
            <h4 style={{ fontSize: '1.1rem', fontWeight: 700, marginBottom: '0.5rem' }}>⚡ Zero Page
Reloads</h4>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              All navigation happens without white flashes or waiting for the server to render full HTML
documents.
            </p>
          </div>
          <div className="card">
            <h4 style={{ fontSize: '1.1rem', fontWeight: 700, marginBottom: '0.5rem' }}>🔗 Bookmarkable
URLs</h4>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Every view has a unique address bar URL that can be directly bookmarked, refreshed, or
shared.
            </p>
          </div>
        </div>
      </div>
    </div>
  );
}

export default About;
```

• **src/pages/Contact.jsx :**

```
import React, { useState } from 'react';
import { Link } from 'react-router-dom';

function Contact() {
  const [submitted, setSubmitted] = useState(false);
  const [formData, setFormData] = useState({ name: '', email: '', subject: '', message: '' });

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(true);
  };

  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>Contact</span>
      </div>

      <div style={{ maxWidth: '850px', margin: '0 auto' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '0.5rem' }}>Contact Department
Support</h1>
        <p style={{ color: 'var(--text-muted)', marginBottom: '2rem' }}>
          Have questions regarding the curriculum or practical journal? Send us a message.
        </p>

        <div className="grid-2">
          {/* Contact Details Card */}
          <div className="card" style={{ background: '#f8fafc' }}>
            <h3 style={{ fontSize: '1.25rem', fontWeight: 700, marginBottom: '1rem' }}>🏢 Department
Office</h3>
            <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem', fontSize: '0.925rem',
color: 'var(--text-muted)' }}>
```

```
            <p>📍 <strong>Address:</strong> Department of Computer Engineering, Technology Campus</p>
            <p>✉️ <strong>Email:</strong> webtech.lab@college.edu</p>
            <p>📞 <strong>Phone:</strong> +91 98765 43210</p>
            <p>⏰ <strong>Lab Hours:</strong> Mon - Fri: 9:00 AM - 5:00 PM</p>
          </div>
        </div>

        {/* Contact Form */}
        <div className="card">
          {submitted ? (
            <div style={{ textAlign: 'center', padding: '2rem 1rem' }}>
              <div style={{ fontSize: '3rem', marginBottom: '0.5rem' }}>✅</div>
              <h3 style={{ fontSize: '1.25rem', fontWeight: 700, color: 'var(--primary)' }}>Message
Received!</h3>
              <p style={{ color: 'var(--text-muted)', margin: '0.75rem 0 1.5rem', fontSize: '0.9rem'
}}>
                Thank you, <strong>{formData.name}</strong>. We will get back to your email
({formData.email}) shortly.
              </p>
              <button
                className="btn btn-outline btn-sm"
                onClick={() => { setSubmitted(false); setFormData({ name: '', email: '', subject: '',
message: '' }); }}
              >
                Send Another Inquiry
              </button>
            </div>
          ) : (
            <form onSubmit={handleSubmit}>
              <div style={{ marginBottom: '1rem' }}>
                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom:
'0.25rem' }}>Full Name</label>
                <input
                  type="text"
                  required
                  placeholder="Your Name"
                  value={formData.name}
                  onChange={(e) => setFormData({ ...formData, name: e.target.value })}
                  style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px', border: '1px
solid var(--border-color)', outline: 'none' }}
                />
              </div>
              <div style={{ marginBottom: '1rem' }}>
                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom:
'0.25rem' }}>Email Address</label>
                <input
                  type="email"
                  required
                  placeholder="Your Email"
                  value={formData.email}
                  onChange={(e) => setFormData({ ...formData, email: e.target.value })}
                  style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px', border: '1px
solid var(--border-color)', outline: 'none' }}
                />
              </div>
              <div style={{ marginBottom: '1rem' }}>
                <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom:
'0.25rem' }}>Message</label>
                <textarea
                  rows="3"
                  required
                  placeholder="Your query or feedback..."
                  value={formData.message}
                  onChange={(e) => setFormData({ ...formData, message: e.target.value })}
                  style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px', border: '1px
solid var(--border-color)', outline: 'none', fontFamily: 'inherit' }}
                ></textarea>
              </div>
              <button type="submit" className="btn btn-primary" style={{ width: '100%' }}>
                ✉️ Send Message
```

```
                </button>
              </form>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}

export default Contact;
```

• **src/pages/NotFound.jsx :**

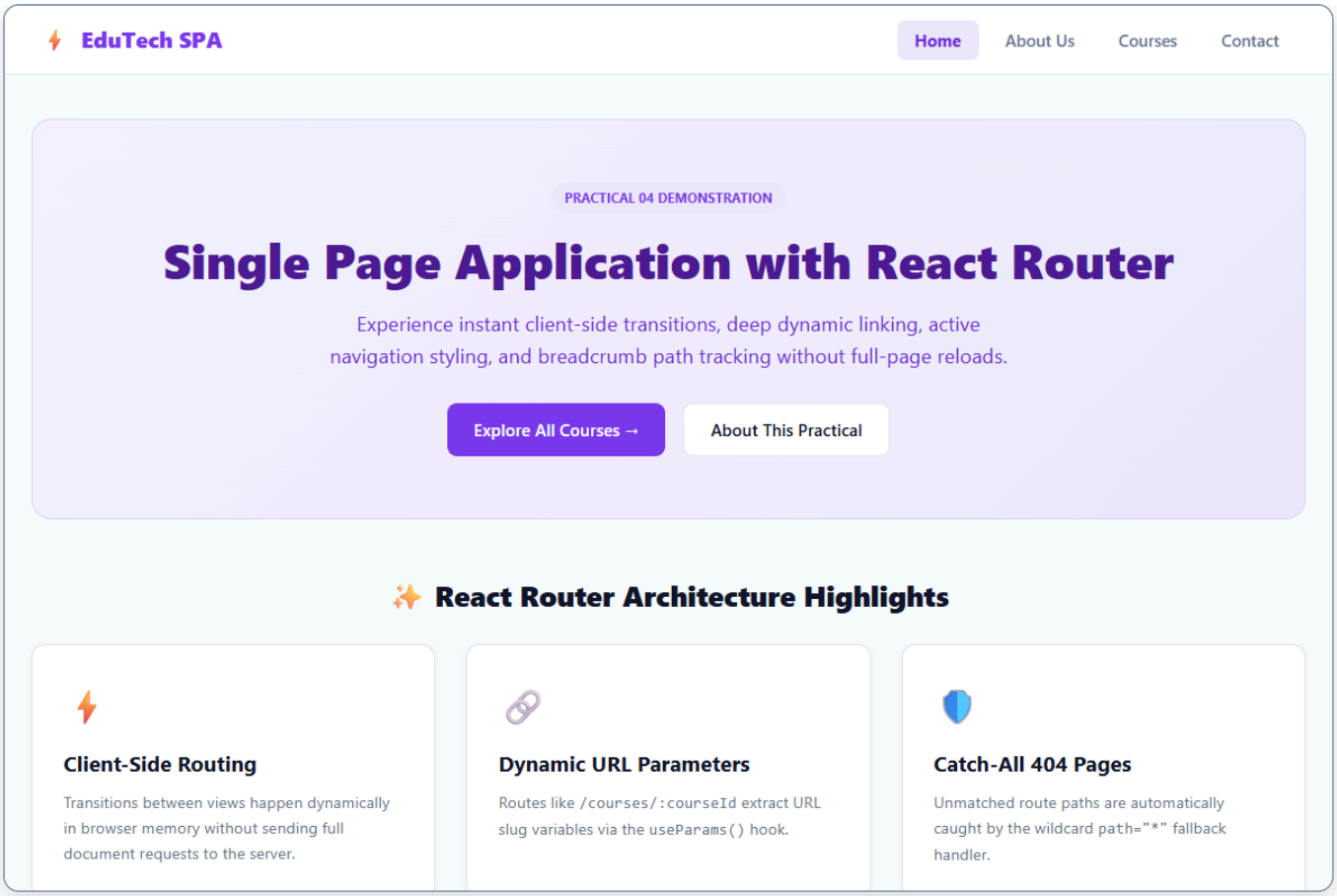
```
import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div style={{ textAlign: 'center', padding: '5rem 1rem' }}>
      <h1 style={{ fontSize: '5rem', fontWeight: 900, color: 'var(--primary)', marginBottom: '0.5rem'
}}>404</h1>
      <h2 style={{ fontSize: '1.75rem', fontWeight: 700, marginBottom: '1rem' }}>Page Not Found</h2>
      <p style={{ color: 'var(--text-muted)', marginBottom: '2rem' }}>
        The page you are looking for does not exist or has been moved.
      </p>
      <Link to="/" className="btn btn-primary">
        Return to Home
      </Link>
    </div>
  );
}

export default NotFound;
```

4. Output :

Single Page Application with React Router v6, dynamic route slugs (:courseId) via useParams, programmatic navigation via useNavigate, and 404 wildcard fallback handling.


EduTech SPA
Home
About Us
Courses
Contact
PRACTICAL 04 DEMONSTRATION
Single Page Application with React Router
Experience instant client-side transitions, deep dynamic linking, active navigation styling, and breadcrumb path tracking without full-page reloads.
Explore All Courses →
About This Practical
✨ React Router Architecture Highlights
Client-Side Routing
Transitions between views happen dynamically in browser memory without sending full document requests to the server.
Dynamic URL Parameters
Routes like /courses/:courseId extract URL slug variables via the useParams() hook.
Catch-All 404 Pages
Unmatched route paths are automatically caught by the wildcard path="*" fallback handler.

Practical – 05 : Create a backend server using Express.js implementing routing and CRUD operation.

1. Commands :

```
mkdir MERN_05 && cd MERN_05
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_05/
|-- package.json
|-- server.js
|-- routes/productRoutes.js
|-- controllers/productController.js
|-- middleware/logger.js
|-- middleware/errorHandler.js
|-- data/products.json
|-- public/index.html
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-05",
  "version": "1.0.0",
  "description": "Express.js Backend Server implementing Modular Routing and CRUD operations",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const requestLogger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');
const productRoutes = require('./routes/productRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Built-in & Third-party Middleware
app.use(cors());
app.use(express.json()); // Parses application/json body
app.use(express.urlencoded({ extended: true }));
app.use(requestLogger); // Custom Request Logger

// Static visual playground for browser testing & verification
app.get('/playground', (req, res) => {
```

```
  res.send(`
<!DOCTYPE html>
<html>
<head>
  <title>Express CRUD Tester</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: #0f172a; color:
white; padding: 2rem; }
    .card { background: #1e293b; padding: 1.5rem; border-radius: 8px; margin-bottom: 1rem; border: 1px
solid #334155; }
    button { background: #6366f1; color: white; border: none; padding: 0.5rem 1rem; border-radius: 6px;
cursor: pointer; margin-right: 0.5rem; }
    button:hover { background: #4f46e5; }
    pre { background: #090d16; padding: 1rem; border-radius: 6px; overflow: auto; max-height: 300px;
color: #38bdf8; }
  </style>
</head>
<body>
  <h1>⚡ Express.js Product CRUD Test Playground</h1>
  <div class="card">
    <button onclick="fetchApi('/api/products')">GET All Products</button>
    <button onclick="fetchApi('/api/products/1')">GET Product 1</button>
    <button onclick="createProd()">POST New Product</button>
    <button onclick="deleteProd()">DELETE Product 4</button>
  </div>
  <div class="card">
    <h3>API Output:</h3>
    <pre id="output">Click a button above to test API endpoints...</pre>
  </div>
  <script>
    async function fetchApi(url) {
      const res = await fetch(url);
      const data = await res.json();
      document.getElementById('output').textContent = JSON.stringify(data, null, 2);
    }
    async function createProd() {
      const res = await fetch('/api/products', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: 'Gaming Chair RGB', category: 'Furniture', price: 12999, stock: 15
})
      });
      const data = await res.json();
      document.getElementById('output').textContent = JSON.stringify(data, null, 2);
    }
    async function deleteProd() {
      const res = await fetch('/api/products/4', { method: 'DELETE' });
      const data = await res.json();
      document.getElementById('output').textContent = JSON.stringify(data, null, 2);
    }
  </script>
</body>
</html>
  `);
});

// Welcome / Health Route
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Welcome to Practical 05: Express.js Routing & CRUD API Server',
    endpoints: {
      getAllProducts: 'GET /api/products',
      getProductById: 'GET /api/products/:id',
      createProduct: 'POST /api/products',
      updateProduct: 'PUT /api/products/:id',
      deleteProduct: 'DELETE /api/products/:id',
      testPlayground: 'GET /playground'
    },
    timestamp: new Date().toISOString()
```

```
  });
});

// Modular Routes Registration
app.use('/api/products', productRoutes);

// Fallback 404 Route Handler for undefined endpoints
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Cannot ${req.method} ${req.originalUrl}. Route not found.`
  });
});

// Central Error Handler Middleware
app.use(errorHandler);

// Start HTTP Server
app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`⚛️ Express CRUD Server is running on Port: ${PORT}`);
  console.log(`⚛️ Local URL: http://localhost:${PORT}`);
  console.log(`⚛️ Products Endpoint: http://localhost:${PORT}/api/products`);
  console.log(`================================================`);
});
```

- **routes/productRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
} = require('../controllers/productController');

// Define REST routes for /api/products
router.route('/')
  .get(getAllProducts)
  .post(createProduct);

router.route('/:id')
  .get(getProductById)
  .put(updateProduct)
  .delete(deleteProduct);

module.exports = router;
```

- **controllers/productController.js :**

```
const fs = require('fs');
const path = require('path');

const dataFilePath = path.join(__dirname, '..', 'data', 'products.json');

// Helper to read products
const readProducts = () => {
  try {
    const data = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(data);
  } catch (error) {
    return [];
  }
};

// Helper to save products
```

```
const saveProducts = (products) => {
  fs.writeFileSync(dataFilePath, JSON.stringify(products, null, 2), 'utf8');
};

/**
 * @desc Get all products (with optional filtering & search)
 * @route GET /api/products
 */
const getAllProducts = (req, res) => {
  let products = readProducts();
  const { category, search, minPrice, maxPrice } = req.query;

  // Filter by category
  if (category) {
    products = products.filter(p => p.category.toLowerCase() === category.toLowerCase());
  }

  // Search by name
  if (search) {
    products = products.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
  }

  // Filter by price range
  if (minPrice) {
    products = products.filter(p => p.price >= parseFloat(minPrice));
  }
  if (maxPrice) {
    products = products.filter(p => p.price <= parseFloat(maxPrice));
  }

  res.status(200).json({
    success: true,
    count: products.length,
    data: products
  });
};

/**
 * @desc Get single product by ID
 * @route GET /api/products/:id
 */
const getProductById = (req, res) => {
  const products = readProducts();
  const productId = parseInt(req.params.id);
  const product = products.find(p => p.id === productId);

  if (!product) {
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} was not found.`
    });
  }

  res.status(200).json({
    success: true,
    data: product
  });
};

/**
 * @desc Create new product
 * @route POST /api/products
 */
const createProduct = (req, res) => {
  const { name, category, price, stock } = req.body;

  // Validation
  if (!name || !category || price === undefined) {
    return res.status(400).json({
      success: false,
```

```
      message: 'Validation failed: "name", "category", and "price" are required fields.'
    });
  }

  if (isNaN(price) || price < 0) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed: "price" must be a positive number.'
    });
  }

  const products = readProducts();
  const newProduct = {
    id: products.length > 0 ? Math.max(...products.map(p => p.id)) + 1 : 1,
    name: name.trim(),
    category: category.trim(),
    price: parseFloat(price),
    stock: stock !== undefined ? parseInt(stock) : 0,
    inStock: stock !== undefined ? parseInt(stock) > 0 : true,
    createdAt: new Date().toISOString()
  };

  products.push(newProduct);
  saveProducts(products);

  res.status(201).json({
    success: true,
    message: 'Product created successfully.',
    data: newProduct
  });
};

/**
 * @desc Update product completely (PUT)
 * @route PUT /api/products/:id
 */
const updateProduct = (req, res) => {
  const productId = parseInt(req.params.id);
  const { name, category, price, stock, inStock } = req.body;

  const products = readProducts();
  const index = products.findIndex(p => p.id === productId);

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} does not exist.`
    });
  }

  // Full update
  products[index] = {
    ...products[index],
    name: name !== undefined ? name.trim() : products[index].name,
    category: category !== undefined ? category.trim() : products[index].category,
    price: price !== undefined ? parseFloat(price) : products[index].price,
    stock: stock !== undefined ? parseInt(stock) : products[index].stock,
    inStock: inStock !== undefined ? inStock : (stock !== undefined ? parseInt(stock) > 0 :
products[index].inStock),
    updatedAt: new Date().toISOString()
  };

  saveProducts(products);

  res.status(200).json({
    success: true,
    message: 'Product updated successfully.',
    data: products[index]
  });
};
```

```
/**
 * @desc Delete product by ID
 * @route DELETE /api/products/:id
 */
const deleteProduct = (req, res) => {
  const productId = parseInt(req.params.id);
  const products = readProducts();
  const productToDelete = products.find(p => p.id === productId);

  if (!productToDelete) {
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} not found.`
    });
  }

  const updatedProducts = products.filter(p => p.id !== productId);
  saveProducts(updatedProducts);

  res.status(200).json({
    success: true,
    message: `Product "${productToDelete.name}" (ID: ${productId}) has been deleted.`,
    data: productToDelete
  });
};

module.exports = {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
};
```

- **middleware/logger.js :**

```
/**
 * Request Logger Middleware
 * Logs HTTP Method, Request Path, Status Code & Time Taken
 */
const requestLogger = (req, res, next) => {
  const start = Date.now();
  const timestamp = new Date().toISOString();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`[${timestamp}] ${req.method} ${req.originalUrl} | Status: ${res.statusCode}
(${duration}ms)`);
  });
  next();
};

module.exports = requestLogger;
```

- **middleware/errorHandler.js :**

```
/**
 * Central Error Handling Middleware
 */
const errorHandler = (err, req, res, next) => {
  console.error('Server Error:', err.stack || err.message);
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  res.status(statusCode).json({
    success: false,
    message: err.message || 'Internal Server Error',
    error: process.env.NODE_ENV === 'production' ? null : err.stack
  });
};
```

```
module.exports = errorHandler;
```

- **data/products.json :**

```
[
  {
    "id": 1,
    "name": "Mechanical Keyboard RGB",
    "category": "Electronics",
    "price": 3499,
    "stock": 25,
    "inStock": true,
    "createdAt": "2025-01-10T10:00:00.000Z"
  },
  {
    "id": 2,
    "name": "Wireless Ergonomic Mouse",
    "category": "Electronics",
    "price": 1799,
    "stock": 40,
    "inStock": true,
    "createdAt": "2025-01-12T11:30:00.000Z"
  },
  {
    "id": 3,
    "name": "27-inch 4K UHD Monitor",
    "category": "Displays",
    "price": 24999,
    "stock": 10,
    "inStock": true,
    "createdAt": "2025-01-15T14:15:00.000Z"
  },
  {
    "id": 5,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:14.964Z"
  },
  {
    "id": 6,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:17.755Z"
  },
  {
    "id": 7,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:18.202Z"
  },
  {
    "id": 8,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:18.410Z"
  },
```

```
  {
    "id": 9,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:18.601Z"
  },
  {
    "id": 10,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:18.793Z"
  },
  {
    "id": 11,
    "name": "Gaming Chair RGB",
    "category": "Furniture",
    "price": 12999,
    "stock": 15,
    "inStock": true,
    "createdAt": "2026-10-21T10:50:18.993Z"
  }
]
```

- **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 05 - Express.js Routing &amp;amp; CRUD</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
background: #f8fafc; color: #0f172a; }

    /* Header */
    .header {
      background: linear-gradient(135deg, #1e1b4b 0%, #312e81 50%, #4338ca 100%);
      color: white;
      padding: 2.2rem 1.5rem;
      text-align: center;
      position: relative;
    }
    .header-badge {
      display: inline-block;
      background: rgba(255, 255, 255, 0.2);
      padding: 0.3rem 0.9rem;
      border-radius: 9999px;
      font-size: 0.8rem;
      font-weight: 700;
      letter-spacing: 0.05em;
      text-transform: uppercase;
      margin-bottom: 0.6rem;
    }
    .header-title { font-size: 2rem; font-weight: 800; margin-bottom: 0.4rem; }
    .header-subtitle { font-size: 0.95rem; color: #c7d2fe; }

    /* Main Container */
    .container { max-width: 1100px; margin: 0 auto; padding: 1.75rem 1.25rem; }

    .banner {
      background: #eff6ff;
      border: 1px solid #bfdbfe;
```

```
      border-left: 4px solid #3b82f6;
      border-radius: 8px;
      padding: 1rem 1.25rem;
      margin-bottom: 1.5rem;
      font-size: 0.88rem;
      color: #1e40af;
    }
    .banner h4 { margin-bottom: 0.25rem; font-weight: 700; display: flex; align-items: center; gap:
0.4rem; }

    .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; }
    .card { background: white; border: 1px solid #e2e8f0; border-radius: 12px; padding: 1.5rem; box-
shadow: 0 1px 3px rgba(0,0,0,0.05); }
    .card-title { font-size: 1.15rem; font-weight: 700; margin-bottom: 1.2rem; display: flex; align-
items: center; gap: 0.5rem; color: #1e293b; }

    /* Endpoint List */
    .ep-item { display: flex; justify-content: space-between; align-items: center; padding: 0.65rem
0.75rem; border: 1px solid #e2e8f0; border-radius: 8px; margin-bottom: 0.7rem; background: #f8fafc; }
    .ep-left { display: flex; align-items: center; gap: 0.6rem; font-family: monospace; font-size:
0.88rem; }
    .method-get { background: #dbeafe; color: #1d4ed8; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-post { background: #dcfce7; color: #15803d; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-put { background: #fef3c7; color: #b45309; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-del { background: #fee2e2; color: #b91c1c; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }

    .btn-test { background: #4f46e5; color: white; border: none; padding: 0.35rem 0.8rem; border-radius:
6px; font-size: 0.8rem; font-weight: 600; cursor: pointer; }
    .btn-test:hover { background: #4338ca; }

    /* Console Terminal */
    .console-box { background: #0b1329; color: #f8fafc; border-radius: 8px; padding: 1.25rem; font-
family: 'Consolas', 'Courier New', monospace; font-size: 0.85rem; line-height: 1.5; min-height: 380px;
overflow-y: auto; }
    .console-header { color: #60a5fa; font-weight: 600; margin-bottom: 0.75rem; border-bottom: 1px solid
#1e293b; padding-bottom: 0.5rem; display: flex; align-items: center; gap: 0.4rem; }
    .json-key { color: #93c5fd; }
    .json-str { color: #86efac; }
    .json-num { color: #fde047; }
    .json-bool { color: #f472b6; }
    .tip-box { margin-top: 1rem; border-top: 1px solid #1e293b; padding-top: 0.75rem; color: #94a3b8;
font-size: 0.78rem; display: flex; align-items: center; gap: 0.4rem; }
  </style>
</head>
<body>
  <div class="header">
    <div class="header-badge">Practical No. 05</div>
    <h1 class="header-title">Express.js Backend Server &amp;amp; CRUD Routing</h1>
    <p class="header-subtitle">Interactive API Playground, Route Visualizer &amp;amp; Product CRUD
Operations</p>
  </div>

  <div class="container">
    <div class="banner">
      <h4>💡 How to Run the Express Server</h4>
      <p>Open command prompt inside this folder and run: <code>node server.js</code>. The server will
start on port 5000. You can test endpoints below directly or through Postman / cURL!</p>
    </div>

    <div class="grid">
      <div class="card">
        <h3 class="card-title">📡 API Route Endpoints</h3>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/products</div>
          <button class="btn-test" onclick="testGetAll()">Test GET All</button>
```

```
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/products/1</div>
          <button class="btn-test" onclick="testGetOne()">Test GET by ID</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-post">POST</span> /api/products</div>
          <button class="btn-test" onclick="testPost()">Test POST</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-put">PUT</span> /api/products/1</div>
          <button class="btn-test" onclick="testPut()">Test PUT</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-del">DELETE</span> /api/products/2</div>
          <button class="btn-test" style="background: #dc2626;" onclick="testDelete()">Test
DELETE</button>
        </div>
      </div>

      <div class="card">
        <h3 class="card-title">🖥️ Live JSON Response Console</h3>
        <div class="console-box">
          <div class="console-header">
            <span>●</span> Live Server at localhost:5000 is running correctly.<br>
            <span>Displaying Express.js API Response:</span>
          </div>
          <pre id="code-content" style="white-space: pre-wrap;">{
  "success": true,
  "count": 4,
  "data": [
    {
      "id": 1,
      "name": "Mechanical Keyboard RGB",
      "category": "Electronics",
      "price": 3499,
      "stock": 25,
      "inStock": true,
      "createdAt": "2025-01-10T10:00:00.000Z"
    },
    {
      "id": 2,
      "name": "Wireless Ergonomic Mouse",
      "category": "Electronics",
      "price": 1799,
      "stock": 40,
      "inStock": true
    }
  ]
}</pre>
          <div class="tip-box">
            💡 Tip: Run 'node server.js' in terminal to start the live Node.js server!
          </div>
        </div>
      </div>
    </div>
  </div>

  <script>
    async function testGetAll() {
      try {
        const res = await fetch('/api/products');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
```

```
    async function testGetOne() {
      try {
        const res = await fetch('/api/products/1');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testPost() {
      try {
        const res = await fetch('/api/products', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ name: 'Gaming Headset', category: 'Audio', price: 2999, stock: 15 })
        });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testPut() {
      try {
        const res = await fetch('/api/products/1', {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ price: 3299, stock: 20 })
        });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testDelete() {
      try {
        const res = await fetch('/api/products/2', { method: 'DELETE' });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
  </script>
</body>
</html>
```

4. Output :

Express.js backend server with modular router, custom request logging middleware, central error handling, and complete CRUD operations on product entities.

PRACTICAL NO. 05

Express.js Backend Server & CRUD Routing

Interactive API Playground, Route Visualizer & Product CRUD Operations

How to Run the Express Server

Open command prompt inside this folder and run: node server.js. The server will start on port 5000. You can test endpoints below directly or through Postman / cURL!

API Route Endpoints

GET	/api/products	Test GET All
GET	/api/products/1	Test GET by ID
POST	/api/products	Test POST
PUT	/api/products/1	Test PUT
DELETE	/api/products/2	Test DELETE

Live JSON Response Console

```
Live Server at localhost:5000 is running correctly.   Displaying Express.js API Response:

{
  "success": true,
  "count": 4,
  "data": [
    {
      "id": 1,
      "name": "Mechanical Keyboard RGB",
      "category": "Electronics",
      "price": 3499,
      "stock": 25,
      "inStock": true,
      "createdAt": "2025-01-10T10:00:00.000Z"
    },
    {
      "id": 2,
      "name": "Wireless Ergonomic Mouse",
```

Practical – 06 : Build RESTful APIs for Student Records Management using Express.js and Postman testing.

1. Commands :

```
mkdir MERN_06 && cd MERN_06
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_06/
|-- package.json
|-- server.js
|-- routes/studentRoutes.js
|-- controllers/studentController.js
|-- middleware/validator.js
|-- middleware/logger.js
|-- data/students.json
|-- public/index.html
```

3. Source Code :

- **package.json :**

```
{
  "name": "mern-06",
  "version": "1.0.0",
  "description": "Student Records REST API using Express.js with JSON Data Store & Postman Collection",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const requestLogger = require('./middleware/logger');
const studentRoutes = require('./routes/studentRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(requestLogger);

// Home route
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
```

```
    message: 'Practical 06: RESTful APIs for Student Records Management',
    endpoints: {
      getAllStudents: 'GET /api/students',
      getStudentByIdOrRoll: 'GET /api/students/:id',
      createStudent: 'POST /api/students',
      updateStudent: 'PUT /api/students/:id',
      deleteStudent: 'DELETE /api/students/:id',
      getOverviewStats: 'GET /api/students/stats/overview'
    },
    postmanCollection: 'Student_Records_API.postman_collection.json'
  });
});

app.use('/api/students', studentRoutes);

// Fallback 404
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Resource ${req.originalUrl} not found on this server.`
  });
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`⚛️ Practical 06: Student Records REST API Server`);
  console.log(`⚛️ Server running at: http://localhost:${PORT}`);
  console.log(`⚛️ Base API Endpoint: http://localhost:${PORT}/api/students`);
  console.log(`⚛️ Postman Collection ready in project folder`);
  console.log(`================================================`);
});
```

- **routes/studentRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent,
  getStudentStats
} = require('../controllers/studentController');
const { validateStudentPayload } = require('../middleware/validator');

// Analytics / Stats endpoint (placed before :id route)
router.get('/stats/overview', getStudentStats);

// Main Student REST endpoints
router.route('/')
  .get(getAllStudents)
  .post(validateStudentPayload, createStudent);

router.route('/:id')
  .get(getStudentById)
  .put(validateStudentPayload, updateStudent)
  .delete(deleteStudent);

module.exports = router;
```

- **controllers/studentController.js :**

```
const fs = require('fs');
const path = require('path');

const dataFilePath = path.join(__dirname, '..', 'data', 'students.json');
```

```
const getStudentsData = () => {
  try {
    const raw = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(raw);
  } catch (err) {
    return [];
  }
};

const saveStudentsData = (data) => {
  fs.writeFileSync(dataFilePath, JSON.stringify(data, null, 2), 'utf8');
};

/**
 * @desc Get All Students with search, filtering, sorting, pagination
 * @route GET /api/students
 */
const getAllStudents = (req, res) => {
  let students = getStudentsData();
  const { department, semester, minCgpa, search, sortBy } = req.query;

  // Filter by department
  if (department) {
    students = students.filter(s => s.department.toLowerCase() === department.toLowerCase());
  }

  // Filter by semester
  if (semester) {
    students = students.filter(s => s.semester === parseInt(semester));
  }

  // Filter by minimum CGPA
  if (minCgpa) {
    students = students.filter(s => s.cgpa >= parseFloat(minCgpa));
  }

  // Search by keyword (name, rollNo, email)
  if (search) {
    const q = search.toLowerCase();
    students = students.filter(s =>
      s.name.toLowerCase().includes(q) ||
      s.rollNo.toLowerCase().includes(q) ||
      s.email.toLowerCase().includes(q)
    );
  }

  // Sort by cgpa or name
  if (sortBy === 'cgpa_desc') {
    students.sort((a, b) => b.cgpa - a.cgpa);
  } else if (sortBy === 'name') {
    students.sort((a, b) => a.name.localeCompare(b.name));
  }

  res.status(200).json({
    success: true,
    totalRecords: students.length,
    data: students
  });
};

/**
 * @desc Get Student By ID or Roll Number
 * @route GET /api/students/:id
 */
const getStudentById = (req, res) => {
  const students = getStudentsData();
  const param = req.params.id;
  const student = students.find(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
param.toLowerCase());
```

```
  if (!student) {
    return res.status(404).json({
      success: false,
      message: `Student with ID or Roll No "${param}" was not found.`
    });
  }

  res.status(200).json({
    success: true,
    data: student
  });
};

/**
 * @desc Create New Student Record
 * @route POST /api/students
 */
const createStudent = (req, res) => {
  const students = getStudentsData();
  const { rollNo, name, email, phone, department, semester, cgpa } = req.body;

  // Check unique rollNo
  const existingRoll = students.find(s => s.rollNo.toLowerCase() === rollNo.trim().toLowerCase());
  if (existingRoll) {
    return res.status(409).json({
      success: false,
      message: `Student with Roll Number "${rollNo}" already exists in records.`
    });
  }

  // Check unique email
  const existingEmail = students.find(s => s.email.toLowerCase() === email.trim().toLowerCase());
  if (existingEmail) {
    return res.status(409).json({
      success: false,
      message: `Student with Email "${email}" is already registered.`
    });
  }

  const nextId = students.length > 0 ? Math.max(...students.map(s => s.id)) + 1 : 1;
  const newStudent = {
    id: nextId,
    rollNo: rollNo.trim().toUpperCase(),
    name: name.trim(),
    email: email.trim().toLowerCase(),
    phone: phone ? phone.trim() : 'N/A',
    department: department.trim(),
    semester: parseInt(semester),
    cgpa: cgpa !== undefined ? parseFloat(cgpa) : 0.0,
    enrolledDate: new Date().toISOString().split('T')[0]
  };

  students.push(newStudent);
  saveStudentsData(students);

  res.status(201).json({
    success: true,
    message: 'Student record created successfully.',
    data: newStudent
  });
};

/**
 * @desc Update Student Record
 * @route PUT /api/students/:id
 */
const updateStudent = (req, res) => {
  const students = getStudentsData();
  const param = req.params.id;
  const index = students.findIndex(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
```

```
param.toLowerCase());

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: `Student record "${param}" not found for update.`
    });
  }

  const { name, email, phone, department, semester, cgpa } = req.body;

  students[index] = {
    ...students[index],
    name: name !== undefined ? name.trim() : students[index].name,
    email: email !== undefined ? email.trim().toLowerCase() : students[index].email,
    phone: phone !== undefined ? phone.trim() : students[index].phone,
    department: department !== undefined ? department.trim() : students[index].department,
    semester: semester !== undefined ? parseInt(semester) : students[index].semester,
    cgpa: cgpa !== undefined ? parseFloat(cgpa) : students[index].cgpa,
    updatedAt: new Date().toISOString()
  };

  saveStudentsData(students);

  res.status(200).json({
    success: true,
    message: 'Student record updated successfully.',
    data: students[index]
  });
};

/**
 * @desc Delete Student Record
 * @route DELETE /api/students/:id
 */
const deleteStudent = (req, res) => {
  const students = getStudentsData();
  const param = req.params.id;
  const student = students.find(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
param.toLowerCase());

  if (!student) {
    return res.status(404).json({
      success: false,
      message: `Student record "${param}" not found for deletion.`
    });
  }

  const filtered = students.filter(s => s.id !== student.id);
  saveStudentsData(filtered);

  res.status(200).json({
    success: true,
    message: `Student record for "${student.name}" (Roll: ${student.rollNo}) deleted successfully.`,
    data: student
  });
};

/**
 * @desc Get Department-wise summary & statistics
 * @route GET /api/students/stats/overview
 */
const getStudentStats = (req, res) => {
  const students = getStudentsData();
  const total = students.length;
  const deptCounts = {};
  let totalCgpa = 0;

  students.forEach(s => {
    deptCounts[s.department] = (deptCounts[s.department] || 0) + 1;
```

```
    totalCgpa += s.cgpa || 0;
  });

  const avgCgpa = total > 0 ? (totalCgpa / total).toFixed(2) : 0;

  res.status(200).json({
    success: true,
    statistics: {
      totalStudents: total,
      averageCgpa: parseFloat(avgCgpa),
      departmentDistribution: deptCounts
    }
  });
};

module.exports = {
  getAllStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent,
  getStudentStats
};
```

- **middleware/validator.js :**

```
/**
 * Validation Middleware for Student Record Creation and Update
 */
const validateStudentPayload = (req, res, next) => {
  const { rollNo, name, email, department, semester, cgpa } = req.body;
  const errors = [];

  // Required fields for POST
  if (req.method === 'POST') {
    if (!rollNo || !rollNo.toString().trim()) errors.push('Field "rollNo" is required.');
    if (!name || !name.trim()) errors.push('Field "name" is required.');
    if (!email || !email.trim()) errors.push('Field "email" is required.');
    if (!department || !department.trim()) errors.push('Field "department" is required.');
    if (semester === undefined) errors.push('Field "semester" is required.');
  }

  // Email format check if provided
  if (email) {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email.trim())) {
      errors.push('Field "email" must be a valid email format.');
    }
  }

  // Semester range check if provided
  if (semester !== undefined) {
    const semNum = parseInt(semester);
    if (isNaN(semNum) || semNum < 1 || semNum > 8) {
      errors.push('Field "semester" must be an integer between 1 and 8.');
    }
  }

  // CGPA range check if provided
  if (cgpa !== undefined) {
    const cgpaNum = parseFloat(cgpa);
    if (isNaN(cgpaNum) || cgpaNum < 0 || cgpaNum > 10) {
      errors.push('Field "cgpa" must be a float between 0.00 and 10.00.');
    }
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
```

```
      message: 'Validation failed on student record payload.',
      errors
    });
  }

  next();
};

module.exports = { validateStudentPayload };
```

• **middleware/logger.js :**

```
/**
 * Request Logger Middleware
 */
const requestLogger = (req, res, next) => {
  const start = Date.now();
  const timestamp = new Date().toISOString();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`[${timestamp}] ${req.method} ${req.originalUrl} | Status: ${res.statusCode}
(${duration}ms)`);
  });
  next();
};

module.exports = requestLogger;
```

• **data/students.json :**

```
[
  {
    "id": 1,
    "rollNo": "26084231038",
    "name": "Prince Prajapati",
    "email": "prince@college.edu",
    "phone": "9876543210",
    "department": "Computer Engineering",
    "semester": 6,
    "cgpa": 8.95,
    "enrolledDate": "2022-08-15"
  },
  {
    "id": 2,
    "rollNo": "21IT042",
    "name": "Aarav Patel",
    "email": "aarav.patel@college.edu",
    "phone": "9823456781",
    "department": "Information Technology",
    "semester": 6,
    "cgpa": 9.12,
    "enrolledDate": "2022-08-15"
  },
  {
    "id": 3,
    "rollNo": "22AI015",
    "name": "Sneha Roy",
    "email": "sneha.roy@college.edu",
    "phone": "9734567890",
    "department": "AI & Data Science",
    "semester": 4,
    "cgpa": 8.70,
    "enrolledDate": "2023-08-10"
  },
  {
    "id": 4,
    "rollNo": "21EC028",
    "name": "Rohan Deshmukh",
```

```
    "email": "rohan.d@college.edu",
    "phone": "9845678901",
    "department": "Electronics & Comm",
    "semester": 6,
    "cgpa": 7.85,
    "enrolledDate": "2022-08-15"
  }
]
```

• **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 06 - Student Records RESTful API</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
background: #f8fafc; color: #0f172a; }

    .header {
      background: linear-gradient(135deg, #075985 0%, #0284c7 50%, #0369a1 100%);
      color: white;
      padding: 2.2rem 1.5rem;
      text-align: center;
    }
    .header-badge {
      display: inline-block;
      background: rgba(255, 255, 255, 0.2);
      padding: 0.3rem 0.9rem;
      border-radius: 9999px;
      font-size: 0.8rem;
      font-weight: 700;
      letter-spacing: 0.05em;
      text-transform: uppercase;
      margin-bottom: 0.6rem;
    }
    .header-title { font-size: 2rem; font-weight: 800; margin-bottom: 0.4rem; }
    .header-subtitle { font-size: 0.95rem; color: #bae6fd; }

    .container { max-width: 1100px; margin: 0 auto; padding: 1.75rem 1.25rem; }

    .banner {
      background: #eff6ff;
      border: 1px solid #bfdbfe;
      border-left: 4px solid #0284c7;
      border-radius: 8px;
      padding: 1rem 1.25rem;
      margin-bottom: 1.5rem;
      font-size: 0.88rem;
      color: #0369a1;
    }
    .banner h4 { margin-bottom: 0.25rem; font-weight: 700; display: flex; align-items: center; gap:
0.4rem; }

    .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; }
    .card { background: white; border: 1px solid #e2e8f0; border-radius: 12px; padding: 1.5rem; box-
shadow: 0 1px 3px rgba(0,0,0,0.05); }
    .card-title { font-size: 1.15rem; font-weight: 700; margin-bottom: 1.2rem; display: flex; align-
items: center; gap: 0.5rem; color: #1e293b; }

    .ep-item { display: flex; justify-content: space-between; align-items: center; padding: 0.65rem
0.75rem; border: 1px solid #e2e8f0; border-radius: 8px; margin-bottom: 0.7rem; background: #f8fafc; }
    .ep-left { display: flex; align-items: center; gap: 0.6rem; font-family: monospace; font-size:
0.88rem; }
    .method-get { background: #dbeafe; color: #1d4ed8; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-post { background: #dcfce7; color: #15803d; font-weight: 700; padding: 0.2rem 0.5rem; border-
```

```
radius: 4px; font-size: 0.75rem; }
    .method-del { background: #fee2e2; color: #b91c1c; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }

    .btn-test { background: #0284c7; color: white; border: none; padding: 0.35rem 0.8rem; border-radius:
6px; font-size: 0.8rem; font-weight: 600; cursor: pointer; }
    .btn-test:hover { background: #0369a1; }

    .console-box { background: #0b1329; color: #f8fafc; border-radius: 8px; padding: 1.25rem; font-
family: 'Consolas', 'Courier New', monospace; font-size: 0.85rem; line-height: 1.5; min-height: 380px;
overflow-y: auto; }
    .console-header { color: #38bdf8; font-weight: 600; margin-bottom: 0.75rem; border-bottom: 1px solid
#1e293b; padding-bottom: 0.5rem; display: flex; align-items: center; gap: 0.4rem; }
    .tip-box { margin-top: 1rem; border-top: 1px solid #1e293b; padding-top: 0.75rem; color: #94a3b8;
font-size: 0.78rem; display: flex; align-items: center; gap: 0.4rem; }
  </style>
</head>
<body>
  <div class="header">
    <div class="header-badge">Practical No. 06</div>
    <h1 class="header-title">Student Records RESTful API with Postman Testing</h1>
    <p class="header-subtitle">REST API Endpoints, Status Codes &amp;amp; Postman Test Collection
Integration</p>
  </div>

  <div class="container">
    <div class="banner">
      <h4>📮 Postman Test Collection Available</h4>
      <p>Import <code>Student_Records_API.postman_collection.json</code> in your Postman desktop client
to execute the entire 11-step REST API test suite!</p>
    </div>

    <div class="grid">
      <div class="card">
        <h3 class="card-title">📡 RESTful Endpoints</h3>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/students</div>
          <button class="btn-test" onclick="testGetAll()">Test GET All</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/students/26084231038</div>
          <button class="btn-test" onclick="testGetByRoll()">Test by Roll No</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/students/stats/overview</div>
          <button class="btn-test" onclick="testStats()">Test Stats API</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-post">POST</span> /api/students</div>
          <button class="btn-test" onclick="testPost()">Test POST Student</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-del">DELETE</span> /api/students/21CS099</div>
          <button class="btn-test" style="background: #dc2626;" onclick="testDelete()">Test
DELETE</button>
        </div>
      </div>

      <div class="card">
        <h3 class="card-title">🖥️ Response Output</h3>
        <div class="console-box">
          <div class="console-header">
            <span>●</span> Live Server at localhost:5000 is connected.<br>
            <span>Express REST API Response:</span>
          </div>
```

```
          <pre id="code-content" style="white-space: pre-wrap;">{
  "id": 1,
  "rollNo": "26084231038",
  "name": "Prince Prajapati",
  "email": "prince@college.edu",
  "department": "Computer Engineering",
  "semester": 6,
  "cgpa": 8.95
}</pre>
          <div class="tip-box">
            💡 Tip: Run 'node server.js' in terminal to start the live Node.js REST server!
          </div>
        </div>
      </div>
    </div>
  </div>

  <script>
    async function testGetAll() {
      try {
        const res = await fetch('/api/students');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testGetByRoll() {
      try {
        const res = await fetch('/api/students/26084231038');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testStats() {
      try {
        const res = await fetch('/api/students/stats/overview');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testPost() {
      try {
        const res = await fetch('/api/students', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ rollNo: '22CS099', name: 'Kavya Patel', email: 'kavya@college.edu',
department: 'Computer Engineering', semester: 4, cgpa: 9.45 })
        });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testDelete() {
      try {
        const res = await fetch('/api/students/4', { method: 'DELETE' });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
  </script>
</body>
</html>
```

4. Output :

RESTful API for student records management with payload validation middleware, query filtering, sorting, status codes, and department summary statistics.

PRACTICAL NO. 06

Student Records RESTful API with Postman Testing

REST API Endpoints, Status Codes & Postman Test Collection Integration

Postman Test Collection Available

Import `Student_Records_API.postman_collection.json` in your Postman desktop client to execute the entire 11-step REST API test suite!

RESTful Endpoints

GET	/api/students	Test GET All
GET	/api/students/26084231038	Test by Roll No
GET	/api/students/stats/overview	Test Stats API
POST	/api/students	Test POST Student
DELETE	/api/students/21CS099	Test DELETE

Response Output

```
● Live Server at localhost:5000 is connected.   Express REST API Response:

{
  "id": 1,
  "rollNo": "26084231038",
  "name": "Prince Prajapati",
  "email": "prince@college.edu",
  "department": "Computer Engineering",
  "semester": 6,
  "cgpa": 8.95
}

💡 Tip: Run 'node server.js' in terminal to start the live Node.js REST server!
```

Practical – 07 : Develop an Employee Management API integrating Express.js with MongoDB Atlas.

1. Commands :

```
mkdir MERN_07 && cd MERN_07
npm init -y
npm install express mongoose dotenv cors morgan
node seed.js
node server.js
```

2. Project structure :

```
MERN_07/
|-- .env
|-- package.json
|-- server.js
|-- seed.js
|-- config/db.js
|-- models/Employee.js
|-- routes/employeeRoutes.js
|-- controllers/employeeController.js
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5000
MONGO_URI=mongodb+srv://prince:prince18@cluster0.76xckkx.mongodb.net/employee_db?
retryWrites=true&w=majority
```

- **package.json :**

```
{
  "name": "mern-07",
  "version": "1.0.0",
  "description": "Employee Management RESTful API using Express.js and MongoDB Atlas with Schema
Validation and Aggregations",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "seed": "node seed.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.6.1",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');
```

```
const employeeRoutes = require('./routes/employeeRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Connect to MongoDB Atlas
connectDB();

// Middlewares
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Request logger
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`[MongoDB API] ${req.method} ${req.originalUrl} - ${res.statusCode} (${Date.now() -
start}ms)`);
  });
  next();
});

// Welcome / Health Route
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 07: Employee Management API with Express & MongoDB Atlas',
    endpoints: {
      getAllEmployees: 'GET /api/employees',
      getEmployeeByIdOrEmpId: 'GET /api/employees/:id',
      createEmployee: 'POST /api/employees',
      updateEmployee: 'PUT /api/employees/:id',
      deleteEmployee: 'DELETE /api/employees/:id',
      departmentAnalytics: 'GET /api/employees/stats/summary'
    }
  });
});

// Mount Routes
app.use('/api/employees', employeeRoutes);

// Fallback 404
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Route ${req.originalUrl} not found on this Employee Management API.`
  });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('Unhandled Error:', err.message);
  res.status(500).json({
    success: false,
    message: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`⚛️ Practical 07: Employee Management API`);
  console.log(`⚛️ Server running at: http://localhost:${PORT}`);
  console.log(`⚛️ Endpoints at: http://localhost:${PORT}/api/employees`);
  console.log(`⚛️ MongoDB URI: ${process.env.MONGO_URI ? 'Loaded from .env' : 'Not Provided'}`);
  console.log(`================================================`);
});
```

- **seed.js :**

```
require('dotenv').config();
const dns = require('dns');
try { dns.setDefaultResultOrder('ipv4first'); } catch (e) {}
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch (e) {}
const mongoose = require('mongoose');
const Employee = require('./models/Employee');

const sampleEmployees = [
  {
    empId: 'EMP101',
    firstName: 'Prince',
    lastName: 'Prajapati',
    email: 'prince.prajapati@company.com',
    phone: '9876543210',
    department: 'Engineering',
    designation: 'Lead Full Stack Engineer',
    salary: 85000,
    status: 'Active'
  },
  {
    empId: 'EMP102',
    firstName: 'Ananya',
    lastName: 'Sharma',
    email: 'ananya.sharma@company.com',
    phone: '9812345678',
    department: 'Product',
    designation: 'Senior Product Manager',
    salary: 95000,
    status: 'Active'
  },
  {
    empId: 'EMP103',
    firstName: 'Vikram',
    lastName: 'Malhotra',
    email: 'vikram.m@company.com',
    phone: '9765432109',
    department: 'Engineering',
    designation: 'Backend Architect',
    salary: 110000,
    status: 'Active'
  },
  {
    empId: 'EMP104',
    firstName: 'Pooja',
    lastName: 'Desai',
    email: 'pooja.desai@company.com',
    phone: '9845678901',
    department: 'Human Resources',
    designation: 'HR Specialist',
    salary: 55000,
    status: 'Active'
  },
  {
    empId: 'EMP105',
    firstName: 'Rohan',
    lastName: 'Verma',
    email: 'rohan.v@company.com',
    phone: '9834567812',
    department: 'Finance',
    designation: 'Financial Analyst',
    salary: 65000,
    status: 'On Leave'
  }
];

const seedDatabase = async () => {
  try {
    console.log('Connecting to MongoDB database...');
    await mongoose.connect(process.env.MONGO_URI);
    console.log('✅ Connected to MongoDB.');
```

```
    console.log('Clearing existing employee documents...');
    await Employee.deleteMany({});

    console.log('Inserting sample employee records...');
    await Employee.insertMany(sampleEmployees);

    console.log(`⚛️ Successfully seeded ${sampleEmployees.length} employees into MongoDB!`);
    process.exit(0);
  } catch (error) {
    console.error('❌ Seeder Failed:', error.message);
    process.exit(1);
  }
};

seedDatabase();
```

• **config/db.js :**

```
const dns = require('dns');
try { dns.setDefaultResultOrder('ipv4first'); } catch (e) {}
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch (e) {}
const mongoose = require('mongoose');

/**
 * Connect to MongoDB / MongoDB Atlas using Mongoose
 */
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000 // 5 seconds timeout
    });
    console.log(`✅ MongoDB Connected Successfully: ${conn.connection.host}`);
    console.log(`⚛️ Database Name: ${conn.connection.name}`);
  } catch (error) {
    console.error(`❌ MongoDB Connection Error: ${error.message}`);
    console.log(`⚛️ Tip: Make sure your MongoDB Atlas connection string in .env is correct and your IP
address is whitelisted in Atlas Network Access.`);
  }
};

module.exports = connectDB;
```

• **models/Employee.js :**

```
const mongoose = require('mongoose');

const employeeSchema = new mongoose.Schema(
  {
    empId: {
      type: String,
      required: [true, 'Employee ID is required.'],
      unique: true,
      trim: true,
      uppercase: true
    },
    firstName: {
      type: String,
      required: [true, 'First Name is required.'],
      trim: true,
      minlength: [2, 'First Name must be at least 2 characters.']
    },
    lastName: {
      type: String,
      required: [true, 'Last Name is required.'],
      trim: true
    },
    email: {
```

```
      type: String,
      required: [true, 'Email address is required.'],
      unique: true,
      trim: true,
      lowercase: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Please provide a valid email address.']
    },
    phone: {
      type: String,
      required: [true, 'Phone number is required.'],
      trim: true
    },
    department: {
      type: String,
      required: [true, 'Department is required.'],
      enum: {
        values: ['Engineering', 'Human Resources', 'Finance', 'Marketing', 'Product', 'Operations'],
        message: '{VALUE} is not a supported department.'
      }
    },
    designation: {
      type: String,
      required: [true, 'Designation / Job Title is required.'],
      trim: true
    },
    salary: {
      type: Number,
      required: [true, 'Salary is required.'],
      min: [0, 'Salary cannot be negative.']
    },
    joiningDate: {
      type: Date,
      default: Date.now
    },
    status: {
      type: String,
      enum: ['Active', 'Inactive', 'On Leave'],
      default: 'Active'
    }
  },
  {
    timestamps: true, // Automatically manages createdAt and updatedAt
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

// Virtual property for full name
employeeSchema.virtual('fullName').get(function () {
  return `${this.firstName} ${this.lastName}`;
});

const Employee = mongoose.model('Employee', employeeSchema);

module.exports = Employee;
```

- **routes/employeeRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllEmployees,
  getEmployeeById,
  createEmployee,
  updateEmployee,
  deleteEmployee,
  getEmployeeSummaryStats
} = require('../controllers/employeeController');
```

```
// Aggregate statistics route (Must be before /:id)
router.get('/stats/summary', getEmployeeSummaryStats);

// Main Employee routes
router.route('/')
  .get(getAllEmployees)
  .post(createEmployee);

router.route('/:id')
  .get(getEmployeeById)
  .put(updateEmployee)
  .delete(deleteEmployee);

module.exports = router;
```

- **controllers/employeeController.js :**

```
const Employee = require('../models/Employee');

/**
 * @desc Get All Employees with filtering, search, sorting & pagination
 * @route GET /api/employees
 */
const getAllEmployees = async (req, res) => {
  try {
    const { department, status, search, minSalary, maxSalary, sortBy, page = 1, limit = 10 } = req.query;
    const query = {};

    // Filter by department
    if (department) {
      query.department = department;
    }

    // Filter by status
    if (status) {
      query.status = status;
    }

    // Filter by salary range
    if (minSalary || maxSalary) {
      query.salary = {};
      if (minSalary) query.salary.$gte = Number(minSalary);
      if (maxSalary) query.salary.$lte = Number(maxSalary);
    }

    // Search keyword across firstName, lastName, email, designation
    if (search) {
      query.$or = [
        { firstName: { $regex: search, $options: 'i' } },
        { lastName: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
        { designation: { $regex: search, $options: 'i' } },
        { empId: { $regex: search, $options: 'i' } }
      ];
    }

    // Sorting
    let sortOption = { createdAt: -1 };
    if (sortBy === 'salary_desc') sortOption = { salary: -1 };
    else if (sortBy === 'salary_asc') sortOption = { salary: 1 };
    else if (sortBy === 'name') sortOption = { firstName: 1 };

    // Pagination
    const skip = (Number(page) - 1) * Number(limit);
    const totalRecords = await Employee.countDocuments(query);
    const employees = await Employee.find(query)
      .sort(sortOption)
      .skip(skip)
      .limit(Number(limit));
```

```
    res.status(200).json({
      success: true,
      count: employees.length,
      totalRecords,
      totalPages: Math.ceil(totalRecords / Number(limit)),
      currentPage: Number(page),
      data: employees
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to retrieve employees.',
      error: error.message
    });
  }
};

/**
 * @desc Get Single Employee by MongoDB _id OR custom empId
 * @route GET /api/employees/:id
 */
const getEmployeeById = async (req, res) => {
  try {
    const { id } = req.params;

    // Search by ObjectId or custom empId (e.g., EMP101)
    let employee;
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      employee = await Employee.findById(id);
    } else {
      employee = await Employee.findOne({ empId: id.toUpperCase() });
    }

    if (!employee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" was not found.`
      });
    }

    res.status(200).json({
      success: true,
      data: employee
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Error fetching employee details.',
      error: error.message
    });
  }
};

/**
 * @desc Create New Employee Record in MongoDB
 * @route POST /api/employees
 */
const createEmployee = async (req, res) => {
  try {
    const employee = await Employee.create(req.body);
    res.status(201).json({
      success: true,
      message: 'Employee registered successfully in MongoDB Atlas.',
      data: employee
    });
  } catch (error) {
    // Handle Duplicate Key Error
    if (error.code === 11000) {
      const field = Object.keys(error.keyPattern)[0];
```

```
      return res.status(409).json({
        success: false,
        message: `Duplicate entry error: An employee with this ${field} already exists.`
      });
    }

    // Handle Mongoose Validation Errors
    if (error.name === 'ValidationError') {
      const messages = Object.values(error.errors).map(val => val.message);
      return res.status(400).json({
        success: false,
        message: 'Validation failed on employee fields.',
        errors: messages
      });
    }

    res.status(500).json({
      success: false,
      message: 'Failed to create employee.',
      error: error.message
    });
  }
};

/**
 * @desc Update Employee Details in MongoDB
 * @route PUT /api/employees/:id
 */
const updateEmployee = async (req, res) => {
  try {
    const { id } = req.params;
    let query = {};
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      query = { _id: id };
    } else {
      query = { empId: id.toUpperCase() };
    }

    const updatedEmployee = await Employee.findOneAndUpdate(
      query,
      req.body,
      { new: true, runValidators: true }
    );

    if (!updatedEmployee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" not found for update.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Employee record updated successfully.',
      data: updatedEmployee
    });
  } catch (error) {
    if (error.name === 'ValidationError') {
      const messages = Object.values(error.errors).map(val => val.message);
      return res.status(400).json({
        success: false,
        message: 'Validation failed on update.',
        errors: messages
      });
    }

    res.status(500).json({
      success: false,
      message: 'Failed to update employee.',
      error: error.message
```

```
    });
  }
};

/**
 * @desc Delete Employee from MongoDB
 * @route DELETE /api/employees/:id
 */
const deleteEmployee = async (req, res) => {
  try {
    const { id } = req.params;
    let query = {};
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      query = { _id: id };
    } else {
      query = { empId: id.toUpperCase() };
    }

    const deletedEmployee = await Employee.findOneAndDelete(query);

    if (!deletedEmployee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" not found for deletion.`
      });
    }

    res.status(200).json({
      success: true,
      message: `Employee "${deletedEmployee.firstName} ${deletedEmployee.lastName}"
(${deletedEmployee.empId}) deleted successfully.`,
      data: deletedEmployee
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to delete employee.',
      error: error.message
    });
  }
};

/**
 * @desc Department Aggregate Metrics & Analytics (MongoDB Aggregation Pipeline)
 * @route GET /api/employees/stats/summary
 */
const getEmployeeSummaryStats = async (req, res) => {
  try {
    const stats = await Employee.aggregate([
      {
        $group: {
          _id: '$department',
          totalEmployees: { $sum: 1 },
          avgSalary: { $avg: '$salary' },
          minSalary: { $min: '$salary' },
          maxSalary: { $max: '$salary' }
        }
      },
      {
        $project: {
          department: '$_id',
          totalEmployees: 1,
          avgSalary: { $round: ['$avgSalary', 2] },
          minSalary: 1,
          maxSalary: 1,
          _id: 0
        }
      },
      { $sort: { totalEmployees: -1 } }
    ]);
```

```
    const totalHeadcount = await Employee.countDocuments();

    res.status(200).json({
      success: true,
      totalHeadcount,
      departmentStats: stats
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to compute employee summary statistics.',
      error: error.message
    });
  }
};

module.exports = {
  getAllEmployees,
  getEmployeeById,
  createEmployee,
  updateEmployee,
  deleteEmployee,
  getEmployeeSummaryStats
};
```

- **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 07 - Employee Management API with MongoDB Atlas</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
background: #f8fafc; color: #0f172a; }

    .header {
      background: linear-gradient(135deg, #064e3b 0%, #065f46 50%, #059669 100%);
      color: white;
      padding: 2.2rem 1.5rem;
      text-align: center;
    }
    .header-badge {
      display: inline-block;
      background: rgba(255, 255, 255, 0.2);
      padding: 0.3rem 0.9rem;
      border-radius: 9999px;
      font-size: 0.8rem;
      font-weight: 700;
      letter-spacing: 0.05em;
      text-transform: uppercase;
      margin-bottom: 0.6rem;
    }
    .header-title { font-size: 2rem; font-weight: 800; margin-bottom: 0.4rem; }
    .header-subtitle { font-size: 0.95rem; color: #a7f3d0; }

    .container { max-width: 1100px; margin: 0 auto; padding: 1.75rem 1.25rem; }

    .banner {
      background: #ecfdf5;
      border: 1px solid #a7f3d0;
      border-left: 4px solid #059669;
      border-radius: 8px;
      padding: 1rem 1.25rem;
      margin-bottom: 1.5rem;
      font-size: 0.88rem;
      color: #065f46;
```

```
    }
    .banner h4 { margin-bottom: 0.25rem; font-weight: 700; display: flex; align-items: center; gap:
0.4rem; }

    .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 1.5rem; }
    .card { background: white; border: 1px solid #e2e8f0; border-radius: 12px; padding: 1.5rem; box-
shadow: 0 1px 3px rgba(0,0,0,0.05); }
    .card-title { font-size: 1.15rem; font-weight: 700; margin-bottom: 1.2rem; display: flex; align-
items: center; gap: 0.5rem; color: #1e293b; }

    .ep-item { display: flex; justify-content: space-between; align-items: center; padding: 0.65rem
0.75rem; border: 1px solid #e2e8f0; border-radius: 8px; margin-bottom: 0.7rem; background: #f8fafc; }
    .ep-left { display: flex; align-items: center; gap: 0.6rem; font-family: monospace; font-size:
0.88rem; }
    .method-get { background: #dbeafe; color: #1d4ed8; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-post { background: #dcfce7; color: #15803d; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }
    .method-del { background: #fee2e2; color: #b91c1c; font-weight: 700; padding: 0.2rem 0.5rem; border-
radius: 4px; font-size: 0.75rem; }

    .btn-test { background: #059669; color: white; border: none; padding: 0.35rem 0.8rem; border-radius:
6px; font-size: 0.8rem; font-weight: 600; cursor: pointer; }
    .btn-test:hover { background: #047857; }

    .console-box { background: #0b1329; color: #f8fafc; border-radius: 8px; padding: 1.25rem; font-
family: 'Consolas', 'Courier New', monospace; font-size: 0.85rem; line-height: 1.5; min-height: 380px;
overflow-y: auto; }
    .console-header { color: #34d399; font-weight: 600; margin-bottom: 0.75rem; border-bottom: 1px solid
#1e293b; padding-bottom: 0.5rem; display: flex; align-items: center; gap: 0.4rem; }
    .tip-box { margin-top: 1rem; border-top: 1px solid #1e293b; padding-top: 0.75rem; color: #94a3b8;
font-size: 0.78rem; display: flex; align-items: center; gap: 0.4rem; }
  </style>
</head>
<body>
  <div class="header">
    <div class="header-badge">Practical No. 07</div>
    <h1 class="header-title">Employee Management API with MongoDB Atlas</h1>
    <p class="header-subtitle">Mongoose ODM, Schema Validation, Aggregation Analytics &amp;amp; Cloud
Database Integration</p>
  </div>

  <div class="container">
    <div class="banner">
      <h4>🍃 MongoDB Atlas Integration Details</h4>
      <p>Configure your MongoDB Atlas cluster connection string in .env. Run <code>npm run seed</code> to
populate sample records, and <code>node server.js</code> to start the live server.</p>
    </div>

    <div class="grid">
      <div class="card">
        <h3 class="card-title">📡 Employee API Endpoints</h3>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/employees</div>
          <button class="btn-test" onclick="testGetAll()">Test GET All</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/employees/EMP101</div>
          <button class="btn-test" onclick="testGetByEmpId()">Test by empId</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-get">GET</span> /api/employees/stats/summary</div>
          <button class="btn-test" onclick="testAggregation()">Test Aggregation</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-post">POST</span> /api/employees</div>
```

```
          <button class="btn-test" onclick="testPost()">Test POST</button>
        </div>

        <div class="ep-item">
          <div class="ep-left"><span class="method-del">DELETE</span> /api/employees/EMP106</div>
          <button class="btn-test" style="background: #dc2626;" onclick="testDelete()">Test
DELETE</button>
        </div>
      </div>

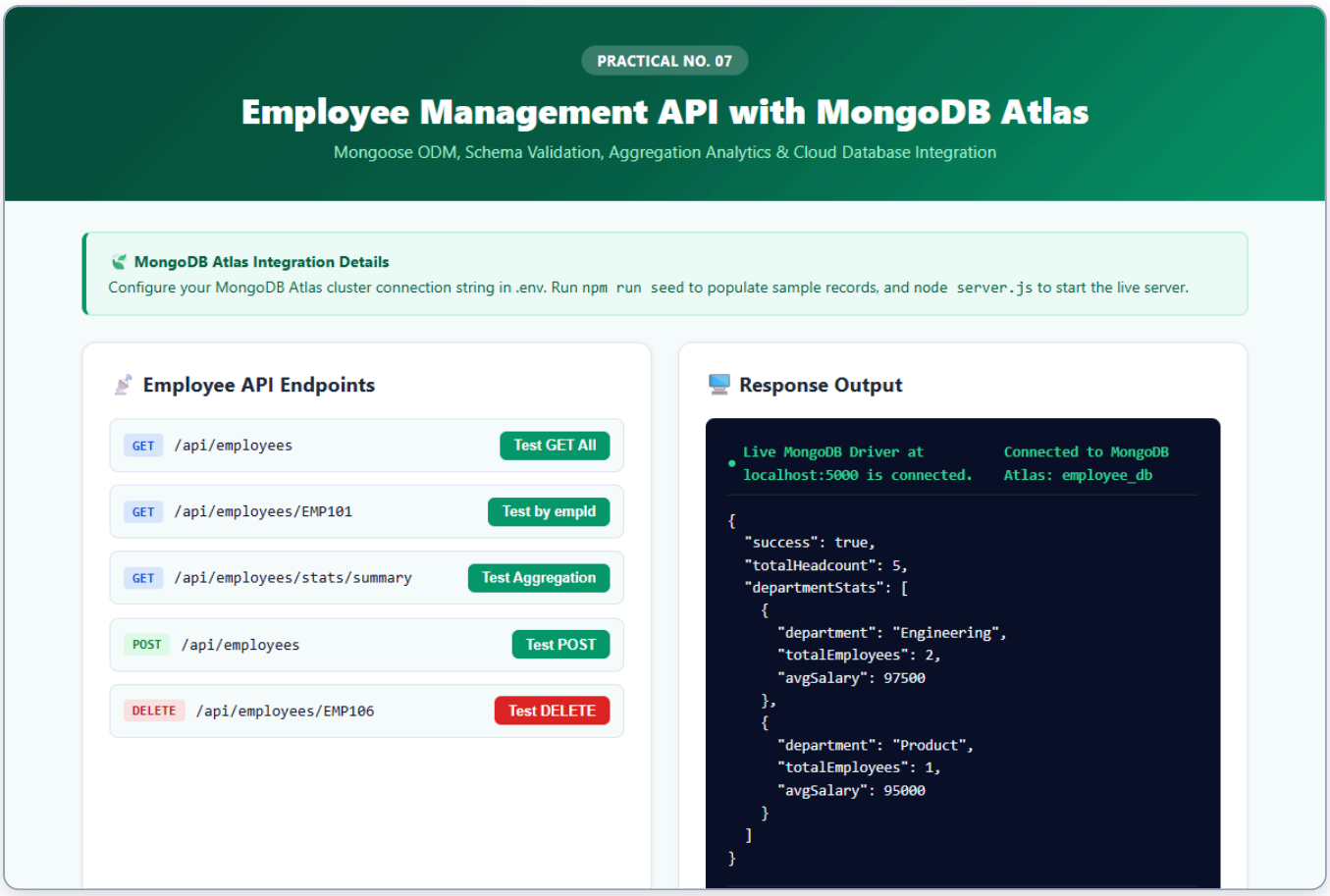
      <div class="card">
        <h3 class="card-title">🖥️ Response Output</h3>
        <div class="console-box">
          <div class="console-header">
            <span>●</span> Live MongoDB Driver at localhost:5000 is connected.<br>
            <span>Connected to MongoDB Atlas: employee_db</span>
          </div>
          <pre id="code-content" style="white-space: pre-wrap;">{
  "success": true,
  "totalHeadcount": 5,
  "departmentStats": [
    {
      "department": "Engineering",
      "totalEmployees": 2,
      "avgSalary": 97500
    },
    {
      "department": "Product",
      "totalEmployees": 1,
      "avgSalary": 95000
    }
  ]
}</pre>
          <div class="tip-box">
            💡 Tip: Connected live to cluster ac-hgdghnn-shard-00-02.76xckkx.mongodb.net!
          </div>
        </div>
      </div>
    </div>
  </div>

  <script>
    async function testGetAll() {
      try {
        const res = await fetch('/api/employees');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testGetByEmpId() {
      try {
        const res = await fetch('/api/employees/EMP101');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testAggregation() {
      try {
        const res = await fetch('/api/employees/stats/summary');
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testPost() {
      try {
        const res = await fetch('/api/employees', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ empId: 'EMP106', firstName: 'Karan', lastName: 'Mehta', email:
'karan@company.com', phone: '9876543210', department: 'Engineering', designation: 'QA Engineer', salary:
```

```
60000 })
        });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
    async function testDelete() {
      try {
        const res = await fetch('/api/employees/EMP106', { method: 'DELETE' });
        const data = await res.json();
        document.getElementById('code-content').textContent = JSON.stringify(data, null, 2);
      } catch (e) {}
    }
  </script>
</body>
</html>
```

4. Output :

Cloud-connected Express API utilizing Mongoose ODM, MongoDB Atlas connection, strict schema validation, and aggregation pipeline for departmental analytics.



Practical – 08 : Create a backend server using Express.js implementing routing, middleware and request handling.

1. Commands :

```
mkdir MERN_08 && cd MERN_08
npm init -y
npm install express cors dotenv morgan
node server.js
```

2. Project structure :

```
MERN_08/
|-- .env
|-- package.json
|-- server.js
|-- routes/userRoutes.js
|-- routes/inspectorRoutes.js
|-- controllers/userController.js
|-- middleware/loggerMiddleware.js
|-- middleware/authMiddleware.js
|-- middleware/validateMiddleware.js
|-- middleware/errorHandler.js
|-- data/users.json
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5008
NODE_ENV=development
API_SECRET_KEY=prince_master_key_2026
```

- **package.json :**

```
{
  "name": "mern-08",
  "version": "1.0.0",
  "description": "Practical 08: Express.js Backend Server implementing Modular Routing, Custom & Built-in
Middleware, and Advanced Request Handling",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
```

```
const loggerMiddleware = require('./middleware/loggerMiddleware');
const errorHandler = require('./middleware/errorHandler');
const userRoutes = require('./routes/userRoutes');
const inspectorRoutes = require('./routes/inspectorRoutes');

const app = express();
const PORT = process.env.PORT || 5008;

// 1. Built-in & Third-party Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// 2. Application-level Custom Middleware
app.use(loggerMiddleware);

// 3. Welcome / Health Route
app.get('/api/health', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 08: Express.js Routing, Middleware & Request Handling API',
    middlewareStack: [
      'cors',
      'express.json()',
      'express.urlencoded()',
      'express.static()',
      'custom loggerMiddleware',
      'router-level middleware',
      'authMiddleware (x-api-key)',
      'validateMiddleware',
      'central errorHandler'
    ],
    timestamp: new Date().toISOString()
  });
});

// 4. Mount Modular Routers
app.use('/api/users', userRoutes);
app.use('/api', inspectorRoutes);

// 5. Fallback 404 Route Handler
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Cannot ${req.method} ${req.originalUrl}. Route not registered.`
  });
});

// 6. Central Error Handling Middleware (must be after routes)
app.use(errorHandler);

// Start Server
app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`⚛️ Practical 08: Routing, Middleware & Request Handling`);
  console.log(`⚛️ Server running at: http://localhost:${PORT}`);
  console.log(`⚛️ API Endpoints: http://localhost:${PORT}/api/users`);
  console.log(`⚛️ Request Inspector: http://localhost:${PORT}/api/inspect`);
  console.log(`================================================`);
});

module.exports = app;
```

- **routes/userRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
```

```
  getAllUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser
} = require('../controllers/userController');
const { authenticateApiKey, authorizeRoles } = require('../middleware/authMiddleware');
const { validateIdParam, validateUserBody } = require('../middleware/validateMiddleware');

// Router-level middleware tracking user routes
router.use((req, res, next) => {
  req.routerLevelTimestamp = Date.now();
  next();
});

// GET /api/users (query filtering)
router.get('/', getAllUsers);

// GET /api/users/:id (param handling)
router.get('/:id', validateIdParam, getUserById);

// POST /api/users (protected + validated body)
router.post('/', authenticateApiKey, validateUserBody, createUser);

// PUT /api/users/:id (protected + validated id)
router.put('/:id', authenticateApiKey, validateIdParam, updateUser);

// DELETE /api/users/:id (protected + role restricted to 'admin')
router.delete('/:id', authenticateApiKey, authorizeRoles('admin'), validateIdParam, deleteUser);

module.exports = router;
```

- **routes/inspectorRoutes.js :**

```
const express = require('express');
const router = express.Router();

/**
 * @desc Echo and inspect full HTTP request structure
 * Demonstrates: req.headers, req.query, req.params, req.body, req.ip, req.method
 */
router.all('/inspect', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'HTTP Request Inspection Matrix',
    protocol: req.protocol,
    httpVersion: req.httpVersion,
    method: req.method,
    originalUrl: req.originalUrl,
    path: req.path,
    ip: req.ip || req.connection.remoteAddress,
    queryParameters: req.query,
    headersReceived: {
      host: req.headers.host,
      userAgent: req.headers['user-agent'],
      contentType: req.headers['content-type'],
      apiKey: req.headers['x-api-key'] || 'Not Sent',
      customUserRole: req.headers['x-user-role'] || 'Not Sent'
    },
    bodyPayload: req.body,
    serverTimestamp: new Date().toISOString()
  });
});

/**
 * @desc Demonstrate intentional middleware error triggers
 */
router.get('/simulate-error', (req, res, next) => {
  const errorType = req.query.type || 'standard';
```

```
  if (errorType === 'bad-request') {
    const err = new Error('Simulated 400 Bad Request: Malformed query payload');
    err.statusCode = 400;
    return next(err);
  }
  if (errorType === 'unauthorized') {
    const err = new Error('Simulated 401 Unauthorized: Session token expired');
    err.statusCode = 401;
    return next(err);
  }
  const err = new Error('Simulated 500 Uncaught Server Exception');
  err.statusCode = 500;
  next(err);
});

module.exports = router;
```

• **controllers/userController.js :**

```
const fs = require('fs');
const path = require('path');

const dataFile = path.join(__dirname, '..', 'data', 'users.json');

const getUsersData = () => {
  try {
    const raw = fs.readFileSync(dataFile, 'utf8');
    return JSON.parse(raw);
  } catch (e) {
    return [];
  }
};

const saveUsersData = (data) => {
  fs.writeFileSync(dataFile, JSON.stringify(data, null, 2), 'utf8');
};

/**
 * @desc Filter, search, and paginate users from query params
 * Demonstrates req.query parsing
 */
const getAllUsers = (req, res) => {
  let users = getUsersData();
  const { department, role, search, status, limit, page } = req.query;

  if (department) {
    users = users.filter((u) => u.department.toLowerCase() === department.toLowerCase());
  }

  if (role) {
    users = users.filter((u) => u.role.toLowerCase() === role.toLowerCase());
  }

  if (status) {
    users = users.filter((u) => u.status.toLowerCase() === status.toLowerCase());
  }

  if (search) {
    const q = search.toLowerCase();
    users = users.filter((u) => u.name.toLowerCase().includes(q) || u.email.toLowerCase().includes(q));
  }

  const pageNum = parseInt(page, 10) || 1;
  const limitNum = parseInt(limit, 10) || 10;
  const startIndex = (pageNum - 1) * limitNum;
  const paginatedUsers = users.slice(startIndex, startIndex + limitNum);

  res.status(200).json({
    success: true,
```

```
    totalRecords: users.length,
    returnedCount: paginatedUsers.length,
    currentPage: pageNum,
    filtersApplied: { department, role, search, status },
    data: paginatedUsers
  });
};

/**
 * @desc Get user by ID
 * Demonstrates req.params parsing
 */
const getUserById = (req, res) => {
  const users = getUsersData();
  const user = users.find((u) => u.id === req.numericId);

  if (!user) {
    return res.status(404).json({
      success: false,
      message: `User with ID ${req.numericId} not found.`
    });
  }

  res.status(200).json({
    success: true,
    data: user
  });
};

/**
 * @desc Create user
 * Demonstrates req.body parsing + middleware validation
 */
const createUser = (req, res) => {
  const users = getUsersData();
  const { name, email, role, department, status = 'active' } = req.body;

  const existingEmail = users.find((u) => u.email.toLowerCase() === email.toLowerCase());
  if (existingEmail) {
    return res.status(409).json({
      success: false,
      message: `Email "${email}" is already registered.`
    });
  }

  const nextId = users.length > 0 ? Math.max(...users.map((u) => u.id)) + 1 : 1;
  const newUser = {
    id: nextId,
    username: email.split('@')[0],
    name: name.trim(),
    email: email.trim().toLowerCase(),
    role: role.toLowerCase(),
    department: department.trim(),
    status: status.toLowerCase(),
    createdAt: new Date().toISOString()
  };

  users.push(newUser);
  saveUsersData(users);

  res.status(201).json({
    success: true,
    message: 'User created successfully.',
    data: newUser
  });
};

/**
 * @desc Update user
 * Demonstrates PUT request with body & params
```

```
 */
const updateUser = (req, res) => {
  const users = getUsersData();
  const index = users.findIndex((u) => u.id === req.numericId);

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: `User with ID ${req.numericId} not found for update.`
    });
  }

  const { name, role, department, status } = req.body;
  users[index] = {
    ...users[index],
    name: name ? name.trim() : users[index].name,
    role: role ? role.toLowerCase() : users[index].role,
    department: department ? department.trim() : users[index].department,
    status: status ? status.toLowerCase() : users[index].status,
    updatedAt: new Date().toISOString()
  };

  saveUsersData(users);

  res.status(200).json({
    success: true,
    message: 'User updated successfully.',
    data: users[index]
  });
};

/**
 * @desc Delete user
 * Demonstrates DELETE request with role authorization
 */
const deleteUser = (req, res) => {
  const users = getUsersData();
  const user = users.find((u) => u.id === req.numericId);

  if (!user) {
    return res.status(404).json({
      success: false,
      message: `User with ID ${req.numericId} not found.`
    });
  }

  const filtered = users.filter((u) => u.id !== req.numericId);
  saveUsersData(filtered);

  res.status(200).json({
    success: true,
    message: `User "${user.name}" (ID: ${user.id}) has been removed.`,
    data: user
  });
};

module.exports = {
  getAllUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser
};
```

- **middleware/loggerMiddleware.js :**

```
/**
 * Application-level Logging Middleware
 * Attaches request timestamp and tracks execution time
```

```
 */
const loggerMiddleware = (req, res, next) => {
  req.requestTime = new Date().toISOString();
  const startTime = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - startTime;
    const logColor = res.statusCode >= 400 ? '\x1b[31m' : '\x1b[32m';
    console.log(
      `[${req.requestTime}] ${req.method} ${req.originalUrl} → ${logColor}${res.statusCode}\x1b[0m
(${duration}ms)`
    );
  });

  next();
};

module.exports = loggerMiddleware;
```

• **middleware/authMiddleware.js :**

```
/**
 * Authentication and Role Authorization Middleware
 * Inspects incoming request headers: x-api-key or Authorization Bearer token
 */
const authenticateApiKey = (req, res, next) => {
  const apiKey = req.headers['x-api-key'] || req.query.apiKey;
  const validKey = process.env.API_SECRET_KEY || 'prince_master_key_2026';

  if (!apiKey) {
    return res.status(401).json({
      success: false,
      message: 'Access Denied: Missing "x-api-key" in request headers or "?apiKey=" in query string.',
      hint: `Provide header x-api-key: ${validKey}`
    });
  }

  if (apiKey !== validKey) {
    return res.status(403).json({
      success: false,
      message: 'Access Forbidden: Invalid API Key provided.'
    });
  }

  // Simulate authenticated identity attached to request object
  req.user = {
    username: 'prince18',
    role: req.headers['x-user-role'] || 'admin',
    authenticatedAt: new Date().toISOString()
  };

  next();
};

const authorizeRoles = (...allowedRoles) => {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Forbidden: Current role "${req.user ? req.user.role : 'Guest'}" does not have
permission. Required: [${allowedRoles.join(', ')}]`
      });
    }
    next();
  };
};

module.exports = { authenticateApiKey, authorizeRoles };
```

• **middleware/validateMiddleware.js :**

```
/**
 * Validation Middleware for Request Parameters, Query and Body
 */
const validateIdParam = (req, res, next) => {
  const { id } = req.params;
  const numericId = parseInt(id, 10);

  if (isNaN(numericId) || numericId <= 0) {
    return res.status(400).json({
      success: false,
      message: `Invalid ID parameter: "${id}". Expected a positive integer.`
    });
  }

  req.numericId = numericId;
  next();
};

const validateUserBody = (req, res, next) => {
  const { name, email, role, department } = req.body;
  const errors = [];

  if (!name || name.trim().length < 3) {
    errors.push('Field "name" is required and must be at least 3 characters.');
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!email || !emailRegex.test(email)) {
    errors.push('Valid "email" address is required.');
  }

  const validRoles = ['admin', 'editor', 'viewer'];
  if (!role || !validRoles.includes(role.toLowerCase())) {
    errors.push(`Field "role" must be one of: [${validRoles.join(', ')}]`);
  }

  if (!department || department.trim().length < 2) {
    errors.push('Field "department" is required.');
  }

  if (errors.length > 0) {
    return res.status(422).json({
      success: false,
      message: 'Request Body Validation Failed',
      errors
    });
  }

  next();
};

module.exports = { validateIdParam, validateUserBody };
```

• **middleware/errorHandler.js :**

```
/**
 * Central Error-Handling Middleware (4 parameters)
 */
const errorHandler = (err, req, res, next) => {
  const statusCode = err.statusCode || (res.statusCode === 200 ? 500 : res.statusCode);

  console.error(`[Error Handler] ${err.name || 'Error'}: ${err.message}`);

  res.status(statusCode).json({
    success: false,
    error: {
      type: err.name || 'ApplicationError',
```

```
      message: err.message || 'An unexpected internal server error occurred.',
      path: req.originalUrl,
      method: req.method,
      timestamp: new Date().toISOString()
    }
  });
};

module.exports = errorHandler;
```

• **data/users.json :**

```
[
  {
    "id": 1,
    "username": "prince18",
    "name": "Prince Prajapati",
    "email": "prince@university.edu",
    "role": "admin",
    "department": "Engineering",
    "status": "active"
  },
  {
    "id": 2,
    "username": "ananya_s",
    "name": "Ananya Sharma",
    "email": "ananya@university.edu",
    "role": "editor",
    "department": "Design",
    "status": "active"
  },
  {
    "id": 3,
    "username": "vikram_m",
    "name": "Vikram Malhotra",
    "email": "vikram@university.edu",
    "role": "viewer",
    "department": "Marketing",
    "status": "pending"
  },
  {
    "id": 4,
    "username": "rohan_v",
    "name": "Rohan Verma",
    "email": "rohan@university.edu",
    "role": "editor",
    "department": "Engineering",
    "status": "active"
  }
]
```

• **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Practical 08 - Routing, Middleware & Request Handling</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; background: #0f172a;
color: #f8fafc; min-height: 100vh; }

    .header {
      background: linear-gradient(135deg, #312e81 0%, #4338ca 50%, #6366f1 100%);
      padding: 2.2rem 1.5rem;
      text-align: center;
```

```
    box-shadow: 0 4px 20px rgba(0,0,0,0.3);
  }
  .header-badge {
    display: inline-block;
    background: rgba(255, 255, 255, 0.2);
    padding: 0.35rem 1rem;
    border-radius: 9999px;
    font-size: 0.8rem;
    font-weight: 700;
    letter-spacing: 0.05em;
    text-transform: uppercase;
    margin-bottom: 0.6rem;
  }
  .header-title { font-size: 2.2rem; font-weight: 800; margin-bottom: 0.4rem; }
  .header-subtitle { font-size: 1rem; color: #e0e7ff; }

  .container { max-width: 1250px; margin: 0 auto; padding: 1.75rem 1.25rem; }

  .pipeline-bar {
    background: #1e293b;
    border: 1px solid #334155;
    border-radius: 12px;
    padding: 1.25rem;
    margin-bottom: 1.5rem;
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    justify-content: space-between;
    gap: 0.75rem;
  }
  .pipeline-step {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    background: #0f172a;
    border: 1px solid #475569;
    padding: 0.5rem 0.9rem;
    border-radius: 8px;
    font-size: 0.82rem;
    font-weight: 600;
  }
  .pipeline-arrow { color: #818cf8; font-weight: bold; }

  .grid { display: grid; grid-template-columns: 1.1fr 1fr; gap: 1.5rem; }
  @media (max-width: 900px) { .grid { grid-template-columns: 1fr; } }

  .card {
    background: #1e293b;
    border: 1px solid #334155;
    border-radius: 14px;
    padding: 1.5rem;
    box-shadow: 0 4px 6px rgba(0,0,0,0.2);
  }
  .card-title {
    font-size: 1.15rem;
    font-weight: 700;
    margin-bottom: 1.25rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    color: #e2e8f0;
    border-bottom: 1px solid #334155;
    padding-bottom: 0.75rem;
  }

  .config-box {
    background: #0f172a;
    border: 1px solid #334155;
    border-radius: 8px;
    padding: 1rem;
```

```
      margin-bottom: 1.25rem;
      display: grid;
      grid-template-columns: 1fr 1fr;
      gap: 1rem;
    }
    .config-item label { display: block; font-size: 0.78rem; color: #94a3b8; margin-bottom: 0.35rem;
font-weight: 600; }
    .config-item input, .config-item select {
      width: 100%;
      background: #1e293b;
      border: 1px solid #475569;
      color: #f8fafc;
      padding: 0.5rem 0.75rem;
      border-radius: 6px;
      font-size: 0.85rem;
    }

    .ep-item {
      display: flex;
      align-items: center;
      justify-content: space-between;
      padding: 0.7rem 0.9rem;
      background: #0f172a;
      border: 1px solid #334155;
      border-radius: 8px;
      margin-bottom: 0.65rem;
      transition: border-color 0.2s;
    }
    .ep-item:hover { border-color: #6366f1; }
    .ep-left { display: flex; align-items: center; gap: 0.6rem; font-family: monospace; font-size:
0.88rem; }
    .badge-method {
      padding: 0.2rem 0.55rem;
      border-radius: 4px;
      font-size: 0.72rem;
      font-weight: 800;
      letter-spacing: 0.05em;
    }
    .method-get { background: #1e3a8a; color: #93c5fd; }
    .method-post { background: #064e3b; color: #6ee7b7; }
    .method-put { background: #78350f; color: #fde68a; }
    .method-del { background: #7f1d1d; color: #fca5a5; }

    .btn-action {
      background: #4f46e5;
      color: white;
      border: none;
      padding: 0.4rem 0.9rem;
      border-radius: 6px;
      font-size: 0.8rem;
      font-weight: 600;
      cursor: pointer;
      transition: background 0.2s;
    }
    .btn-action:hover { background: #4338ca; }
    .btn-danger { background: #dc2626; }
    .btn-danger:hover { background: #b91c1c; }

    .console-header {
      display: flex;
      justify-content: space-between;
      align-items: center;
      margin-bottom: 0.75rem;
      font-size: 0.85rem;
    }
    .status-badge {
      padding: 0.25rem 0.65rem;
      border-radius: 9999px;
      font-weight: 700;
      font-size: 0.75rem;
```

```
    }
    .status-200 { background: #064e3b; color: #6ee7b7; border: 1px solid #059669; }
    .status-400 { background: #7f1d1d; color: #fca5a5; border: 1px solid #dc2626; }
    .status-waiting { background: #334155; color: #94a3b8; }

    .console-body {
      background: #090d16;
      border: 1px solid #1e293b;
      border-radius: 10px;
      padding: 1.25rem;
      font-family: 'Fira Code', 'Courier New', monospace;
      font-size: 0.84rem;
      min-height: 480px;
      max-height: 600px;
      overflow-y: auto;
      color: #38bdf8;
      white-space: pre-wrap;
      line-height: 1.5;
    }
  </style>
</head>
<body>

  <header class="header">
    <span class="header-badge">PRACTICAL NO. 08</span>
    <h1 class="header-title">Express.js Routing, Middleware & Request Handling</h1>
    <p class="header-subtitle">Application-level, Router-level, Built-in, Auth Middleware & Advanced
Request Introspection</p>
  </header>

  <div class="container">
    <div class="pipeline-bar">
      <div class="pipeline-step"><span>🌐</span> Incoming Request</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>🕒</span> Logger & Timestamp</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>📦</span> Body & Static Parsers</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>🔑</span> Auth & Role Guard</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>✅</span> Param / Body Validator</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>🚀</span> Route Controller</div>
      <span class="pipeline-arrow">&rarr;</span>
      <div class="pipeline-step"><span>🛡️</span> Central Error Handler</div>
    </div>

    <div class="grid">
      <!-- Left Column: Controls & Endpoints -->
      <div class="card">
        <div class="card-title">
          <span>⚙️ Header Configuration & Route Tests</span>
          <span style="font-size: 0.75rem; color: #818cf8;">Live Server Port: 5008</span>
        </div>

        <div class="config-box">
          <div class="config-item">
            <label>Header "x-api-key"</label>
            <input type="text" id="cfgApiKey" value="prince_master_key_2026">
          </div>
          <div class="config-item">
            <label>Header "x-user-role"</label>
            <select id="cfgRole">
              <option value="admin">admin (Full Access)</option>
              <option value="editor">editor (Read/Write)</option>
              <option value="viewer">viewer (Read Only)</option>
            </select>
          </div>
        </div>
```

```
        <!-- Endpoints List -->
        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-get">GET</span>
            <span>/api/users</span>
          </div>
          <button class="btn-action" onclick="fetchEndpoint('GET', '/api/users')">Query All</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-get">GET</span>
            <span>/api/users?department=Engineering</span>
          </div>
          <button class="btn-action" onclick="fetchEndpoint('GET', '/api/users?
department=Engineering')">Query Filter</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-get">GET</span>
            <span>/api/users/:id (e.g. ID: 1)</span>
          </div>
          <button class="btn-action" onclick="fetchEndpoint('GET', '/api/users/1')">Test Param</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-post">POST</span>
            <span>/api/users (Protected)</span>
          </div>
          <button class="btn-action" onclick="testCreateUser()">Create Body</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-put">PUT</span>
            <span>/api/users/:id (Protected)</span>
          </div>
          <button class="btn-action" onclick="testUpdateUser()">Update Record</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-del">DELETE</span>
            <span>/api/users/:id (Admin Required)</span>
          </div>
          <button class="btn-action btn-danger" onclick="testDeleteUser()">Delete (Admin)</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-get">GET</span>
            <span>/api/inspect (Full HTTP Matrix)</span>
          </div>
          <button class="btn-action" onclick="fetchEndpoint('GET', '/api/inspect?
source=dashboard&version=v1')">Inspect HTTP</button>
        </div>

        <div class="ep-item">
          <div class="ep-left">
            <span class="badge-method method-get" style="background: #831843; color: #fbcfe8;">ERR</span>
            <span>/api/simulate-error (Central Handler)</span>
          </div>
          <button class="btn-action btn-danger" onclick="fetchEndpoint('GET', '/api/simulate-error?
type=bad-request')">Trigger 400</button>
        </div>
      </div>

      <!-- Right Column: Response Console -->
```

```
      <div class="card">
        <div class="card-title">
          <span>🖥️ Live Middleware & HTTP Response Console</span>
          <span id="responseStatus" class="status-badge status-waiting">Waiting for request...</span>
        </div>

        <div class="console-header">
          <span id="requestMeta" style="color: #94a3b8;">Click any test endpoint to inspect middleware
output</span>
          <button class="btn-action" style="padding: 0.2rem 0.6rem; font-size: 0.75rem;"
onclick="clearConsole()">Clear</button>
        </div>

        <pre id="consoleOutput" class="console-body">// Live response data will appear here...
// Practical 08: Express.js Routing, Middleware & Request Handling</pre>
      </div>
    </div>
  </div>

  <script>
    const baseUrl = window.location.origin;

    async function fetchEndpoint(method, path, body = null) {
      const output = document.getElementById('consoleOutput');
      const statusBadge = document.getElementById('responseStatus');
      const meta = document.getElementById('requestMeta');
      const apiKey = document.getElementById('cfgApiKey').value;
      const role = document.getElementById('cfgRole').value;

      output.textContent = `Connecting to ${method} ${path}...`;
      meta.textContent = `${method} ${path}`;

      const headers = {
        'Content-Type': 'application/json',
        'x-api-key': apiKey,
        'x-user-role': role
      };

      const options = { method, headers };
      if (body) {
        options.body = JSON.stringify(body);
      }

      try {
        const startTime = Date.now();
        const res = await fetch(baseUrl + path, options);
        const duration = Date.now() - startTime;
        const data = await res.json();

        statusBadge.textContent = `HTTP ${res.status} (${duration}ms)`;
        if (res.status >= 200 && res.status < 300) {
          statusBadge.className = 'status-badge status-200';
        } else {
          statusBadge.className = 'status-badge status-400';
        }

        output.textContent = JSON.stringify(data, null, 2);
      } catch (err) {
        statusBadge.textContent = 'Network Error';
        statusBadge.className = 'status-badge status-400';
        output.textContent = `Error: ${err.message}`;
      }
    }

    function testCreateUser() {
      const sample = {
        name: "Neha Patel",
        email: "neha.patel" + Math.floor(Math.random() * 900 + 100) + "@university.edu",
        role: "editor",
        department: "Engineering",
```

```
        status: "active"
      };
      fetchEndpoint('POST', '/api/users', sample);
    }

    function testUpdateUser() {
      const sample = {
        name: "Prince Prajapati (Lead)",
        role: "admin",
        department: "Core Engineering",
        status: "active"
      };
      fetchEndpoint('PUT', '/api/users/1', sample);
    }

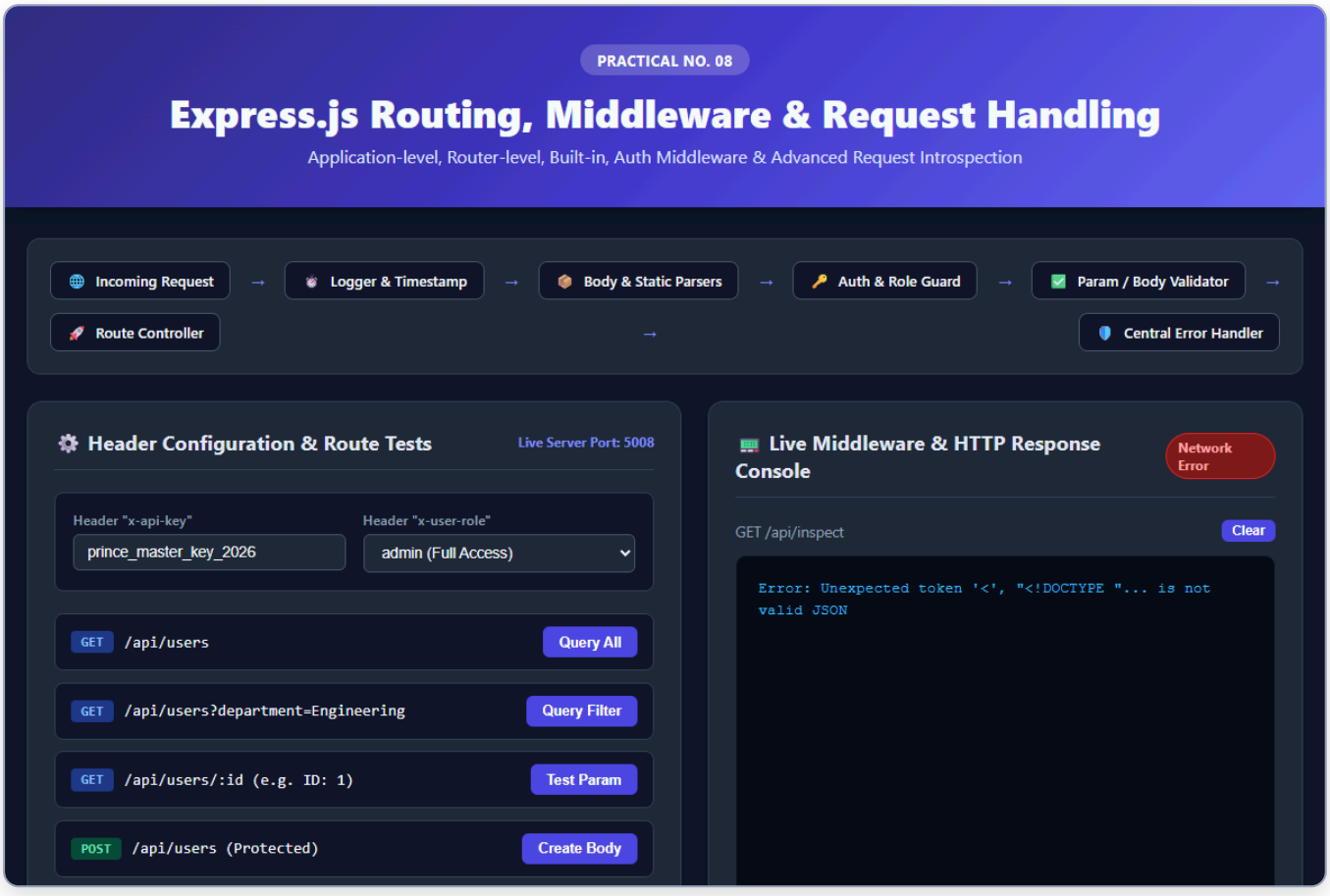
    function testDeleteUser() {
      fetchEndpoint('DELETE', '/api/users/3');
    }

    function clearConsole() {
      document.getElementById('consoleOutput').textContent = '// Console cleared.';
      document.getElementById('responseStatus').textContent = 'Waiting for request...';
      document.getElementById('responseStatus').className = 'status-badge status-waiting';
    }

    // Auto-fetch health on initial page load
    window.addEventListener('DOMContentLoaded', () => {
      fetchEndpoint('GET', '/api/inspect');
    });
  </script>
</body>
</html>
```

4. Output :

Comprehensive Express request handling demonstrating application-level logger, router-level tracking, API-key authentication, role authorization, parameter/body validation, and error handling.



Practical – 09 : Create a file upload module using Multer and store file metadata in MongoDB.

1. Commands :

```
mkdir MERN_09 && cd MERN_09
npm init -y
npm install express mongoose multer cors dotenv morgan
node server.js
```

2. Project structure :

```
MERN_09/
|-- .env
|-- package.json
|-- server.js
|-- config/db.js
|-- config/multerConfig.js
|-- models/FileRecord.js
|-- routes/fileRoutes.js
|-- controllers/fileController.js
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5009
NODE_ENV=development
MONGO_URI=mongodb+srv://prince:prince18@cluster0.76xckkx.mongodb.net/file_upload_db?
retryWrites=true&w=majority
MAX_FILE_SIZE=5242880
```

- **package.json :**

```
{
  "name": "mern-09",
  "version": "1.0.0",
  "description": "Practical 09: File Upload Module using Multer with File Metadata Persistence in MongoDB
Atlas",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.6.1",
    "morgan": "^1.10.0",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
```

```
const path = require('path');
const multer = require('multer');
const connectDB = require('./config/db');
const fileRoutes = require('./routes/fileRoutes');

const app = express();
const PORT = process.env.PORT || 5009;

// Connect to MongoDB Atlas
connectDB();

// Middlewares
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));
// Static route to serve uploaded assets
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// Request Logger
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`[Multer API] ${req.method} ${req.originalUrl} - ${res.statusCode} (${Date.now() -
start}ms)`);
  });
  next();
});

// Welcome / Health Route
app.get('/api/health', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 09: Multer File Upload & MongoDB Metadata API',
    endpoints: {
      uploadSingle: 'POST /api/files/upload-single (field: file)',
      uploadMultiple: 'POST /api/files/upload-multiple (field: files)',
      listFiles: 'GET /api/files',
      download: 'GET /api/files/download/:id',
      delete: 'DELETE /api/files/:id'
    }
  });
});

// Mount Routes
app.use('/api/files', fileRoutes);

// Multer & General Error Handling Middleware
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    if (err.code === 'LIMIT_FILE_SIZE') {
      return res.status(400).json({
        success: false,
        message: 'File upload error: File size exceeds the allowed limit of 5 MB.'
      });
    }
    return res.status(400).json({
      success: false,
      message: `Multer Error: ${err.message}`
    });
  }

  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    message: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
```

```
  console.log(`================================================`);
  console.log(`⚛️ Practical 09: Multer File Upload & MongoDB API`);
  console.log(`⚛️ Server running at: http://localhost:${PORT}`);
  console.log(`⚛️ File Manager UI: http://localhost:${PORT}`);
  console.log(`================================================`);
});

module.exports = app;
```

• **config/db.js :**

```
const dns = require('dns');
try { dns.setDefaultResultOrder('ipv4first'); } catch (e) {}
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch (e) {}
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000
    });
    console.log(`✅ MongoDB Connected Successfully: ${conn.connection.host}`);
    console.log(`⚛️ Database: ${conn.connection.name}`);
  } catch (error) {
    console.error(`❌ MongoDB Connection Error: ${error.message}`);
    console.log(`⚛️ Tip: Check Atlas network access / IP whitelisting or MONGO_URI in .env`);
  }
};

module.exports = connectDB;
```

• **config/multerConfig.js :**

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

const uploadDir = path.join(__dirname, '..', 'uploads');
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

// 1. Configure Disk Storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    const cleanName = file.originalname.replace(/[^a-zA-Z0-9.]/g, '_');
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1e6);
    const ext = path.extname(cleanName);
    const base = path.basename(cleanName, ext);
    cb(null, `${base}-${uniqueSuffix}${ext}`);
  }
});

// 2. Configure File Type Filter
const fileFilter = (req, file, cb) => {
  const allowedMimeTypes = [
    'image/jpeg',
    'image/png',
    'image/webp',
    'image/gif',
    'application/pdf',
    'text/plain',
    'application/msword',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
    'application/json'
```

```
  ];

  if (allowedMimeTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    const error = new Error(`File type "${file.mimetype}" is not supported. Allowed: Images (JPG, PNG,
WEBP), PDF, Word Docs, Text.`);
    error.statusCode = 400;
    cb(error, false);
  }
};

// 3. Export Multer Instance
const upload = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: parseInt(process.env.MAX_FILE_SIZE, 10) || 5 * 1024 * 1024 // 5 MB default
  }
});

module.exports = upload;
```

- **models/FileRecord.js :**

```
const mongoose = require('mongoose');

const fileRecordSchema = new mongoose.Schema(
  {
    originalName: {
      type: String,
      required: [true, 'Original file name is required.'],
      trim: true
    },
    storedName: {
      type: String,
      required: [true, 'Stored file name is required.'],
      unique: true
    },
    mimeType: {
      type: String,
      required: true
    },
    sizeBytes: {
      type: Number,
      required: true
    },
    formattedSize: {
      type: String
    },
    storagePath: {
      type: String,
      required: true
    },
    category: {
      type: String,
      enum: ['Image', 'Document', 'Code/Data', 'Other'],
      default: 'Document'
    },
    uploaderName: {
      type: String,
      default: 'Prince Prajapati'
    },
    description: {
      type: String,
      trim: true,
      default: 'Uploaded via Multer file module'
    }
  },
```

```
  {
    timestamps: true
  }
);

// Pre-save hook to format human-readable size and auto-categorize
fileRecordSchema.pre('save', function (next) {
  if (this.sizeBytes < 1024) {
    this.formattedSize = `${this.sizeBytes} B`;
  } else if (this.sizeBytes < 1024 * 1024) {
    this.formattedSize = `${(this.sizeBytes / 1024).toFixed(1)} KB`;
  } else {
    this.formattedSize = `${(this.sizeBytes / (1024 * 1024)).toFixed(2)} MB`;
  }

  if (this.mimeType.startsWith('image/')) {
    this.category = 'Image';
  } else if (this.mimeType.includes('pdf') || this.mimeType.includes('word') ||
this.mimeType.includes('text')) {
    this.category = 'Document';
  } else if (this.mimeType.includes('json') || this.mimeType.includes('javascript')) {
    this.category = 'Code/Data';
  } else {
    this.category = 'Other';
  }

  next();
});

const FileRecord = mongoose.model('FileRecord', fileRecordSchema);
module.exports = FileRecord;
```

- **routes/fileRoutes.js :**

```
const express = require('express');
const router = express.Router();
const upload = require('../config/multerConfig');
const {
  uploadSingleFile,
  uploadMultipleFiles,
  getAllFiles,
  downloadFile,
  deleteFile
} = require('../controllers/fileController');

// Upload single file (field name: 'file')
router.post('/upload-single', upload.single('file'), uploadSingleFile);

// Upload multiple files (up to 5, field name: 'files')
router.post('/upload-multiple', upload.array('files', 5), uploadMultipleFiles);

// Get list of uploaded file records from MongoDB
router.get('/', getAllFiles);

// Download file binary
router.get('/download/:id', downloadFile);

// Delete file record and binary
router.delete('/:id', deleteFile);

module.exports = router;
```

- **controllers/fileController.js :**

```
const fs = require('fs');
const path = require('path');
const FileRecord = require('../models/FileRecord');
```

```
/**
 * @desc Upload Single File & Persist Metadata in MongoDB
 * @route POST /api/files/upload-single
 */
const uploadSingleFile = async (req, res, next) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: 'No file received. Please provide a file under the field "file".'
      });
    }

    const { uploaderName, description } = req.body;

    const fileDoc = await FileRecord.create({
      originalName: req.file.originalname,
      storedName: req.file.filename,
      mimeType: req.file.mimetype,
      sizeBytes: req.file.size,
      storagePath: req.file.path,
      uploaderName: uploaderName || 'Prince Prajapati',
      description: description || 'Practical 09 upload'
    });

    res.status(201).json({
      success: true,
      message: 'File uploaded and metadata stored in MongoDB Atlas successfully.',
      file: fileDoc
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @desc Upload Multiple Files (up to 5) & Persist in MongoDB
 * @route POST /api/files/upload-multiple
 */
const uploadMultipleFiles = async (req, res, next) => {
  try {
    if (!req.files || req.files.length === 0) {
      return res.status(400).json({
        success: false,
        message: 'No files uploaded. Provide files under the field "files".'
      });
    }

    const { uploaderName, description } = req.body;

    const docs = req.files.map((f) => ({
      originalName: f.originalname,
      storedName: f.filename,
      mimeType: f.mimetype,
      sizeBytes: f.size,
      storagePath: f.path,
      uploaderName: uploaderName || 'Prince Prajapati',
      description: description || 'Multi-file batch upload'
    }));

    const savedRecords = await FileRecord.insertMany(docs);

    res.status(201).json({
      success: true,
      message: `Successfully uploaded and saved ${savedRecords.length} files into MongoDB.`,
      count: savedRecords.length,
      files: savedRecords
    });
  } catch (error) {
    next(error);
```

```
  }
};

/**
 * @desc List all stored file metadata records from MongoDB
 * @route GET /api/files
 */
const getAllFiles = async (req, res, next) => {
  try {
    const { category, search } = req.query;
    const query = {};

    if (category && category !== 'All') {
      query.category = category;
    }

    if (search) {
      query.originalName = { $regex: search, $options: 'i' };
    }

    const files = await FileRecord.find(query).sort({ createdAt: -1 });
    const totalStorageBytes = files.reduce((acc, f) => acc + f.sizeBytes, 0);

    res.status(200).json({
      success: true,
      totalCount: files.length,
      totalStorageUsed: (totalStorageBytes / (1024 * 1024)).toFixed(2) + ' MB',
      data: files
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @desc Download File Binary from Disk
 * @route GET /api/files/download/:id
 */
const downloadFile = async (req, res, next) => {
  try {
    const file = await FileRecord.findById(req.params.id);
    if (!file) {
      return res.status(404).json({ success: false, message: 'File record not found.' });
    }

    if (!fs.existsSync(file.storagePath)) {
      return res.status(410).json({ success: false, message: 'File exists in database but physical file
is missing from disk.' });
    }

    res.download(file.storagePath, file.originalName);
  } catch (error) {
    next(error);
  }
};

/**
 * @desc Delete file from Disk and delete metadata from MongoDB
 * @route DELETE /api/files/:id
 */
const deleteFile = async (req, res, next) => {
  try {
    const file = await FileRecord.findById(req.params.id);
    if (!file) {
      return res.status(404).json({ success: false, message: 'File record not found.' });
    }

    if (fs.existsSync(file.storagePath)) {
      fs.unlinkSync(file.storagePath);
    }
```

```
    await FileRecord.findByIdAndDelete(req.params.id);

    res.status(200).json({
      success: true,
      message: `File "${file.originalName}" permanently deleted from disk and MongoDB.`,
      deletedId: req.params.id
    });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  uploadSingleFile,
  uploadMultipleFiles,
  getAllFiles,
  downloadFile,
  deleteFile
};
```

- **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Practical 09 - Multer File Upload & MongoDB Metadata</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; background: #0b1329;
color: #f8fafc; min-height: 100vh; }

    .header {
      background: linear-gradient(135deg, #065f46 0%, #047857 50%, #10b981 100%);
      padding: 2.2rem 1.5rem;
      text-align: center;
      box-shadow: 0 4px 20px rgba(0,0,0,0.3);
    }
    .header-badge {
      display: inline-block;
      background: rgba(255, 255, 255, 0.2);
      padding: 0.35rem 1rem;
      border-radius: 9999px;
      font-size: 0.8rem;
      font-weight: 700;
      letter-spacing: 0.05em;
      text-transform: uppercase;
      margin-bottom: 0.6rem;
    }
    .header-title { font-size: 2.2rem; font-weight: 800; margin-bottom: 0.4rem; }
    .header-subtitle { font-size: 1rem; color: #d1fae5; }

    .container { max-width: 1250px; margin: 0 auto; padding: 1.75rem 1.25rem; }

    .grid { display: grid; grid-template-columns: 440px 1fr; gap: 1.5rem; }
    @media (max-width: 960px) { .grid { grid-template-columns: 1fr; } }

    .card {
      background: #151f38;
      border: 1px solid #1e293b;
      border-radius: 14px;
      padding: 1.5rem;
      box-shadow: 0 4px 6px rgba(0,0,0,0.25);
    }
    .card-title {
      font-size: 1.15rem;
      font-weight: 700;
```

```
      margin-bottom: 1.25rem;
      display: flex;
      align-items: center;
      justify-content: space-between;
      color: #e2e8f0;
      border-bottom: 1px solid #223254;
      padding-bottom: 0.75rem;
    }

    /* Upload Dropzone */
    .dropzone {
      border: 2px dashed #10b981;
      border-radius: 12px;
      background: rgba(16, 185, 129, 0.04);
      padding: 2rem 1.5rem;
      text-align: center;
      cursor: pointer;
      transition: all 0.2s ease;
      margin-bottom: 1.25rem;
    }
    .dropzone:hover {
      background: rgba(16, 185, 129, 0.08);
      border-color: #34d399;
    }
    .dropzone-icon { font-size: 2.5rem; margin-bottom: 0.5rem; }
    .dropzone-text { font-size: 0.95rem; font-weight: 600; color: #e2e8f0; }
    .dropzone-hint { font-size: 0.78rem; color: #94a3b8; margin-top: 0.3rem; }

    .form-group { margin-bottom: 1rem; }
    .form-label { display: block; font-size: 0.8rem; font-weight: 600; color: #94a3b8; margin-bottom:
0.35rem; }
    .form-input {
      width: 100%;
      background: #0b1329;
      border: 1px solid #334155;
      color: #f8fafc;
      padding: 0.6rem 0.8rem;
      border-radius: 8px;
      font-size: 0.88rem;
    }

    .btn-submit {
      width: 100%;
      background: #10b981;
      color: #042f2e;
      border: none;
      padding: 0.75rem;
      border-radius: 8px;
      font-size: 0.95rem;
      font-weight: 700;
      cursor: pointer;
      transition: background 0.2s;
    }
    .btn-submit:hover { background: #34d399; }

    /* File Table */
    .table-wrap { overflow-x: auto; margin-top: 1rem; }
    table { width: 100%; border-collapse: collapse; font-size: 0.86rem; }
    th { background: #0b1329; color: #94a3b8; text-align: left; padding: 0.75rem 0.9rem; font-weight:
600; border-bottom: 1px solid #334155; }
    td { padding: 0.75rem 0.9rem; border-bottom: 1px solid #1e293b; vertical-align: middle; }
    tr:hover { background: rgba(255, 255, 255, 0.02); }

    .badge-cat {
      display: inline-block;
      padding: 0.2rem 0.6rem;
      border-radius: 9999px;
      font-size: 0.72rem;
      font-weight: 700;
    }
```

```
    .cat-Image { background: #1e3a8a; color: #93c5fd; }
    .cat-Document { background: #78350f; color: #fde68a; }
    .cat-Other { background: #334155; color: #cbd5e1; }

    .btn-sm-action {
      background: #1e293b;
      border: 1px solid #334155;
      color: #38bdf8;
      padding: 0.3rem 0.6rem;
      border-radius: 6px;
      font-size: 0.75rem;
      cursor: pointer;
      text-decoration: none;
      display: inline-flex;
      align-items: center;
      gap: 0.25rem;
    }
    .btn-sm-action:hover { background: #334155; }
    .btn-sm-del { color: #f87171; border-color: #7f1d1d; }
    .btn-sm-del:hover { background: #7f1d1d; color: white; }

    .console-box {
      background: #060a14;
      border: 1px solid #1e293b;
      border-radius: 8px;
      padding: 1rem;
      font-family: monospace;
      font-size: 0.8rem;
      color: #34d399;
      max-height: 200px;
      overflow-y: auto;
      margin-top: 1.25rem;
      white-space: pre-wrap;
    }
  </style>
</head>
<body>

  <header class="header">
    <span class="header-badge">PRACTICAL NO. 09</span>
    <h1 class="header-title">Multer File Upload &amp;amp; MongoDB Metadata Storage</h1>
    <p class="header-subtitle">DiskStorage Engine, Multi-MIME Validation, 5MB Size Limits &amp;amp; Cloud
Document Persistence</p>
  </header>

  <div class="container">
    <div class="grid">
      <!-- Left Column: Upload Form -->
      <div class="card">
        <div class="card-title">
          <span>📤 File Upload Module</span>
          <span style="font-size: 0.78rem; color: #34d399;">Max: 5 MB</span>
        </div>

        <form id="uploadForm" enctype="multipart/form-data">
          <div class="dropzone" onclick="document.getElementById('fileInput').click()">
            <div class="dropzone-icon">📁</div>
            <div class="dropzone-text" id="dropzoneText">Click to Select Files</div>
            <div class="dropzone-hint">Supported: PNG, JPG, WEBP, PDF, DOCX, TXT (Single or Batch up to
5)</div>
            <input type="file" id="fileInput" name="files" multiple style="display: none;"
onchange="handleFileSelected(event)">
          </div>

          <div class="form-group">
            <label class="form-label">Uploader Name</label>
            <input type="text" id="uploaderName" name="uploaderName" class="form-input" value="Prince
Prajapati">
          </div>
```

```
          <div class="form-group">
            <label class="form-label">File Description / Notes</label>
            <input type="text" id="description" name="description" class="form-input" placeholder="e.g.
MERN practical lab report">
          </div>

          <button type="button" class="btn-submit" onclick="submitUpload()">Upload to Server &amp;
MongoDB</button>
        </form>

        <div class="console-box" id="uploadConsole">// Upload response log will display here...</div>
      </div>

      <!-- Right Column: Stored Files from MongoDB -->
      <div class="card">
        <div class="card-title">
          <span>⚛️ Stored File Records (MongoDB Atlas)</span>
          <button class="btn-sm-action" onclick="loadFiles()">🔄 Refresh</button>
        </div>

        <div style="display: flex; gap: 0.75rem; margin-bottom: 1rem;">
          <input type="text" id="searchInput" class="form-input" placeholder="Search files by name..."
oninput="filterFiles()">
        </div>

        <div class="table-wrap">
          <table>
            <thead>
              <tr>
                <th>Original Name</th>
                <th>Category</th>
                <th>Size</th>
                <th>Uploader</th>
                <th style="text-align: right;">Actions</th>
              </tr>
            </thead>
            <tbody id="filesTableBody">
              <tr>
                <td colspan="5" style="text-align: center; color: #94a3b8; padding: 2rem;">Loading files
from database...</td>
              </tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

  <script>
    let allFiles = [];

    function handleFileSelected(e) {
      const files = e.target.files;
      const text = document.getElementById('dropzoneText');
      if (files.length === 1) {
        text.textContent = `Selected: ${files[0].name} (${(files[0].size/1024).toFixed(1)} KB)`;
      } else if (files.length > 1) {
        text.textContent = `Selected: ${files.length} files`;
      }
    }

    async function submitUpload() {
      const fileInput = document.getElementById('fileInput');
      const log = document.getElementById('uploadConsole');

      if (!fileInput.files || fileInput.files.length === 0) {
        alert('Please choose at least one file to upload!');
        return;
      }
```

```
      const formData = new FormData();
      const uploader = document.getElementById('uploaderName').value;
      const desc = document.getElementById('description').value;

      formData.append('uploaderName', uploader);
      formData.append('description', desc);

      let endpoint = '/api/files/upload-single';
      if (fileInput.files.length > 1) {
        endpoint = '/api/files/upload-multiple';
        for (let i = 0; i < fileInput.files.length; i++) {
          formData.append('files', fileInput.files[i]);
        }
      } else {
        formData.append('file', fileInput.files[0]);
      }

      log.textContent = `Uploading ${fileInput.files.length} file(s)...`;

      try {
        const res = await fetch(endpoint, {
          method: 'POST',
          body: formData
        });
        const data = await res.json();
        log.textContent = JSON.stringify(data, null, 2);

        if (res.ok) {
          fileInput.value = '';
          document.getElementById('dropzoneText').textContent = 'Click to Select Files';
          loadFiles();
        }
      } catch (err) {
        log.textContent = `Upload Error: ${err.message}`;
      }
    }

    async function loadFiles() {
      const tbody = document.getElementById('filesTableBody');
      try {
        const res = await fetch('/api/files');
        const json = await res.json();
        allFiles = json.data || [];
        renderTable(allFiles);
      } catch (err) {
        tbody.innerHTML = `<tr><td colspan="5" style="text-align: center; color: #f87171;">Failed to load
files: ${err.message}</td></tr>`;
      }
    }

    function renderTable(files) {
      const tbody = document.getElementById('filesTableBody');
      if (!files || files.length === 0) {
        tbody.innerHTML = `<tr><td colspan="5" style="text-align: center; color: #94a3b8; padding:
2.5rem;">No files uploaded yet. Upload a file on the left!</td></tr>`;
        return;
      }

      tbody.innerHTML = files.map(f => `
        <tr>
          <td>
            <div style="font-weight: 600; color: #f8fafc;">${f.originalName}</div>
            <div style="font-size: 0.72rem; color: #94a3b8; font-family: monospace;">${f.storedName}
</div>
          </td>
          <td><span class="badge-cat cat-${f.category || 'Other'}">${f.category || 'Document'}</span>
</td>
          <td style="color: #6ee7b7; font-weight: 600;">${f.formattedSize || (f.sizeBytes + ' B')}</td>
          <td>${f.uploaderName || 'Prince'}</td>
          <td style="text-align: right; white-space: nowrap;">
```

```
            <a href="/api/files/download/${f._id}" class="btn-sm-action" target="_blank">⬇ Download</a>
            <button class="btn-sm-action btn-sm-del" onclick="deleteFile('${f._id}',
'${f.originalName}')">🗑</button>
          </td>
        </tr>
      `).join('');
    }

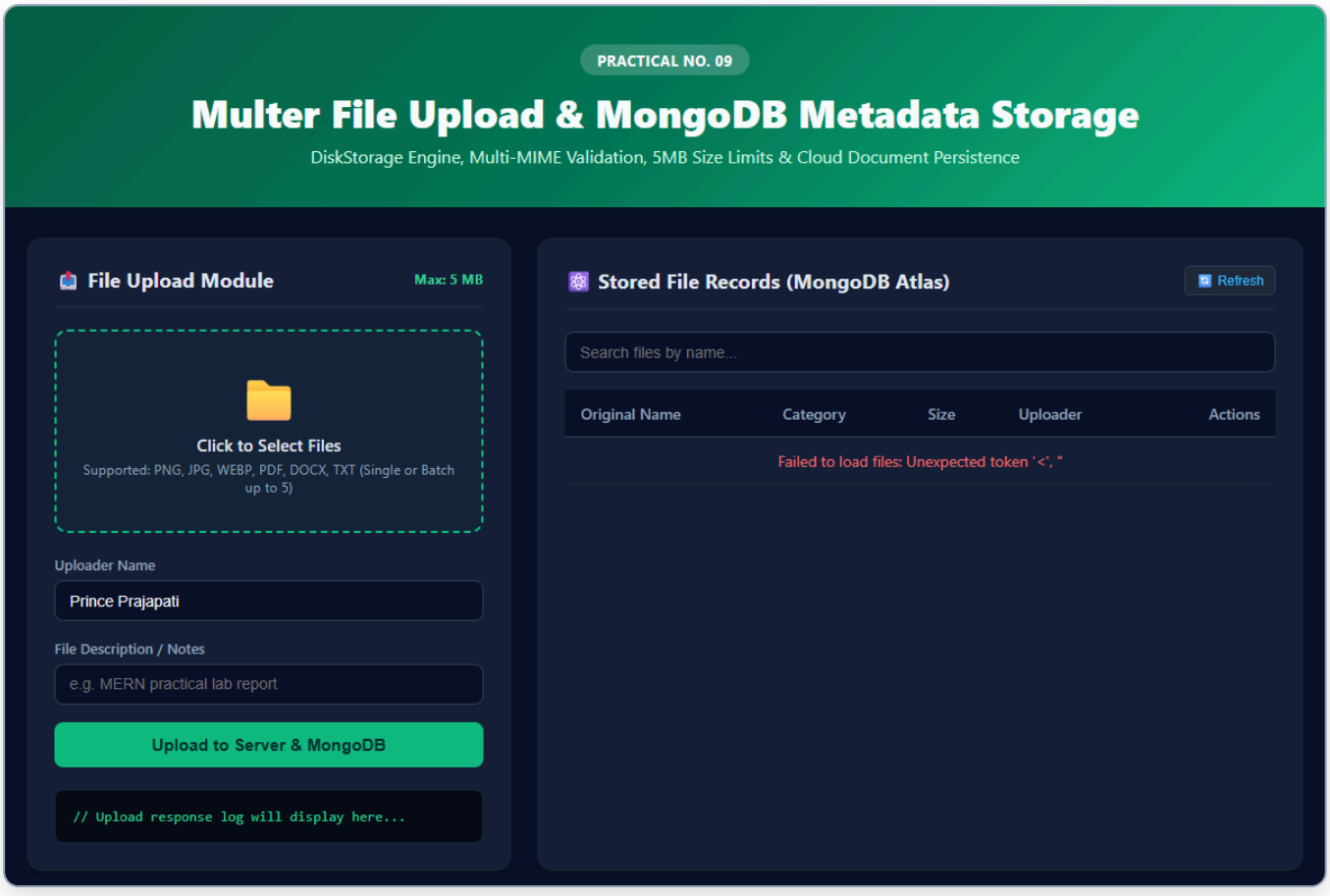
    function filterFiles() {
      const q = document.getElementById('searchInput').value.toLowerCase();
      const filtered = allFiles.filter(f => f.originalName.toLowerCase().includes(q) || (f.uploaderName
&& f.uploaderName.toLowerCase().includes(q)));
      renderTable(filtered);
    }

    async function deleteFile(id, name) {
      if (!confirm(`Delete "${name}" permanently?`)) return;
      try {
        const res = await fetch(`/api/files/${id}`, { method: 'DELETE' });
        const json = await res.json();
        document.getElementById('uploadConsole').textContent = JSON.stringify(json, null, 2);
        loadFiles();
      } catch (err) {
        alert('Delete failed: ' + err.message);
      }
    }

    // Auto-load on open
    window.addEventListener('DOMContentLoaded', loadFiles);
  </script>
</body>
</html>
```

4. Output :

Multipart file upload module utilizing Multer diskStorage, extension and MIME type validation, 5MB size enforcement, disk archival, and metadata persistence in MongoDB Atlas.



Practical – 10 : Develop a complete MERN application integrating React Frontend, Express Backend and MongoDB Database.

1. Commands :

```
# Backend Setup:
cd MERN_10/backend
npm install
node server.js

# Frontend Setup:
cd ../frontend
npm install
npm run dev
```

2. Project structure :

```
MERN_10/
|-- backend/.env
|-- backend/package.json
|-- backend/server.js
|-- backend/seed.js
|-- backend/config/db.js
|-- backend/models/InventoryItem.js
|-- backend/routes/inventoryRoutes.js
|-- backend/controllers/inventoryController.js
|-- backend/public/index.html
|-- frontend/package.json
|-- frontend/index.html
|-- frontend/vite.config.js
|-- frontend/src/main.jsx
|-- frontend/src/index.css
|-- frontend/src/App.jsx
|-- frontend/src/components/StatsCard.jsx
|-- frontend/src/components/InventoryForm.jsx
|-- frontend/src/components/InventoryTable.jsx
```

3. Source Code :

- **backend/.env :**

```
PORT=5010
NODE_ENV=development
MONGO_URI=mongodb+srv://prince:prince18@cluster0.76xckkx.mongodb.net/mern10_inventory_db?
retryWrites=true&w=majority
```

- **backend/package.json :**

```
{
  "name": "mern-10-backend",
  "version": "1.0.0",
  "description": "Backend API for Practical 10 - Complete MERN Integration",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.6.1",
    "morgan": "^1.10.0"
  },
```

```
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

• **backend/server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');
const inventoryRoutes = require('./routes/inventoryRoutes');

const app = express();
const PORT = process.env.PORT || 5010;

// Connect to MongoDB Atlas
connectDB();

// Middlewares
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve frontend build if present or public fallback
const frontendDist = path.join(__dirname, '..', 'frontend', 'dist');
const publicDir = path.join(__dirname, 'public');
app.use(express.static(frontendDist));
app.use(express.static(publicDir));

// Request logger
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`[MERN_10 API] ${req.method} ${req.originalUrl} - ${res.statusCode} (${Date.now() -
start}ms)`);
  });
  next();
});

// API Routes
app.use('/api/inventory', inventoryRoutes);

// Health check
app.get('/api/health', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 10: Complete MERN Full-Stack Inventory Management API',
    database: 'MongoDB Atlas Connected',
    endpoints: {
      getAllItems: 'GET /api/inventory',
      getItemById: 'GET /api/inventory/:id',
      createItem: 'POST /api/inventory',
      updateItem: 'PUT /api/inventory/:id',
      deleteItem: 'DELETE /api/inventory/:id',
      stats: 'GET /api/inventory/stats/overview'
    }
  });
});

// Fallback to React app or JSON 404
app.get('*', (req, res) => {
  const indexPath = path.join(frontendDist, 'index.html');
  const publicIndexPath = path.join(publicDir, 'index.html');
  const fs = require('fs');

  if (fs.existsSync(indexPath)) {
    res.sendFile(indexPath);
```

```
  } else if (fs.existsSync(publicIndexPath)) {
    res.sendFile(publicIndexPath);
  } else {
    res.status(404).json({ success: false, message: 'Route not found' });
  }
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`⚛️ Practical 10: Complete Full-Stack MERN System`);
  console.log(`⚛️ Server running at: http://localhost:${PORT}`);
  console.log(`⚛️ Inventory API: http://localhost:${PORT}/api/inventory`);
  console.log(`================================================`);
});

module.exports = app;
```

• **backend/seed.js :**

```
require('dotenv').config();
const dns = require('dns');
try { dns.setDefaultResultOrder('ipv4first'); } catch (e) {}
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch (e) {}
const mongoose = require('mongoose');
const InventoryItem = require('./models/InventoryItem');

const sampleItems = [
  {
    sku: 'ELC-101',
    name: 'Logitech MX Master 3S Mouse',
    category: 'Electronics',
    quantity: 24,
    unitPrice: 8999,
    supplier: 'Logitech India',
    reorderLevel: 5
  },
  {
    sku: 'ELC-102',
    name: 'Dell UltraSharp 27" 4K USB-C Monitor',
    category: 'Computing',
    quantity: 12,
    unitPrice: 42999,
    supplier: 'Dell Technologies',
    reorderLevel: 4
  },
  {
    sku: 'ACC-201',
    name: 'Keychron Q1 Pro Mechanical Keyboard',
    category: 'Accessories',
    quantity: 6,
    unitPrice: 15499,
    supplier: 'Keychron Direct',
    reorderLevel: 8
  },
  {
    sku: 'FUR-301',
    name: 'Ergonomic Mesh Executive Chair',
    category: 'Furniture',
    quantity: 15,
    unitPrice: 18500,
    supplier: 'GreenSoul Seating',
    reorderLevel: 5
  },
  {
    sku: 'OFC-401',
    name: 'Premium A4 Copier Paper (5 Reams)',
    category: 'Office Supplies',
    quantity: 50,
    unitPrice: 1250,
```

```
    supplier: 'JK Paper Mills',
    reorderLevel: 10
  },
  {
    sku: 'ELC-105',
    name: 'USB-C Multiport Hub (10-in-1)',
    category: 'Accessories',
    quantity: 0,
    unitPrice: 3499,
    supplier: 'Anker Innovations',
    reorderLevel: 10
  }
];

const seedData = async () => {
  try {
    console.log('Connecting to MongoDB...');
    await mongoose.connect(process.env.MONGO_URI);
    console.log('✅ Connected to MongoDB.');

    console.log('Clearing old inventory...');
    await InventoryItem.deleteMany({});

    console.log('Inserting seed items...');
    await InventoryItem.insertMany(sampleItems);

    console.log(`⚛️ Successfully seeded ${sampleItems.length} inventory products!`);
    process.exit(0);
  } catch (err) {
    console.error('❌ Seeder failed:', err.message);
    process.exit(1);
  }
};

seedData();
```

• **backend/config/db.js :**

```
const dns = require('dns');
try { dns.setDefaultResultOrder('ipv4first'); } catch (e) {}
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch (e) {}
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000
    });
    console.log(`✅ MongoDB Connected Successfully: ${conn.connection.host}`);
    console.log(`⚛️ Database: ${conn.connection.name}`);
  } catch (error) {
    console.error(`❌ MongoDB Connection Error: ${error.message}`);
  }
};

module.exports = connectDB;
```

• **backend/models/InventoryItem.js :**

```
const mongoose = require('mongoose');

const inventoryItemSchema = new mongoose.Schema(
  {
    sku: {
      type: String,
      required: [true, 'SKU code is required.'],
      unique: true,
      trim: true,
```

```
      uppercase: true
    },
    name: {
      type: String,
      required: [true, 'Product / Item Name is required.'],
      trim: true
    },
    category: {
      type: String,
      required: [true, 'Category is required.'],
      enum: ['Electronics', 'Computing', 'Office Supplies', 'Accessories', 'Furniture']
    },
    quantity: {
      type: Number,
      required: [true, 'Stock quantity is required.'],
      min: [0, 'Quantity cannot be negative.']
    },
    unitPrice: {
      type: Number,
      required: [true, 'Unit price is required.'],
      min: [0, 'Unit price cannot be negative.']
    },
    supplier: {
      type: String,
      default: 'Prince Tech Logistics'
    },
    reorderLevel: {
      type: Number,
      default: 10
    }
  },
  {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

inventoryItemSchema.virtual('stockStatus').get(function () {
  if (this.quantity === 0) return 'Out of Stock';
  if (this.quantity <= this.reorderLevel) return 'Low Stock';
  return 'In Stock';
});

inventoryItemSchema.virtual('totalValue').get(function () {
  return (this.quantity * this.unitPrice).toFixed(2);
});

const InventoryItem = mongoose.model('InventoryItem', inventoryItemSchema);
module.exports = InventoryItem;
```

• **backend/routes/inventoryRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllItems,
  getItemById,
  createItem,
  updateItem,
  deleteItem,
  getInventoryStats
} = require('../controllers/inventoryController');

// Stats route (must precede :id)
router.get('/stats/overview', getInventoryStats);

router.route('/')
  .get(getAllItems)
```

```
  .post(createItem);

router.route('/:id')
  .get(getItemById)
  .put(updateItem)
  .delete(deleteItem);

module.exports = router;
```

• **backend/controllers/inventoryController.js :**

```
const InventoryItem = require('../models/InventoryItem');

/**
 * @desc Get all inventory items with optional search & category filter
 * @route GET /api/inventory
 */
const getAllItems = async (req, res) => {
  try {
    const { category, search, stockStatus, sortBy } = req.query;
    const query = {};

    if (category && category !== 'All') {
      query.category = category;
    }

    if (search) {
      query.$or = [
        { name: { $regex: search, $options: 'i' } },
        { sku: { $regex: search, $options: 'i' } },
        { supplier: { $regex: search, $options: 'i' } }
      ];
    }

    let sort = { createdAt: -1 };
    if (sortBy === 'price_asc') sort = { unitPrice: 1 };
    if (sortBy === 'price_desc') sort = { unitPrice: -1 };
    if (sortBy === 'qty_asc') sort = { quantity: 1 };
    if (sortBy === 'qty_desc') sort = { quantity: -1 };

    let items = await InventoryItem.find(query).sort(sort);

    if (stockStatus && stockStatus !== 'All') {
      items = items.filter((item) => item.stockStatus === stockStatus);
    }

    res.status(200).json({
      success: true,
      count: items.length,
      data: items
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};

/**
 * @desc Get single inventory item by ID
 * @route GET /api/inventory/:id
 */
const getItemById = async (req, res) => {
  try {
    const item = await InventoryItem.findById(req.params.id);
    if (!item) {
      return res.status(404).json({ success: false, message: 'Inventory item not found.' });
    }
    res.status(200).json({ success: true, data: item });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
```

```
  }
};

/**
 * @desc Create new inventory item
 * @route POST /api/inventory
 */
const createItem = async (req, res) => {
  try {
    const item = await InventoryItem.create(req.body);
    res.status(201).json({
      success: true,
      message: `Item "${item.name}" registered in inventory.`,
      data: item
    });
  } catch (error) {
    if (error.code === 11000) {
      return res.status(409).json({ success: false, message: 'An item with this SKU code already exists.'
});
    }
    res.status(400).json({ success: false, message: error.message });
  }
};

/**
 * @desc Update inventory item
 * @route PUT /api/inventory/:id
 */
const updateItem = async (req, res) => {
  try {
    const item = await InventoryItem.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true
    });
    if (!item) {
      return res.status(404).json({ success: false, message: 'Item not found.' });
    }
    res.status(200).json({
      success: true,
      message: `Item "${item.name}" updated successfully.`,
      data: item
    });
  } catch (error) {
    res.status(400).json({ success: false, message: error.message });
  }
};

/**
 * @desc Delete inventory item
 * @route DELETE /api/inventory/:id
 */
const deleteItem = async (req, res) => {
  try {
    const item = await InventoryItem.findByIdAndDelete(req.params.id);
    if (!item) {
      return res.status(404).json({ success: false, message: 'Item not found.' });
    }
    res.status(200).json({
      success: true,
      message: `Item "${item.name}" (${item.sku}) deleted.`,
      deletedId: req.params.id
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};

/**
 * @desc Aggregate metrics for inventory dashboard
 * @route GET /api/inventory/stats/overview
```

```
 */
const getInventoryStats = async (req, res) => {
  try {
    const items = await InventoryItem.find();
    const totalItems = items.length;
    const totalUnits = items.reduce((sum, item) => sum + item.quantity, 0);
    const totalValuation = items.reduce((sum, item) => sum + item.quantity * item.unitPrice, 0);
    const lowStockCount = items.filter((item) => item.quantity <= item.reorderLevel && item.quantity >
0).length;
    const outOfStockCount = items.filter((item) => item.quantity === 0).length;

    res.status(200).json({
      success: true,
      stats: {
        totalItems,
        totalUnits,
        totalValuation: Math.round(totalValuation),
        lowStockCount,
        outOfStockCount
      }
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};

module.exports = {
  getAllItems,
  getItemById,
  createItem,
  updateItem,
  deleteItem,
  getInventoryStats
};
```

- **backend/public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Practical 10 - Full Stack MERN Application</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, -apple-system, sans-serif; background: #0f172a; color: #f8fafc; min-
height: 100vh; }
    .header { background: linear-gradient(135deg, #1e1b4b 0%, #312e81 50%, #4338ca 100%); padding: 2.2rem
1.5rem; text-align: center; }
    .header-badge { display: inline-block; background: rgba(255,255,255,0.2); padding: 0.35rem 1rem;
border-radius: 9999px; font-size: 0.8rem; font-weight: 700; text-transform: uppercase; margin-bottom:
0.6rem; }
    .header-title { font-size: 2.2rem; font-weight: 800; margin-bottom: 0.4rem; }
    .header-subtitle { font-size: 1rem; color: #c7d2fe; }
    .container { max-width: 1250px; margin: 0 auto; padding: 1.75rem 1.25rem; }

    .stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap:
1.25rem; margin-bottom: 2rem; }
    .stat-card { background: #1e293b; border: 1px solid #334155; border-radius: 12px; padding: 1.25rem;
display: flex; align-items: center; gap: 1rem; }
    .stat-icon { width: 48px; height: 48px; border-radius: 10px; display: flex; align-items: center;
justify-content: center; font-size: 1.5rem; }
    .stat-val { font-size: 1.6rem; font-weight: 800; }
    .stat-lbl { font-size: 0.82rem; color: #94a3b8; }

    .main-layout { display: grid; grid-template-columns: 380px 1fr; gap: 1.5rem; align-items: start; }
    @media (max-width: 950px) { .main-layout { grid-template-columns: 1fr; } }
    .card { background: #1e293b; border: 1px solid #334155; border-radius: 14px; padding: 1.5rem; box-
shadow: 0 4px 6px rgba(0,0,0,0.2); }
```

```
    .card-title { font-size: 1.15rem; font-weight: 700; margin-bottom: 1.25rem; display: flex; justify-
content: space-between; align-items: center; border-bottom: 1px solid #334155; padding-bottom: 0.75rem; }

    .form-group { margin-bottom: 1rem; }
    .form-label { display: block; font-size: 0.8rem; font-weight: 600; color: #94a3b8; margin-bottom:
0.35rem; }
    .form-input, .form-select { width: 100%; background: #0f172a; border: 1px solid #334155; color: #fff;
padding: 0.6rem 0.85rem; border-radius: 8px; font-size: 0.88rem; outline: none; }
    .form-input:focus, .form-select:focus { border-color: #6366f1; }

    .btn { display: inline-flex; align-items: center; justify-content: center; gap: 0.5rem; font-family:
inherit; font-size: 0.88rem; font-weight: 600; padding: 0.65rem 1.25rem; border-radius: 8px; border:
none; cursor: pointer; }
    .btn-primary { background: #4f46e5; color: white; width: 100%; }
    .btn-primary:hover { background: #4338ca; }
    .btn-sm { padding: 0.35rem 0.7rem; font-size: 0.78rem; border-radius: 6px; }
    .btn-del { background: rgba(239, 68, 68, 0.2); color: #fca5a5; border: 1px solid #ef4444; }

    .table-wrap { overflow-x: auto; margin-top: 1rem; }
    table { width: 100%; border-collapse: collapse; font-size: 0.86rem; }
    th { background: #0f172a; color: #94a3b8; text-align: left; padding: 0.75rem 0.85rem; border-bottom:
1px solid #334155; }
    td { padding: 0.75rem 0.85rem; border-bottom: 1px solid #334155; vertical-align: middle; }
    tr:hover { background: rgba(255, 255, 255, 0.02); }

    .badge-status { padding: 0.2rem 0.6rem; border-radius: 9999px; font-size: 0.72rem; font-weight: 700;
}
    .status-in { background: rgba(16, 185, 129, 0.2); color: #6ee7b7; border: 1px solid #10b981; }
    .status-low { background: rgba(245, 158, 11, 0.2); color: #fde68a; border: 1px solid #f59e0b; }
    .status-out { background: rgba(239, 68, 68, 0.2); color: #fca5a5; border: 1px solid #ef4444; }
  </style>
</head>
<body>
  <header class="header">
    <span class="header-badge">PRACTICAL NO. 10</span>
    <h1 class="header-title">Full-Stack MERN Inventory Application</h1>
    <p class="header-subtitle">React Client (Vite) &bull; Express.js Backend &bull; MongoDB Atlas Cloud
Database &bull; Node.js Architecture</p>
  </header>

  <div class="container">
    <div class="stats-grid">
      <div class="stat-card">
        <div class="stat-icon" style="background: rgba(79, 70, 229, 0.15); color: #818cf8;">📦</div>
        <div>
          <div class="stat-val" id="statItems">0</div>
          <div class="stat-lbl">Total Products</div>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-icon" style="background: rgba(16, 185, 129, 0.15); color: #34d399;">💰</div>
        <div>
          <div class="stat-val" id="statVal">₹0</div>
          <div class="stat-lbl">Total Valuation</div>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-icon" style="background: rgba(245, 158, 11, 0.15); color: #fbbf24;">⚠️</div>
        <div>
          <div class="stat-val" id="statLow">0</div>
          <div class="stat-lbl">Low Stock Items</div>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-icon" style="background: rgba(239, 68, 68, 0.15); color: #f87171;">🚫</div>
        <div>
          <div class="stat-val" id="statOut">0</div>
          <div class="stat-lbl">Out of Stock</div>
        </div>
      </div>
```

```
    </div>

    <div class="main-layout">
      <!-- Left Column: Add Product Form -->
      <div class="card">
        <div class="card-title">
          <span>➕ Register Product</span>
          <span style="font-size: 0.78rem; color: #818cf8;">MongoDB Atlas</span>
        </div>
        <form id="prodForm" onsubmit="saveProduct(event)">
          <div class="form-group">
            <label class="form-label">SKU Code *</label>
            <input type="text" id="sku" class="form-input" placeholder="e.g. ELC-880" required>
          </div>
          <div class="form-group">
            <label class="form-label">Product Name *</label>
            <input type="text" id="name" class="form-input" placeholder="e.g. Mechanical Keyboard"
required>
          </div>
          <div class="form-group">
            <label class="form-label">Category *</label>
            <select id="category" class="form-select">
              <option value="Electronics">Electronics</option>
              <option value="Computing">Computing</option>
              <option value="Accessories">Accessories</option>
              <option value="Furniture">Furniture</option>
              <option value="Office Supplies">Office Supplies</option>
            </select>
          </div>
          <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 0.75rem;">
            <div class="form-group">
              <label class="form-label">Quantity *</label>
              <input type="number" id="quantity" min="0" class="form-input" value="10" required>
            </div>
            <div class="form-group">
              <label class="form-label">Unit Price (₹) *</label>
              <input type="number" id="unitPrice" min="0" class="form-input" value="2499" required>
            </div>
          </div>
          <div class="form-group">
            <label class="form-label">Supplier</label>
            <input type="text" id="supplier" class="form-input" value="Prince Tech Supply">
          </div>
          <button type="submit" class="btn btn-primary" style="margin-top: 0.5rem;">Add to
Database</button>
        </form>
      </div>

      <!-- Right Column: Table & Filters -->
      <div class="card">
        <div class="card-title">
          <span>📦 Live MongoDB Inventory Records</span>
          <span id="recordCount" style="font-size: 0.8rem; color: #94a3b8;">Loading...</span>
        </div>

        <div style="display: flex; gap: 0.75rem; margin-bottom: 1rem; flex-wrap: wrap;">
          <input type="text" id="searchInput" class="form-input" style="flex: 1; min-width: 200px;"
placeholder="Search SKU, name, supplier..." oninput="fetchData()">
          <select id="categoryFilter" class="form-select" style="width: auto; min-width: 150px;"
onchange="fetchData()">
            <option value="All">All Categories</option>
            <option value="Electronics">Electronics</option>
            <option value="Computing">Computing</option>
            <option value="Accessories">Accessories</option>
            <option value="Furniture">Furniture</option>
            <option value="Office Supplies">Office Supplies</option>
          </select>
        </div>

        <div class="table-wrap">
```

```
          <table>
            <thead>
              <tr>
                <th>SKU & Name</th>
                <th>Category</th>
                <th>Status</th>
                <th>Qty</th>
                <th>Price</th>
                <th>Total Value</th>
                <th style="text-align: right;">Action</th>
              </tr>
            </thead>
            <tbody id="inventoryTable">
              <tr><td colspan="7" style="text-align: center; padding: 2rem;">Loading inventory...</td>
</tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

  <script>
    async function fetchData() {
      const search = document.getElementById('searchInput').value;
      const cat = document.getElementById('categoryFilter').value;
      let url = `/api/inventory?category=${cat}`;
      if (search) url += `&search=${encodeURIComponent(search)}`;

      try {
        const res = await fetch(url);
        const json = await res.json();
        const items = json.data || [];
        renderTable(items);
        document.getElementById('recordCount').textContent = `${items.length} records`;

        const statsRes = await fetch('/api/inventory/stats/overview');
        const statsJson = await statsRes.json();
        if (statsJson.stats) {
          document.getElementById('statItems').textContent = statsJson.stats.totalItems;
          document.getElementById('statVal').textContent = '₹' +
statsJson.stats.totalValuation.toLocaleString('en-IN');
          document.getElementById('statLow').textContent = statsJson.stats.lowStockCount;
          document.getElementById('statOut').textContent = statsJson.stats.outOfStockCount;
        }
      } catch (err) {
        console.error(err);
      }
    }

    function renderTable(items) {
      const tbody = document.getElementById('inventoryTable');
      if (items.length === 0) {
        tbody.innerHTML = '<tr><td colspan="7" style="text-align: center; padding: 2rem; color:
#94a3b8;">No items found.</td></tr>';
        return;
      }

      tbody.innerHTML = items.map(item => {
        let stClass = 'status-in';
        if (item.quantity === 0) stClass = 'status-out';
        else if (item.quantity <= (item.reorderLevel || 10)) stClass = 'status-low';

        return `
          <tr>
            <td>
              <div style="font-weight: 700; color: #fff;">${item.name}</div>
              <div style="font-size: 0.75rem; color: #94a3b8; font-family: monospace;">${item.sku} &bull;
${item.supplier || 'Direct'}</div>
            </td>
```

```
            <td><span style="font-size: 0.78rem; background: #334155; padding: 0.2rem 0.5rem; border-
radius: 4px;">${item.category}</span></td>
            <td><span class="badge-status ${stClass}">${item.stockStatus || 'In Stock'}</span></td>
            <td style="font-weight: 700;">${item.quantity}</td>
            <td>₹${Number(item.unitPrice).toLocaleString('en-IN')}</td>
            <td style="color: #34d399; font-weight: 600;">₹${(item.quantity *
item.unitPrice).toLocaleString('en-IN')}</td>
            <td style="text-align: right;">
              <button class="btn btn-del btn-sm" onclick="deleteItem('${item._id}', '${item.name}')">🗑
</button>
            </td>
          </tr>
        `;
      }).join('');
    }

    async function saveProduct(e) {
      e.preventDefault();
      const payload = {
        sku: document.getElementById('sku').value.trim(),
        name: document.getElementById('name').value.trim(),
        category: document.getElementById('category').value,
        quantity: parseInt(document.getElementById('quantity').value, 10),
        unitPrice: parseFloat(document.getElementById('unitPrice').value),
        supplier: document.getElementById('supplier').value.trim()
      };

      try {
        const res = await fetch('/api/inventory', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(payload)
        });
        const data = await res.json();
        if (res.ok) {
          alert('Item added successfully!');
          document.getElementById('prodForm').reset();
          fetchData();
        } else {
          alert(data.message || 'Error creating product');
        }
      } catch (err) {
        alert(err.message);
      }
    }

    async function deleteItem(id, name) {
      if (!confirm(`Delete "${name}" from inventory?`)) return;
      try {
        await fetch(`/api/inventory/${id}`, { method: 'DELETE' });
        fetchData();
      } catch (err) {
        alert('Delete failed: ' + err.message);
      }
    }

    window.addEventListener('DOMContentLoaded', fetchData);
  </script>
</body>
</html>
```

- **frontend/package.json :**

```
{
  "name": "mern-10-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
```

```
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.2"
  }
}
```

- **frontend/index.html :**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Practical 10 - Full Stack MERN Application</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;600&display=swap"
rel="stylesheet">
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

- **frontend/vite.config.js :**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3010,
    proxy: {
      '/api': {
        target: 'http://localhost:5010',
        changeOrigin: true
      }
    }
  }
});
```

- **frontend/src/main.jsx :**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

- **frontend/src/index.css :**

```css
:root {
  --primary: #4f46e5;
  --primary-hover: #4338ca;
  --primary-light: #e0e7ff;
  --bg-dark: #0f172a;
  --bg-card: #1e293b;
  --border: #334155;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --success: #10b981;
  --warning: #f59e0b;
  --danger: #ef4444;
}

* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: 'Plus Jakarta Sans', system-ui, sans-serif;
  background-color: var(--bg-dark);
  color: var(--text-main);
  line-height: 1.5;
  min-height: 100vh;
}

.container {
  max-width: 1250px;
  margin: 0 auto;
  padding: 1.75rem 1.5rem;
}

.header {
  background: linear-gradient(135deg, #1e1b4b 0%, #312e81 50%, #4338ca 100%);
  padding: 2.25rem 1.5rem;
  text-align: center;
  border-bottom: 1px solid #3730a3;
}
.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.18);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.8rem;
  font-weight: 700;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  margin-bottom: 0.6rem;
}
.header-title { font-size: 2.25rem; font-weight: 800; margin-bottom: 0.4rem; }
.header-subtitle { font-size: 1rem; color: #c7d2fe; }

/* Stats Bar */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.25rem;
  margin-bottom: 2rem;
}
.stat-card {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1.25rem;
  display: flex;
  align-items: center;
  gap: 1rem;
}
.stat-icon {
  width: 48px;
  height: 48px;
  border-radius: 10px;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  font-size: 1.5rem;
}
.stat-val { font-size: 1.6rem; font-weight: 800; color: #fff; }
.stat-lbl { font-size: 0.82rem; color: var(--text-muted); }

/* Layout Grid */
.main-layout {
  display: grid;
  grid-template-columns: 380px 1fr;
  gap: 1.75rem;
  align-items: start;
}
@media (max-width: 992px) {
  .main-layout { grid-template-columns: 1fr; }
}

.card {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 1.5rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.2);
}
.card-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1.25rem;
  border-bottom: 1px solid var(--border);
  padding-bottom: 0.75rem;
}
.card-title { font-size: 1.15rem; font-weight: 700; color: #fff; }

/* Form Controls */
.form-group { margin-bottom: 1rem; }
.form-label { display: block; font-size: 0.8rem; font-weight: 600; color: var(--text-muted); margin-
bottom: 0.35rem; }
.form-input, .form-select {
  width: 100%;
  background: #0f172a;
  border: 1px solid var(--border);
  color: #fff;
  padding: 0.6rem 0.85rem;
  border-radius: 8px;
  font-size: 0.88rem;
  outline: none;
  font-family: inherit;
}
.form-input:focus, .form-select:focus { border-color: var(--primary); }

.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  font-family: inherit;
  font-size: 0.88rem;
  font-weight: 600;
  padding: 0.65rem 1.25rem;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-primary { background: var(--primary); color: white; width: 100%; }
.btn-primary:hover { background: var(--primary-hover); }
.btn-secondary { background: #334155; color: white; }
.btn-sm { padding: 0.35rem 0.75rem; font-size: 0.78rem; }
```

```
.btn-danger { background: rgba(239, 68, 68, 0.2); color: #fca5a5; border: 1px solid #ef4444; }
.btn-danger:hover { background: #ef4444; color: white; }

/* Table */
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; font-size: 0.86rem; }
th { background: #0f172a; color: var(--text-muted); text-align: left; padding: 0.75rem 0.85rem; font-
weight: 600; border-bottom: 1px solid var(--border); }
td { padding: 0.75rem 0.85rem; border-bottom: 1px solid rgba(51, 65, 85, 0.5); vertical-align: middle; }
tr:hover { background: rgba(255, 255, 255, 0.02); }

.badge-status {
  padding: 0.2rem 0.6rem;
  border-radius: 9999px;
  font-size: 0.72rem;
  font-weight: 700;
}
.status-in { background: rgba(16, 185, 129, 0.2); color: #6ee7b7; border: 1px solid #10b981; }
.status-low { background: rgba(245, 158, 11, 0.2); color: #fde68a; border: 1px solid #f59e0b; }
.status-out { background: rgba(239, 68, 68, 0.2); color: #fca5a5; border: 1px solid #ef4444; }

.search-bar-row {
  display: flex;
  gap: 0.75rem;
  margin-bottom: 1.25rem;
  flex-wrap: wrap;
}
```

- **frontend/src/App.jsx :**

```
import React, { useState, useEffect } from 'react';
import StatsCard from './components/StatsCard';
import InventoryForm from './components/InventoryForm';
import InventoryTable from './components/InventoryTable';

const API_BASE = '/api/inventory';

function App() {
  const [items, setItems] = useState([]);
  const [stats, setStats] = useState({});
  const [editingItem, setEditingItem] = useState(null);
  const [search, setSearch] = useState('');
  const [categoryFilter, setCategoryFilter] = useState('All');
  const [statusFilter, setStatusFilter] = useState('All');
  const [alert, setAlert] = useState(null);

  const showAlert = (message, type = 'success') => {
    setAlert({ message, type });
    setTimeout(() => setAlert(null), 3500);
  };

  const fetchInventory = async () => {
    try {
      let url = `${API_BASE}?category=${categoryFilter}&stockStatus=${statusFilter}`;
      if (search) url += `&search=${encodeURIComponent(search)}`;
      const res = await fetch(url);
      const json = await res.json();
      if (json.success) setItems(json.data);
    } catch (err) {
      console.error('Fetch error:', err);
    }
  };

  const fetchStats = async () => {
    try {
      const res = await fetch(`${API_BASE}/stats/overview`);
      const json = await res.json();
      if (json.success) setStats(json.stats);
    } catch (err) {
```

```
      console.error('Stats error:', err);
    }
  };

  useEffect(() => {
    fetchInventory();
    fetchStats();
  }, [categoryFilter, statusFilter, search]);

  const handleSave = async (itemData) => {
    try {
      let res;
      if (editingItem) {
        res = await fetch(`${API_BASE}/${editingItem._id}`, {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(itemData)
        });
      } else {
        res = await fetch(API_BASE, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(itemData)
        });
      }
      const json = await res.json();
      if (res.ok) {
        showAlert(json.message);
        setEditingItem(null);
        fetchInventory();
        fetchStats();
      } else {
        showAlert(json.message || 'Operation failed', 'danger');
      }
    } catch (err) {
      showAlert(`Network error: ${err.message}`, 'danger');
    }
  };

  const handleDelete = async (id, name) => {
    if (!window.confirm(`Delete "${name}" from inventory?`)) return;
    try {
      const res = await fetch(`${API_BASE}/${id}`, { method: 'DELETE' });
      const json = await res.json();
      if (res.ok) {
        showAlert(json.message);
        fetchInventory();
        fetchStats();
      } else {
        showAlert(json.message, 'danger');
      }
    } catch (err) {
      showAlert(`Delete failed: ${err.message}`, 'danger');
    }
  };

  return (
    <div>
      <header className="header">
        <span className="header-badge">PRACTICAL NO. 10</span>
        <h1 className="header-title">Full-Stack MERN Inventory Application</h1>
        <p className="header-subtitle">
          React Client (Vite) &bull; Express.js Backend &bull; MongoDB Atlas Cloud Database &bull;
Node.js Architecture
        </p>
      </header>

      <main className="container">
        {alert && (
          <div
```

```
            style={{
              padding: '0.85rem 1.25rem',
              borderRadius: '8px',
              marginBottom: '1.5rem',
              fontWeight: 600,
              fontSize: '0.9rem',
              background: alert.type === 'danger' ? 'rgba(239, 68, 68, 0.2)' : 'rgba(16, 185, 129, 0.2)',
              border: `1px solid ${alert.type === 'danger' ? '#ef4444' : '#10b981'}`,
              color: alert.type === 'danger' ? '#fca5a5' : '#6ee7b7'
            }}
          >
            {alert.message}
          </div>
        )}

        <StatsCard stats={stats} />

        <div className="main-layout">
          <InventoryForm
            onSave={handleSave}
            editingItem={editingItem}
            onCancel={() => setEditingItem(null)}
          />

          <div className="card">
            <div className="card-header">
              <h3 className="card-title">📦 Real-time MongoDB Inventory Records</h3>
              <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
                Showing {items.length} records
              </span>
            </div>

            <div className="search-bar-row">
              <input
                type="text"
                placeholder="🔍 Search SKU, item name, supplier..."
                className="form-input"
                style={{ flex: 1, minWidth: '220px' }}
                value={search}
                onChange={(e) => setSearch(e.target.value)}
              />
              <select
                className="form-select"
                style={{ width: 'auto', minWidth: '150px' }}
                value={categoryFilter}
                onChange={(e) => setCategoryFilter(e.target.value)}
              >
                <option value="All">All Categories</option>
                <option value="Electronics">Electronics</option>
                <option value="Computing">Computing</option>
                <option value="Accessories">Accessories</option>
                <option value="Furniture">Furniture</option>
                <option value="Office Supplies">Office Supplies</option>
              </select>
              <select
                className="form-select"
                style={{ width: 'auto', minWidth: '140px' }}
                value={statusFilter}
                onChange={(e) => setStatusFilter(e.target.value)}
              >
                <option value="All">All Statuses</option>
                <option value="In Stock">In Stock</option>
                <option value="Low Stock">Low Stock</option>
                <option value="Out of Stock">Out of Stock</option>
              </select>
            </div>

            <InventoryTable
              items={items}
              onEdit={(item) => setEditingItem(item)}
```

```
              onDelete={handleDelete}
            />
          </div>
        </div>
      </main>
    </div>
  );
}

export default App;
```

- **frontend/src/components/StatsCard.jsx:**

```
import React from 'react';

function StatsCard({ stats }) {
  const cards = [
    {
      title: 'Total Products',
      val: stats.totalItems || 0,
      icon: '📦',
      bg: 'rgba(79, 70, 229, 0.15)',
      color: '#818cf8'
    },
    {
      title: 'Total Valuation',
      val: `₹${(stats.totalValuation || 0).toLocaleString('en-IN')}`,
      icon: '💰',
      bg: 'rgba(16, 185, 129, 0.15)',
      color: '#34d399'
    },
    {
      title: 'Low Stock Items',
      val: stats.lowStockCount || 0,
      icon: '⚠️',
      bg: 'rgba(245, 158, 11, 0.15)',
      color: '#fbbf24'
    },
    {
      title: 'Out of Stock',
      val: stats.outOfStockCount || 0,
      icon: '🚫',
      bg: 'rgba(239, 68, 68, 0.15)',
      color: '#f87171'
    }
  ];

  return (
    <div className="stats-grid">
      {cards.map((c, i) => (
        <div key={i} className="stat-card">
          <div className="stat-icon" style={{ background: c.bg, color: c.color }}>
            {c.icon}
          </div>
          <div>
            <div className="stat-val">{c.val}</div>
            <div className="stat-lbl">{c.title}</div>
          </div>
        </div>
      ))}
    </div>
  );
}

export default StatsCard;
```

- **frontend/src/components/InventoryForm.jsx:**

```jsx
import React, { useState, useEffect } from 'react';

const initialForm = {
  sku: '',
  name: '',
  category: 'Electronics',
  quantity: '',
  unitPrice: '',
  supplier: 'Prince Tech Logistics',
  reorderLevel: 5
};

function InventoryForm({ onSave, editingItem, onCancel }) {
  const [formData, setFormData] = useState(initialForm);

  useEffect(() => {
    if (editingItem) {
      setFormData(editingItem);
    } else {
      setFormData(initialForm);
    }
  }, [editingItem]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!formData.sku || !formData.name || formData.quantity === '' || formData.unitPrice === '') {
      alert('Please fill out all mandatory fields (SKU, Name, Quantity, Price).');
      return;
    }

    onSave({
      ...formData,
      quantity: Number(formData.quantity),
      unitPrice: Number(formData.unitPrice),
      reorderLevel: Number(formData.reorderLevel || 5)
    });

    if (!editingItem) {
      setFormData(initialForm);
    }
  };

  return (
    <div className="card">
      <div className="card-header">
        <h3 className="card-title">{editingItem ? '🖍️ Edit Product' : '➕ Register Product'}</h3>
        {editingItem && (
          <button type="button" className="btn btn-secondary btn-sm" onClick={onCancel}>
            Cancel
          </button>
        )}
      </div>

      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label className="form-label">SKU Code *</label>
          <input
            type="text"
            name="sku"
            className="form-input"
            placeholder="e.g. ELC-901"
            value={formData.sku}
            onChange={handleChange}
            disabled={!!editingItem}
            required
```

```
            />
          </div>

          <div className="form-group">
            <label className="form-label">Product Name *</label>
            <input
              type="text"
              name="name"
              className="form-input"
              placeholder="e.g. Wireless Noise Canceling Headphones"
              value={formData.name}
              onChange={handleChange}
              required
            />
          </div>

          <div className="form-group">
            <label className="form-label">Category *</label>
            <select name="category" className="form-select" value={formData.category} onChange=
{handleChange}>
              <option value="Electronics">Electronics</option>
              <option value="Computing">Computing</option>
              <option value="Accessories">Accessories</option>
              <option value="Furniture">Furniture</option>
              <option value="Office Supplies">Office Supplies</option>
            </select>
          </div>

          <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
            <div className="form-group">
              <label className="form-label">Quantity *</label>
              <input
                type="number"
                name="quantity"
                min="0"
                className="form-input"
                value={formData.quantity}
                onChange={handleChange}
                required
              />
            </div>
            <div className="form-group">
              <label className="form-label">Unit Price (₹) *</label>
              <input
                type="number"
                name="unitPrice"
                min="0"
                className="form-input"
                value={formData.unitPrice}
                onChange={handleChange}
                required
              />
            </div>
          </div>

          <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
            <div className="form-group">
              <label className="form-label">Supplier</label>
              <input
                type="text"
                name="supplier"
                className="form-input"
                value={formData.supplier}
                onChange={handleChange}
              />
            </div>
            <div className="form-group">
              <label className="form-label">Reorder Alert Qty</label>
              <input
                type="number"
```

```
              name="reorderLevel"
              className="form-input"
              value={formData.reorderLevel}
              onChange={handleChange}
            />
          </div>
        </div>

        <button type="submit" className="btn btn-primary" style={{ marginTop: '0.5rem' }}>
          {editingItem ? 'Save Updates' : 'Add to MongoDB Atlas'}
        </button>
      </form>
    </div>
  );
}

export default InventoryForm;
```

• **frontend/src/components/InventoryTable.jsx :**

```
import React from 'react';

function InventoryTable({ items, onEdit, onDelete }) {
  if (!items || items.length === 0) {
    return (
      <div style={{ textAlign: 'center', padding: '3rem 1rem', color: 'var(--text-muted)' }}>
        <p style={{ fontSize: '2rem', marginBottom: '0.5rem' }}>📦</p>
        <h4>No inventory items found.</h4>
        <p style={{ fontSize: '0.85rem' }}>Register a new product using the form or clear search filters.
</p>
      </div>
    );
  }

  return (
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>SKU & Name</th>
            <th>Category</th>
            <th>Stock Status</th>
            <th>Qty</th>
            <th>Price</th>
            <th>Total Value</th>
            <th style="text-align: right;">Actions</th>
          </tr>
        </thead>
        <tbody>
          {items.map((item) => {
            let statusClass = 'status-in';
            if (item.quantity === 0) statusClass = 'status-out';
            else if (item.quantity <= item.reorderLevel) statusClass = 'status-low';

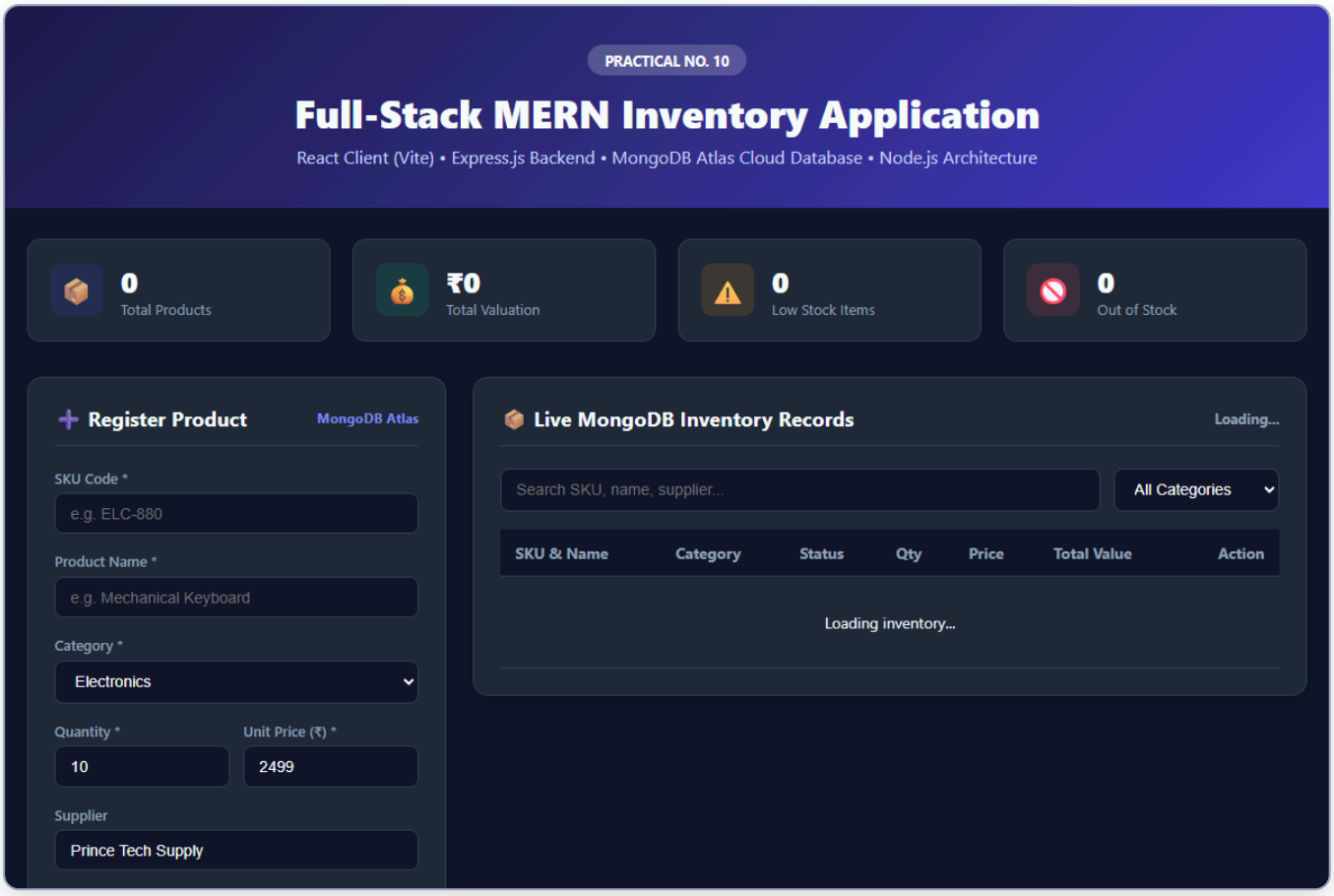
            return (
              <tr key={item._id || item.sku}>
                <td>
                  <div style={{ fontWeight: 700, color: '#fff' }}>{item.name}</div>
                  <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontFamily: 'monospace'
}}>
                    {item.sku} &bull; {item.supplier}
                  </div>
                </td>
                <td>
                  <span style={{ fontSize: '0.8rem', background: '#334155', padding: '0.2rem 0.5rem',
borderRadius: '4px' }}>
                    {item.category}
                  </span>
                </td>
```

```
                <td>
                  <span className={`badge-status ${statusClass}`}>{item.stockStatus}</span>
                </td>
                <td style={{ fontWeight: 700 }}>{item.quantity}</td>
                <td>₹{Number(item.unitPrice).toLocaleString('en-IN')}</td>
                <td style={{ color: '#34d399', fontWeight: 600 }}>
                  ₹{(item.quantity * item.unitPrice).toLocaleString('en-IN')}
                </td>
                <td style={{ textAlign: 'right', whiteSpace: 'nowrap' }}>
                  <button
                    className="btn btn-secondary btn-sm"
                    style={{ marginRight: '0.4rem' }}
                    onClick={() => onEdit(item)}
                  >
                    ✏️
                  </button>
                  <button className="btn btn-danger btn-sm" onClick={() => onDelete(item._id,
item.name)}>
                    🗑️
                  </button>
                </td>
              </tr>
            );
          })}
        </tbody>
      </table>
    </div>
  );
}

export default InventoryTable;
```

4. Output :

Complete end-to-end full stack MERN inventory application connecting React Vite frontend with Express REST backend and cloud-hosted MongoDB Atlas database.



Practical – 11 : Create a Blog Management System implementing CRUD Operations, Authentication and Role-Based Access Control.

1. Commands :

```
mkdir MERN_11 && cd MERN_11
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors dotenv morgan
node seed.js
node server.js
```

2. Project structure :

```
MERN_11/
|-- .env
|-- package.json
|-- server.js
|-- seed.js
|-- config/db.js
|-- models/User.js
|-- models/Post.js
|-- middleware/auth.js
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5011
NODE_ENV=development
MONGO_URI=mongodb+srv://prince:prince18@cluster0.76xckkx.mongodb.net/mern11_blog_db?
retryWrites=true&w=majority
JWT_SECRET=prince_jwt_super_secret_key_2026_rbac
JWT_EXPIRE=30d
```

- **package.json :**

```
{
  "name": "mern-11",
  "version": "1.0.0",
  "description": "Practical 11: Blog Management System implementing CRUD Operations, JWT Authentication
and Role-Based Access Control (RBAC)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "seed": "node seed.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.6.1",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const jwt = require('jsonwebtoken');
const connectDB = require('./config/db');
const User = require('./models/User');
const Post = require('./models/Post');
const { protect, authorize } = require('./middleware/auth');

const app = express();
const PORT = process.env.PORT || 5011;

connectDB();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

const generateToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET || 'secret123', { expiresIn: '7d'
});

// Auth: Register
app.post('/api/auth/register', async (req, res) => {
  try {
    const { name, email, password, role } = req.body;
    let user = await User.findOne({ email });
    if (user) return res.status(400).json({ success: false, message: 'User already exists' });

    user = await User.create({ name, email, password, role: role || 'reader' });
    res.status(201).json({
      success: true,
      token: generateToken(user._id),
      user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});

// Auth: Login
app.post('/api/auth/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (user && (await user.matchPassword(password))) {
      res.json({
        success: true,
        token: generateToken(user._id),
        user: { id: user._id, name: user.name, email: user.email, role: user.role }
      });
    } else {
      res.status(401).json({ success: false, message: 'Invalid email or password' });
    }
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Posts: Get All (Public for Readers/Guests)
app.get('/api/posts', async (req, res) => {
  try {
    const posts = await Post.find().sort({ createdAt: -1 });
    res.json({ success: true, count: posts.length, data: posts });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Posts: Create (Author & Admin only)
app.post('/api/posts', protect, authorize('admin', 'author'), async (req, res) => {
```

```
  try {
    const { title, content, category } = req.body;
    const post = await Post.create({
      title,
      content,
      category,
      author: req.user._id,
      authorName: req.user.name
    });
    res.status(201).json({ success: true, data: post });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});

// Posts: Update (Admin OR Author of the post)
app.put('/api/posts/:id', protect, authorize('admin', 'author'), async (req, res) => {
  try {
    let post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ success: false, message: 'Post not found' });

    if (post.author.toString() !== req.user._id.toString() && req.user.role !== 'admin') {
      return res.status(403).json({ success: false, message: 'Not authorized to edit this post' });
    }

    post = await Post.findByIdAndUpdate(req.params.id, req.body, { new: true });
    res.json({ success: true, data: post });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});

// Posts: Delete (Admin OR Author of the post)
app.delete('/api/posts/:id', protect, authorize('admin', 'author'), async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ success: false, message: 'Post not found' });

    if (post.author.toString() !== req.user._id.toString() && req.user.role !== 'admin') {
      return res.status(403).json({ success: false, message: 'Not authorized to delete this post' });
    }

    await Post.findByIdAndDelete(req.params.id);
    res.json({ success: true, message: 'Post deleted successfully' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

app.listen(PORT, () => console.log(`⚛️ Practical 11 (Blog & RBAC) running on port ${PORT}`));
module.exports = app;
```

- **seed.js :**

```
require('dotenv').config();
const mongoose = require('mongoose');
const User = require('./models/User');
const Post = require('./models/Post');

const seed = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  await User.deleteMany({});
  await Post.deleteMany({});

  const admin = await User.create({ name: 'Prince Prajapati', email: 'prince@admin.com', password:
'password123', role: 'admin' });
  const author = await User.create({ name: 'Ananya Sharma', email: 'ananya@author.com', password:
'password123', role: 'author' });
  const reader = await User.create({ name: 'Rahul Verma', email: 'rahul@reader.com', password:
```

```
'password123', role: 'reader' });

  await Post.create([
    { title: 'Mastering Full-Stack MERN Architecture', content: 'Comprehensive guide to building scalable
React and Node.js microservices with MongoDB Atlas.', category: 'Tech', author: admin._id, authorName:
admin.name },
    { title: 'Role-Based Access Control in Express', content: 'Understanding JWT tokens, authorization
middleware, and granular permissions.', category: 'Security', author: author._id, authorName: author.name
}
  ]);

  console.log('✅ Practical 11 seeded successfully with Admin, Author & Reader accounts.');
  process.exit(0);
};

seed();
```

- **config/db.js :**

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
  } catch (err) {
    console.error(`❌ DB Error: ${err.message}`);
  }
};

module.exports = connectDB;
```

- **models/User.js :**

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['admin', 'author', 'reader'], default: 'reader' }
});

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.matchPassword = function (enteredPassword) {
  return bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

- **models/Post.js :**

```
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  category: { type: String, default: 'General' },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  authorName: { type: String }
}, { timestamps: true });
```

```
module.exports = mongoose.model('Post', postSchema);
```

• **middleware/auth.js :**

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ success: false, message: 'Not authorized, no token' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret123');
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    res.status(401).json({ success: false, message: 'Token invalid' });
  }
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: `Role '${req.user.role}' not permitted.` });
  }
  next();
};

module.exports = { protect, authorize };
```

• **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 11 - Blog Management with RBAC</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: system-ui, sans-serif; }
    body { background: #0f172a; color: #f8fafc; padding-bottom: 2rem; }
    .header { background: linear-gradient(135deg, #1e1b4b, #4338ca); padding: 2rem; text-align: center; }
    .badge { display: inline-block; background: rgba(255,255,255,0.2); padding: 0.3rem 0.8rem; border-
radius: 999px; font-size: 0.8rem; font-weight: bold; }
    .container { max-width: 1000px; margin: 1.5rem auto; padding: 0 1rem; }
    .role-bar { background: #1e293b; padding: 1rem; border-radius: 10px; display: flex; justify-content:
space-between; align-items: center; margin-bottom: 1.5rem; }
    .btn { background: #4f46e5; color: white; border: none; padding: 0.5rem 1rem; border-radius: 6px;
cursor: pointer; font-weight: 600; font-size: 0.85rem; }
    .btn-sm { padding: 0.3rem 0.6rem; font-size: 0.78rem; }
    .btn-del { background: #dc2626; }
    .card { background: #1e293b; border-radius: 10px; padding: 1.25rem; margin-bottom: 1rem; border: 1px
solid #334155; }
    input, textarea, select { width: 100%; background: #0f172a; border: 1px solid #334155; color: white;
padding: 0.6rem; border-radius: 6px; margin-bottom: 0.75rem; }
    .post-header { display: flex; justify-content: space-between; align-items: flex-start; margin-bottom:
0.5rem; }
    .post-meta { font-size: 0.8rem; color: #94a3b8; }
    .role-tag { font-size: 0.75rem; padding: 0.2rem 0.5rem; border-radius: 4px; font-weight: bold; text-
transform: uppercase; }
    .role-admin { background: #dc2626; color: white; }
    .role-author { background: #d97706; color: white; }
    .role-reader { background: #2563eb; color: white; }
  </style>
</head>
<body>
  <div class="header">
    <span class="badge">PRACTICAL NO. 11</span>
    <h1 style="margin: 0.5rem 0;">Blog Management with Authentication & RBAC</h1>
```

```
    <p style="color: #c7d2fe; font-size: 0.95rem;">Role-Based Access Control: Admin, Author, and Reader
with JWT Authentication</p>
  </div>

  <div class="container">
    <div class="role-bar">
      <div>
        <span style="color: #94a3b8; font-size: 0.85rem;">Active Identity:</span>
        <strong id="userName" style="color: #38bdf8;">Guest</strong>
        <span id="userRole" class="role-tag role-reader">READER</span>
      </div>
      <div style="display: flex; gap: 0.5rem;">
        <button class="btn btn-sm" onclick="loginDemo('prince@admin.com', 'admin')">Login as
Admin</button>
        <button class="btn btn-sm" style="background: #d97706;" onclick="loginDemo('ananya@author.com',
'author')">Login as Author</button>
        <button class="btn btn-sm" style="background: #2563eb;" onclick="loginDemo('rahul@reader.com',
'reader')">Login as Reader</button>
      </div>
    </div>

    <!-- Create Post (Admin & Author only) -->
    <div class="card" id="composeCard">
      <h3 style="margin-bottom: 0.75rem;">✍️ Create New Article</h3>
      <p id="composeNotice" style="color: #f87171; font-size: 0.85rem; margin-bottom: 0.5rem; display:
none;">
        ⚠️ Readers have read-only access. Switch to Author or Admin above to publish articles.
      </p>
      <input type="text" id="postTitle" placeholder="Article Title...">
      <input type="text" id="postCategory" placeholder="Category (e.g. Tech, Career)..."
value="Engineering">
      <textarea id="postContent" rows="3" placeholder="Write article content here..."></textarea>
      <button class="btn" id="publishBtn" onclick="createPost()">Publish Article</button>
    </div>

    <!-- Posts Feed -->
    <h3 style="margin: 1.5rem 0 0.75rem;">📚 Published Articles Feed</h3>
    <div id="postsList">Loading articles...</div>
  </div>

  <script>
    let token = '';
    let currentUser = { role: 'reader', name: 'Guest' };

    async function loginDemo(email, expectedRole) {
      try {
        const res = await fetch('/api/auth/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password: 'password123' })
        });
        const data = await res.json();
        if (data.success) {
          token = data.token;
          currentUser = data.user;
          updateUI();
          loadPosts();
        }
      } catch (e) {
        alert('Login failed: ' + e.message);
      }
    }

    function updateUI() {
      document.getElementById('userName').textContent = currentUser.name;
      const roleTag = document.getElementById('userRole');
      roleTag.textContent = currentUser.role.toUpperCase();
      roleTag.className = `role-tag role-${currentUser.role}`;

      const isAllowed = currentUser.role === 'admin' || currentUser.role === 'author';
```

```
      document.getElementById('publishBtn').disabled = !isAllowed;
      document.getElementById('composeNotice').style.display = isAllowed ? 'none' : 'block';
    }

    async function loadPosts() {
      const res = await fetch('/api/posts');
      const json = await res.json();
      const list = document.getElementById('postsList');

      if (!json.data || json.data.length === 0) {
        list.innerHTML = '<p style="color: #94a3b8;">No articles published yet.</p>';
        return;
      }

      list.innerHTML = json.data.map(p => {
        const canDelete = currentUser.role === 'admin' || (currentUser.role === 'author' && p.author ===
currentUser.id);
        return `
          <div class="card">
            <div class="post-header">
              <div>
                <h4>${p.title}</h4>
                <div class="post-meta">By ${p.authorName || 'Author'} &bull; Category: ${p.category}
&bull; ${new Date(p.createdAt).toLocaleDateString()}</div>
              </div>
              ${canDelete ? `<button class="btn btn-sm btn-del"
onclick="deletePost('${p._id}')">Delete</button>` : ''}
            </div>
            <p style="font-size: 0.9rem; color: #cbd5e1; margin-top: 0.5rem;">${p.content}</p>
          </div>
        `;
      }).join('');
    }

    async function createPost() {
      const title = document.getElementById('postTitle').value.trim();
      const content = document.getElementById('postContent').value.trim();
      const category = document.getElementById('postCategory').value.trim();

      if (!title || !content) return alert('Title and content are required');

      try {
        const res = await fetch('/api/posts', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${token}`
          },
          body: JSON.stringify({ title, content, category })
        });
        const data = await res.json();
        if (data.success) {
          document.getElementById('postTitle').value = '';
          document.getElementById('postContent').value = '';
          loadPosts();
        } else {
          alert(data.message);
        }
      } catch (e) {
        alert(e.message);
      }
    }

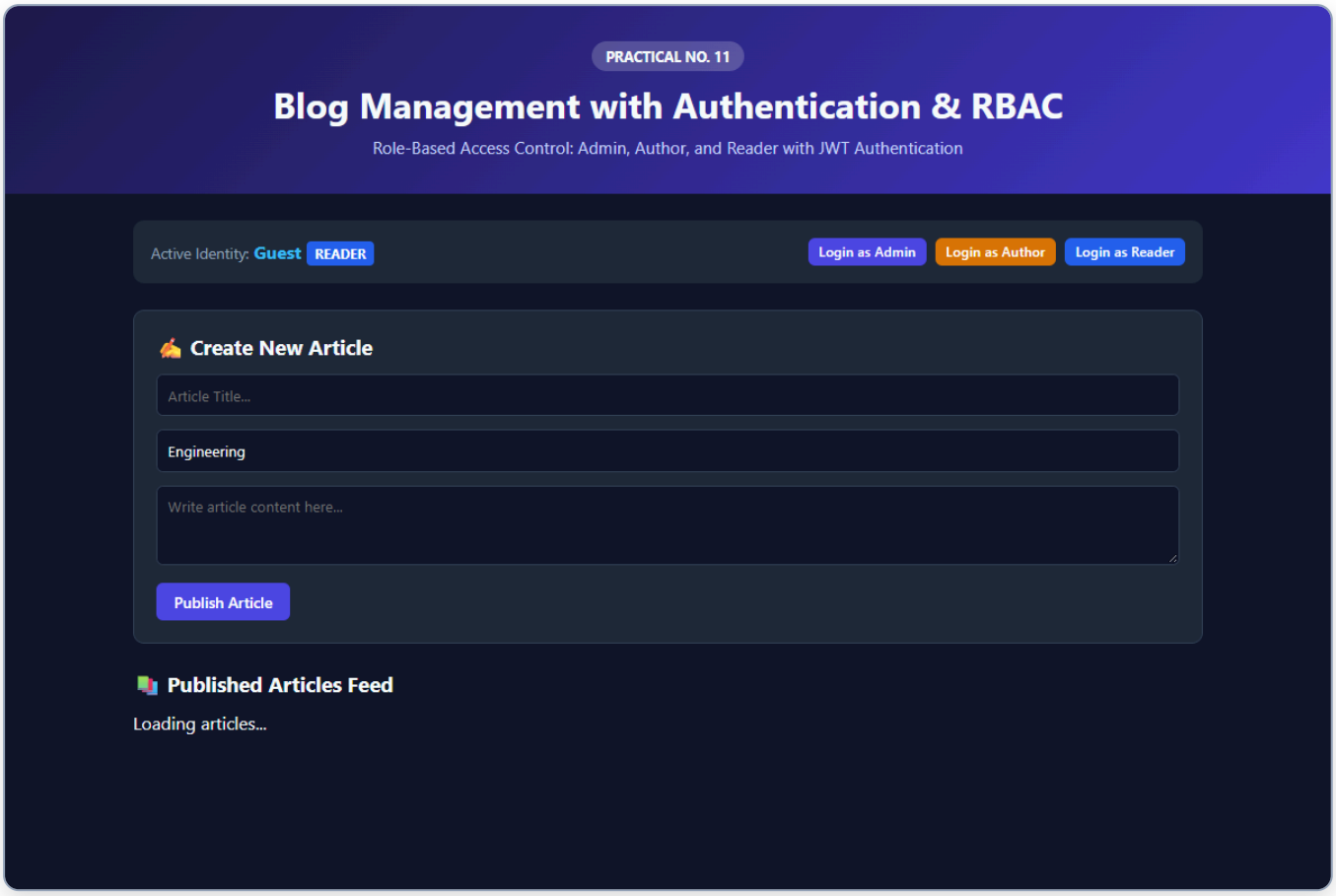
    async function deletePost(id) {
      if (!confirm('Are you sure you want to delete this article?')) return;
      try {
        const res = await fetch(`/api/posts/${id}`, {
          method: 'DELETE',
          headers: { 'Authorization': `Bearer ${token}` }
        });
```

```
        const data = await res.json();
        if (data.success) loadPosts();
        else alert(data.message);
      } catch (e) {
        alert(e.message);
      }
    }

    // Default auto-login as Admin to see full experience
    window.addEventListener('DOMContentLoaded', () => {
      loginDemo('prince@admin.com', 'admin');
    });
  </script>
</body>
</html>
```

4. Output :

Full-featured blog management platform featuring JWT authentication, Bcrypt password hashing, and granular Role-Based Access Control for Admin, Author, and Reader roles.



Practical – 12 : Develop a Real-Time Chat Application using Socket.IO, React and Node.js.

1. Commands :

```
mkdir MERN_12 && cd MERN_12
npm init -y
npm install express socket.io cors dotenv
node server.js
```

2. Project structure :

```
MERN_12/
|-- .env
|-- package.json
|-- server.js
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5012
```

- **package.json :**

```
{
  "name": "mern-12",
  "version": "1.0.0",
  "description": "Practical 12: Real-Time Chat Application using Socket.IO and Node.js",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "socket.io": "^4.7.5"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const path = require('path');
const cors = require('cors');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });
const PORT = process.env.PORT || 5012;

app.use(cors());
app.use(express.static(path.join(__dirname, 'public')));

// In-memory active message store for rooms
const roomMessages = {};

io.on('connection', (socket) => {
```

```
  console.log(`⚡ User connected: ${socket.id}`);

  socket.on('join_room', ({ username, room }) => {
    socket.join(room);
    socket.data = { username, room };

    // Send existing room history
    if (!roomMessages[room]) roomMessages[room] = [];
    socket.emit('room_history', roomMessages[room]);

    // Notify room members
    socket.to(room).emit('system_message', {
      text: `${username} joined the chat`,
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
    });
  });

  socket.on('send_message', (data) => {
    const msg = {
      id: Date.now(),
      sender: data.username,
      text: data.text,
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
    };
    if (roomMessages[data.room]) roomMessages[data.room].push(msg);
    io.to(data.room).emit('receive_message', msg);
  });

  socket.on('typing', ({ username, room }) => {
    socket.to(room).emit('user_typing', username);
  });

  socket.on('disconnect', () => {
    if (socket.data?.room) {
      socket.to(socket.data.room).emit('system_message', {
        text: `${socket.data.username} left the chat`,
        time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
      });
    }
  });
});

server.listen(PORT, () => console.log(`⚛️ Practical 12 (Socket.IO Chat) running on
http://localhost:${PORT}`));
```

• **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 12 - Real-Time Chat using Socket.IO</title>
  <script src="/socket.io/socket.io.js"></script>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: system-ui, sans-serif; }
    body { background: #0f172a; color: #f8fafc; height: 100vh; display: flex; flex-direction: column; }
    .header { background: linear-gradient(135deg, #0284c7, #2563eb); padding: 1.25rem; text-align:
center; }
    .badge { display: inline-block; background: rgba(255,255,255,0.2); padding: 0.2rem 0.6rem; border-
radius: 999px; font-size: 0.75rem; font-weight: bold; }

    .chat-layout { flex: 1; display: grid; grid-template-columns: 260px 1fr; max-width: 1100px; width:
100%; margin: 1rem auto; background: #1e293b; border-radius: 12px; overflow: hidden; border: 1px solid
#334155; }

    .sidebar { background: #0f172a; padding: 1rem; border-right: 1px solid #334155; }
    .room-btn { width: 100%; text-align: left; background: transparent; border: none; color: #94a3b8;
padding: 0.6rem 0.8rem; border-radius: 6px; cursor: pointer; font-size: 0.9rem; margin-bottom: 0.3rem; }
    .room-btn.active, .room-btn:hover { background: #334155; color: #38bdf8; font-weight: bold; }
```

```
    .chat-main { display: flex; flex-direction: column; height: 100%; }
    .chat-header { padding: 1rem; background: #1e293b; border-bottom: 1px solid #334155; font-weight:
bold; display: flex; justify-content: space-between; }
    .messages-area { flex: 1; padding: 1rem; overflow-y: auto; display: flex; flex-direction: column;
gap: 0.6rem; }

    .msg { max-width: 75%; padding: 0.6rem 0.9rem; border-radius: 8px; font-size: 0.9rem; line-height:
1.4; }
    .msg-incoming { background: #334155; align-self: flex-start; }
    .msg-mine { background: #2563eb; color: white; align-self: flex-end; }
    .msg-sender { font-size: 0.72rem; opacity: 0.75; margin-bottom: 0.2rem; }
    .msg-system { align-self: center; font-size: 0.75rem; color: #94a3b8; font-style: italic; background:
rgba(255,255,255,0.05); padding: 0.2rem 0.6rem; border-radius: 4px; }

    .typing-box { font-size: 0.75rem; color: #38bdf8; height: 1.2rem; padding: 0 1rem; }

    .input-bar { display: flex; padding: 0.75rem 1rem; background: #0f172a; border-top: 1px solid
#334155; gap: 0.5rem; }
    .input-bar input { flex: 1; background: #1e293b; border: 1px solid #334155; color: white; padding:
0.6rem 0.9rem; border-radius: 6px; outline: none; }
    .input-bar button { background: #2563eb; color: white; border: none; padding: 0.6rem 1.2rem; border-
radius: 6px; font-weight: bold; cursor: pointer; }
  </style>
</head>
<body>
  <div class="header">
    <span class="badge">PRACTICAL NO. 12</span>
    <h2 style="margin-top: 0.25rem;">Real-Time Chat Application with Socket.IO</h2>
  </div>

  <div class="chat-layout">
    <div class="sidebar">
      <div style="margin-bottom: 1rem;">
        <label style="font-size: 0.75rem; color: #94a3b8;">Your Display Name:</label>
        <input type="text" id="usernameInput" value="Prince" style="width: 100%; background: #1e293b;
border: 1px solid #334155; color: white; padding: 0.4rem; border-radius: 4px; margin-top: 0.25rem;">
      </div>
      <div style="font-size: 0.75rem; color: #94a3b8; margin-bottom: 0.5rem; font-weight:
bold;">ROOMS</div>
      <button class="room-btn active" onclick="switchRoom('general')"># general</button>
      <button class="room-btn" onclick="switchRoom('mern-stack')"># mern-stack</button>
      <button class="room-btn" onclick="switchRoom('react-dev')"># react-dev</button>
    </div>

    <div class="chat-main">
      <div class="chat-header">
        <span id="currentRoomTitle"># general</span>
        <span style="font-size: 0.8rem; color: #10b981;">● Connected</span>
      </div>

      <div class="messages-area" id="messagesArea"></div>
      <div class="typing-box" id="typingIndicator"></div>

      <form class="input-bar" onsubmit="sendMessage(event)">
        <input type="text" id="msgInput" placeholder="Type a message..." oninput="emitTyping()"
autocomplete="off">
        <button type="submit">Send</button>
      </form>
    </div>
  </div>

  <script>
    const socket = io();
    let currentRoom = 'general';
    let typingTimeout;

    function getUsername() {
      return document.getElementById('usernameInput').value.trim() || 'Anonymous';
    }
```

```
    function switchRoom(room) {
      currentRoom = room;
      document.getElementById('currentRoomTitle').textContent = '#' + room;
      document.querySelectorAll('.room-btn').forEach(btn => {
        btn.classList.toggle('active', btn.textContent.includes(room));
      });
      document.getElementById('messagesArea').innerHTML = '';
      socket.emit('join_room', { username: getUsername(), room });
    }

    function sendMessage(e) {
      e.preventDefault();
      const input = document.getElementById('msgInput');
      const text = input.value.trim();
      if (!text) return;
      socket.emit('send_message', { username: getUsername(), room: currentRoom, text });
      input.value = '';
    }

    function emitTyping() {
      socket.emit('typing', { username: getUsername(), room: currentRoom });
    }

    socket.on('receive_message', (msg) => {
      const area = document.getElementById('messagesArea');
      const isMine = msg.sender === getUsername();
      const div = document.createElement('div');
      div.className = `msg ${isMine ? 'msg-mine' : 'msg-incoming'}`;
      div.innerHTML = `<div class="msg-sender">${msg.sender} &bull; ${msg.time}</div><div>${msg.text}
</div>`;
      area.appendChild(div);
      area.scrollTop = area.scrollHeight;
    });

    socket.on('system_message', (data) => {
      const area = document.getElementById('messagesArea');
      const div = document.createElement('div');
      div.className = 'msg-system';
      div.textContent = `${data.text} (${data.time})`;
      area.appendChild(div);
      area.scrollTop = area.scrollHeight;
    });

    socket.on('room_history', (history) => {
      const area = document.getElementById('messagesArea');
      area.innerHTML = '';
      history.forEach(msg => {
        const isMine = msg.sender === getUsername();
        const div = document.createElement('div');
        div.className = `msg ${isMine ? 'msg-mine' : 'msg-incoming'}`;
        div.innerHTML = `<div class="msg-sender">${msg.sender} &bull; ${msg.time}</div><div>${msg.text}
</div>`;
        area.appendChild(div);
      });
      area.scrollTop = area.scrollHeight;
    });

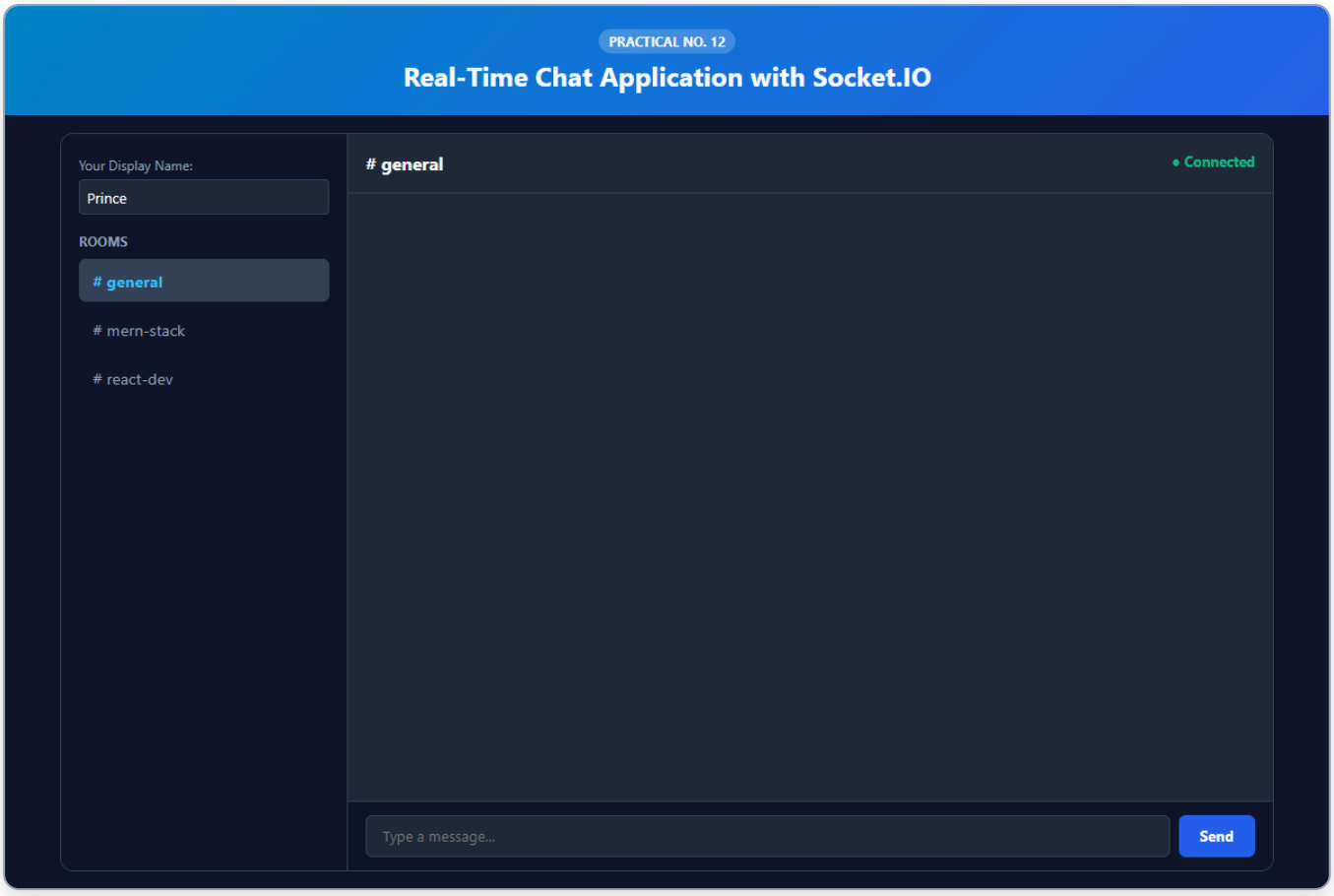
    socket.on('user_typing', (user) => {
      const indicator = document.getElementById('typingIndicator');
      indicator.textContent = `${user} is typing...`;
      clearTimeout(typingTimeout);
      typingTimeout = setTimeout(() => { indicator.textContent = ''; }, 1500);
    });

    window.addEventListener('DOMContentLoaded', () => {
      switchRoom('general');
    });
  </script>
```

```
</body>
</html>
```

4. Output :

Bidirectional real-time WebSocket communication utilizing Socket.IO, multi-room chat partitioning, active message history, and live typing indicators.



Practical – 13 : Mini Project: Design and develop a full-stack industry-oriented MERN application (Document Management System).

1. Commands :

```
mkdir MERN_13 && cd MERN_13
npm init -y
npm install express mongoose multer cors dotenv
node server.js
```

2. Project structure :

```
MERN_13/
|-- .env
|-- package.json
|-- server.js
|-- models/Document.js
|-- public/index.html
```

3. Source Code :

- **.env :**

```
PORT=5013
MONGO_URI=mongodb+srv://prince:prince18@cluster0.76xckkx.mongodb.net/mern13_dms_db?
retryWrites=true&w=majority
```

- **package.json :**

```
{
  "name": "mern-13",
  "version": "1.0.0",
  "description": "Practical 13 Mini Project: Industry-Oriented Document Management System (DMS)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.6.1",
    "multer": "^1.4.5-lts.1"
  }
}
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const multer = require('multer');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
const Document = require('./models/Document');

const app = express();
const PORT = process.env.PORT || 5013;
const uploadDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });

// Multer Storage
```

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => cb(null, `${Date.now()}-${file.originalname.replace(/\s+/g, '_')}`)
});
const upload = multer({ storage, limits: { fileSize: 10 * 1024 * 1024 } });

// DB Connection
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('✅ MongoDB Connected to DMS'))
  .catch(err => console.error('DB Error:', err.message));

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));
app.use('/uploads', express.static(uploadDir));

// Upload Document
app.post('/api/docs/upload', upload.single('file'), async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ success: false, message: 'No file uploaded' });
    const { title, category, version, uploadedBy } = req.body;

    const doc = await Document.create({
      title: title || req.file.originalname,
      category: category || 'Engineering',
      version: version || '1.0',
      originalName: req.file.originalname,
      storedName: req.file.filename,
      fileSize: req.file.size,
      storagePath: req.file.path,
      uploadedBy: uploadedBy || 'Prince Prajapati'
    });

    res.status(201).json({ success: true, message: 'Document archived successfully', data: doc });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// List Documents with Search & Category Filter
app.get('/api/docs', async (req, res) => {
  try {
    const { category, search } = req.query;
    const query = {};
    if (category && category !== 'All') query.category = category;
    if (search) query.title = { $regex: search, $options: 'i' };

    const docs = await Document.find(query).sort({ createdAt: -1 });
    res.json({ success: true, count: docs.length, data: docs });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Download Document
app.get('/api/docs/download/:id', async (req, res) => {
  try {
    const doc = await Document.findById(req.params.id);
    if (!doc || !fs.existsSync(doc.storagePath)) return res.status(404).json({ success: false, message:
'File not found' });
    res.download(doc.storagePath, doc.originalName);
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Delete Document
app.delete('/api/docs/:id', async (req, res) => {
  try {
    const doc = await Document.findById(req.params.id);
```

```
    if (!doc) return res.status(404).json({ success: false, message: 'Document not found' });
    if (fs.existsSync(doc.storagePath)) fs.unlinkSync(doc.storagePath);
    await Document.findByIdAndDelete(req.params.id);
    res.json({ success: true, message: 'Document deleted successfully' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// Analytics Overview
app.get('/api/docs/analytics', async (req, res) => {
  try {
    const docs = await Document.find();
    const totalDocs = docs.length;
    const totalBytes = docs.reduce((acc, d) => acc + d.fileSize, 0);
    const formattedStorage = (totalBytes / (1024 * 1024)).toFixed(2) + ' MB';
    res.json({ success: true, totalDocs, formattedStorage });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

app.listen(PORT, () => console.log(`⚛️ Practical 13 (Document Management System) running on port
${PORT}`));
```

- **models/Document.js :**

```
const mongoose = require('mongoose');

const documentSchema = new mongoose.Schema({
  title: { type: String, required: true },
  category: { type: String, enum: ['Engineering', 'Legal', 'HR', 'Finance'], default: 'Engineering' },
  version: { type: String, default: '1.0' },
  originalName: { type: String, required: true },
  storedName: { type: String, required: true },
  fileSize: { type: Number, required: true },
  formattedSize: { type: String },
  storagePath: { type: String, required: true },
  uploadedBy: { type: String, default: 'Prince Prajapati' }
}, { timestamps: true });

documentSchema.pre('save', function (next) {
  if (this.fileSize < 1024) this.formattedSize = this.fileSize + ' B';
  else if (this.fileSize < 1024 * 1024) this.formattedSize = (this.fileSize / 1024).toFixed(1) + ' KB';
  else this.formattedSize = (this.fileSize / (1024 * 1024)).toFixed(2) + ' MB';
  next();
});

module.exports = mongoose.model('Document', documentSchema);
```

- **public/index.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Practical 13 - Industry MERN Mini Project: Document Management System</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: system-ui, sans-serif; }
    body { background: #0b1120; color: #f8fafc; padding-bottom: 2rem; }
    .header { background: linear-gradient(135deg, #0f766e, #0e7490, #1d4ed8); padding: 2rem; text-align:
center; }
    .badge { display: inline-block; background: rgba(255,255,255,0.2); padding: 0.3rem 0.8rem; border-
radius: 999px; font-size: 0.8rem; font-weight: bold; }
    .container { max-width: 1150px; margin: 1.5rem auto; padding: 0 1rem; }

    .stats-row { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; margin-bottom: 1.5rem;
}
```

```
    .stat-box { background: #1e293b; padding: 1.25rem; border-radius: 10px; border: 1px solid #334155; }
    .stat-num { font-size: 1.6rem; font-weight: bold; color: #38bdf8; }
    .stat-title { font-size: 0.8rem; color: #94a3b8; }

    .main-grid { display: grid; grid-template-columns: 360px 1fr; gap: 1.5rem; }
    @media (max-width: 900px) { .main-grid { grid-template-columns: 1fr; } }

    .card { background: #1e293b; border-radius: 10px; padding: 1.25rem; border: 1px solid #334155; }
    .card h3 { margin-bottom: 1rem; font-size: 1.1rem; }

    input, select { width: 100%; background: #0b1120; border: 1px solid #334155; color: white; padding:
0.6rem; border-radius: 6px; margin-bottom: 0.75rem; font-size: 0.85rem; }
    .btn { background: #0ea5e9; color: white; border: none; padding: 0.6rem 1rem; border-radius: 6px;
cursor: pointer; font-weight: bold; width: 100%; }
    .btn:hover { background: #0284c7; }
    .btn-sm { width: auto; padding: 0.3rem 0.6rem; font-size: 0.78rem; text-decoration: none; display:
inline-block; }
    .btn-del { background: #ef4444; }

    table { width: 100%; border-collapse: collapse; font-size: 0.85rem; }
    th { background: #0b1120; padding: 0.7rem; text-align: left; color: #94a3b8; border-bottom: 1px solid
#334155; }
    td { padding: 0.7rem; border-bottom: 1px solid #334155; vertical-align: middle; }
    .tag { display: inline-block; padding: 0.15rem 0.5rem; border-radius: 4px; font-size: 0.75rem; font-
weight: bold; background: #334155; }
  </style>
</head>
<body>
  <div class="header">
    <span class="badge">PRACTICAL NO. 13 &bull; MINI PROJECT</span>
    <h1 style="margin: 0.4rem 0;">Industry Document Management System (DMS)</h1>
    <p style="color: #cbd5e1; font-size: 0.95rem;">Full-Stack Enterprise MERN Application with File
Archival & Version Control</p>
  </div>

  <div class="container">
    <div class="stats-row">
      <div class="stat-box">
        <div class="stat-num" id="statCount">0</div>
        <div class="stat-title">Archived Documents</div>
      </div>
      <div class="stat-box">
        <div class="stat-num" id="statStorage">0.00 MB</div>
        <div class="stat-title">Cloud Storage Used</div>
      </div>
      <div class="stat-box">
        <div class="stat-num">Enterprise</div>
        <div class="stat-title">Access Policy Tier</div>
      </div>
    </div>

    <div class="main-grid">
      <!-- Upload Card -->
      <div class="card">
        <h3>📤 Upload & Archive Document</h3>
        <form id="uploadForm" onsubmit="uploadDoc(event)">
          <label style="font-size: 0.78rem; color: #94a3b8;">Document Title</label>
          <input type="text" id="docTitle" placeholder="e.g. Master Service Agreement 2026" required>

          <label style="font-size: 0.78rem; color: #94a3b8;">Department / Category</label>
          <select id="docCat">
            <option value="Engineering">Engineering</option>
            <option value="Legal">Legal</option>
            <option value="HR">HR</option>
            <option value="Finance">Finance</option>
          </select>

          <label style="font-size: 0.78rem; color: #94a3b8;">Version</label>
          <input type="text" id="docVer" value="v1.0" required>
```

```
          <label style="font-size: 0.78rem; color: #94a3b8;">Select File (PDF, DOCX, IMG)</label>
          <input type="file" id="docFile" required>

          <button type="submit" class="btn">Archive to Cloud</button>
        </form>
      </div>

      <!-- Explorer Card -->
      <div class="card">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom:
1rem;">
          <h3>📁 Document Explorer</h3>
          <input type="text" id="searchBox" placeholder="Search documents..." oninput="loadDocs()"
style="width: 220px; margin-bottom: 0;">
        </div>

        <div style="overflow-x: auto;">
          <table>
            <thead>
              <tr>
                <th>Title & Version</th>
                <th>Category</th>
                <th>Size</th>
                <th style="text-align: right;">Actions</th>
              </tr>
            </thead>
            <tbody id="docsTable">
              <tr><td colspan="4" style="text-align: center; padding: 2rem;">Loading documents...</td>
</tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

  <script>
    async function loadDocs() {
      const q = document.getElementById('searchBox').value.trim();
      const res = await fetch(`/api/docs?search=${encodeURIComponent(q)}`);
      const json = await res.json();
      const tbody = document.getElementById('docsTable');

      if (!json.data || json.data.length === 0) {
        tbody.innerHTML = '<tr><td colspan="4" style="text-align: center; padding: 2rem; color:
#94a3b8;">No documents found.</td></tr>';
        return;
      }

      tbody.innerHTML = json.data.map(d => `
        <tr>
          <td>
            <div style="font-weight: bold; color: #f8fafc;">${d.title}</div>
            <div style="font-size: 0.75rem; color: #94a3b8;">${d.originalName} &bull; <span
class="tag">${d.version}</span></div>
          </td>
          <td><span class="tag" style="background: #0369a1;">${d.category}</span></td>
          <td style="color: #38bdf8;">${d.formattedSize || (d.fileSize + ' B')}</td>
          <td style="text-align: right; white-space: nowrap;">
            <a href="/api/docs/download/${d._id}" class="btn btn-sm" target="_blank">⬇️ Download</a>
            <button class="btn btn-sm btn-del" onclick="deleteDoc('${d._id}')">🗑️</button>
          </td>
        </tr>
      `).join('');

      // Refresh analytics
      const anaRes = await fetch('/api/docs/analytics');
      const anaJson = await anaRes.json();
      if (anaJson.success) {
        document.getElementById('statCount').textContent = anaJson.totalDocs;
```

```
        document.getElementById('statStorage').textContent = anaJson.formattedStorage;
      }
    }

    async function uploadDoc(e) {
      e.preventDefault();
      const fileInput = document.getElementById('docFile');
      if (!fileInput.files[0]) return alert('Please select a file');

      const formData = new FormData();
      formData.append('title', document.getElementById('docTitle').value);
      formData.append('category', document.getElementById('docCat').value);
      formData.append('version', document.getElementById('docVer').value);
      formData.append('file', fileInput.files[0]);

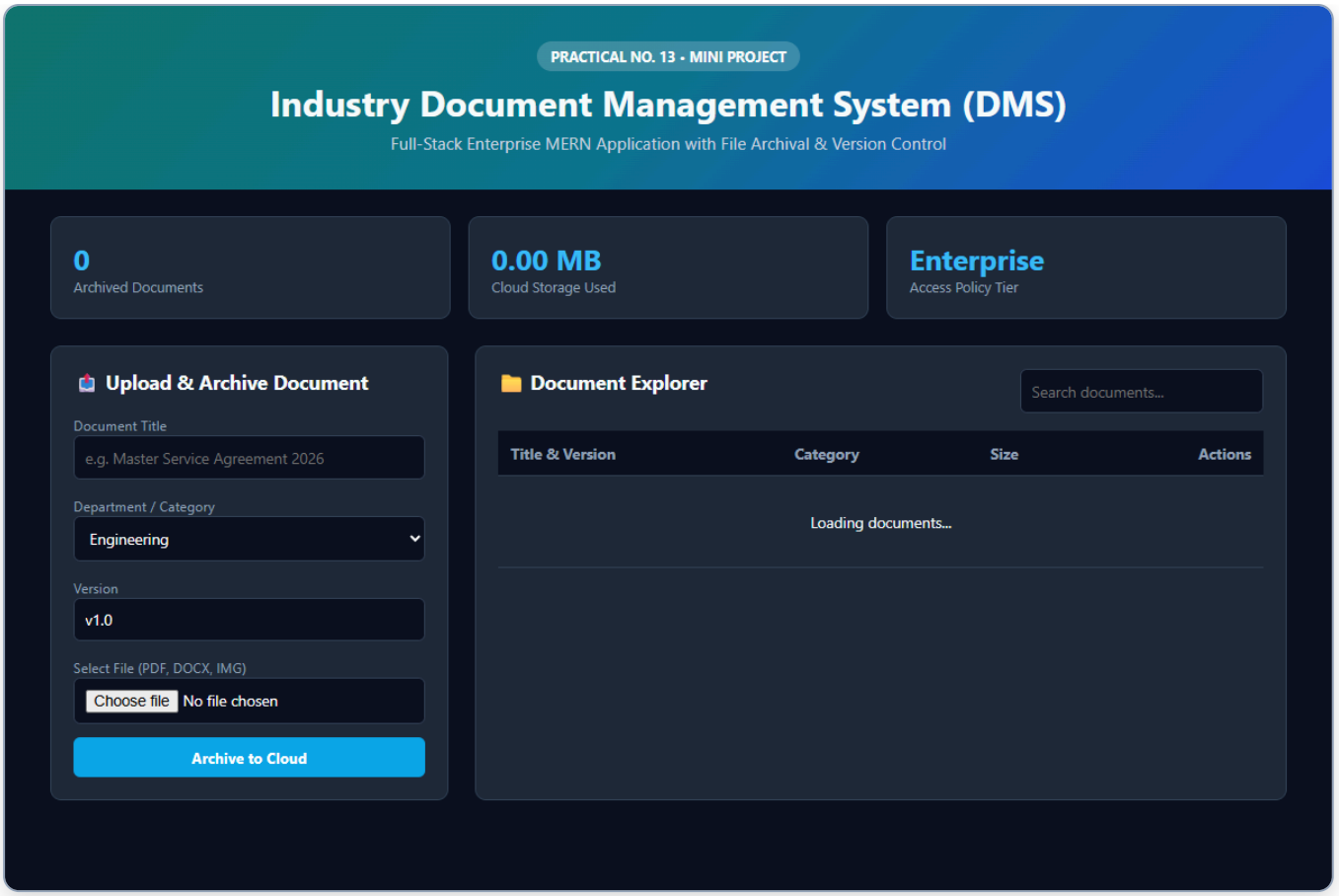
      try {
        const res = await fetch('/api/docs/upload', { method: 'POST', body: formData });
        const data = await res.json();
        if (data.success) {
          document.getElementById('uploadForm').reset();
          document.getElementById('docVer').value = 'v1.0';
          loadDocs();
        } else {
          alert(data.message);
        }
      } catch (err) {
        alert(err.message);
      }
    }

    async function deleteDoc(id) {
      if (!confirm('Are you sure you want to delete this document?')) return;
      await fetch(`/api/docs/${id}`, { method: 'DELETE' });
      loadDocs();
    }

    window.addEventListener('DOMContentLoaded', loadDocs);
  </script>
</body>
</html>
```

4. Output :

Industry-oriented capstone Document Management System (DMS) featuring file categorization, automated version stamping, cloud metadata metrics, and document search/download.


PRACTICAL NO. 13 • MINI PROJECT
Industry Document Management System (DMS)
Full-Stack Enterprise MERN Application with File Archival & Version Control
0
Archived Documents
0.00 MB
Cloud Storage Used
Enterprise
Access Policy Tier
Upload & Archive Document
Document Title
e.g. Master Service Agreement 2026
Department / Category
Engineering
Version
v1.0
Select File (PDF, DOCX, IMG)
Choose file
No file chosen
Archive to Cloud
Document Explorer
Search documents...
Title & Version
Category
Size
Actions
Loading documents...
